

Αριστοτέλειο Πανεπιστήμιο Θεσσαλονίκης
Πολυτεχνική Σχολή
Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Τομέας Ηλεκτρονικής και Υπολογιστών

Επιτάχυνση Συναρτήσεων του λογισμικού Seissol σε FPGA

Διπλωματική Εργασία
της
Τσιτσάνου ΄Αννας

Επιβλέπων: Παπαευσταθίου Ιωάννης
Καθηγητής Α.Π.Θ.

21 Ιανουαρίου 2026

Περίληψη

Η παρούσα διπλωματική εργασία εξερευνά τη δυνατότητα επιτάχυνσης συγκεκριμένων αριθμητικών μικροπυρήνων του λογισμικού SeisSol μέσω χαρτογράφησης τους σε FPGA με τη χρήση εργαλείων υψηλού επιπέδου σύνθεσης (HLS). Αρχικά, πραγματοποιείται μία ανάλυση απόδοσης του λογισμικού σε CPU με τη βοήθεια του Intel VTune Profiler, για τον εντοπισμό των κυριότερων υπολογιστικών hotspots. Έπειτα, τρεις LIBXSMM αριθμητικοί πυρήνες και ένας πυρήνας υπολογισμού ανάπτυξης Taylor, μετατρέπονται σε κατάλληλες C/C++ εκδοχές και υλοποιούνται σε FPGA μέσω του Vitis HLS.

Η εργασία εστιάζει σε τεχνικές αρχιτεκτονικής βελτιστοποίησης, όπως η χρήση τοπικών buffers, η απομάκρυνση εξαρτήσεων μεταξύ επαναλήψεων, το loop pipelining, το επιλεκτικό unrolling και η αξιοποίηση διευρυμένων datapaths. Οι τελικές υλοποιήσεις υλικού αξιολογούνται τόσο ως προς τον χρόνο εκτέλεσης όσο και ως προς τη χρήση πόρων, με τη βοήθεια των εργαλείων ανάλυσης του Vitis.

Τα πειραματικά αποτελέσματα δείχνουν ότι, παρ΄ όλο που οι FPGA υλοποιήσεις επιτυγχάνουν μείωση του Initiation Interval και ικανοποιητική χρήση των διαθέσιμων πόρων του FPGA, λαμβάνοντας υπόψη τη φύση των συναρτήσεων, δεν υπερβαίνουν τις αντίστοιχες υλοποιήσεις λογισμικού. Το εύρημα αυτό συνδέεται με το μικρό μέγεθος και την ήδη υψηλή βελτιστοποίηση των εξεταζόμενων kernels, καθώς και στο σταθερό κόστος εκκίνησης και επικοινωνίας με τον επιταχυντή. Συνολικά, η εργασία αναδεικνύει τα όρια της επιτάχυνσης μέσω FPGA για μικρού μεγέθους αριθμητικούς πυρήνες και υπογραμμίζει τη σημασία της αρχιτεκτονικής ανάλυσης κατά την επιλογή επιταχυντών σε εφαρμογές Υπολογιστικής Υψηλών Επιδόσεων.

Abstract

This thesis explores the potential of speeding up specific numerical microkernels of the SeisSol software by mapping them onto an FPGA using high-level synthesis tools. First, a performance analysis of the CPU-based implementation is carried out with the Intel VTune Profiler to spot the main computational hotspots. Then, three LIBXSMM-based numerical kernels and a Taylor expansion kernel are converted to appropriate C/C++ versions and implemented on an FPGA using Vitis HLS.

The work focuses on architectural optimization methods such as on-chip buffering, eliminating loop-carried dependencies, loop pipelining, selective unrolling, and using widened data paths. The final hardware implementations are assessed based on execution time and resource use through the analysis tools in the Vitis environment.

Experimental results show that, even though the FPGA implementations achieve low initiation intervals and satisfactory overall resource use, given the nature of the functions, they do not surpass their CPU equivalents. This outcome is linked to the small size and high level of optimization of the kernels examined, as well as the fixed overhead that comes with kernel invocation and data transfers. Overall, the thesis highlights the challenges of FPGA-based acceleration for small-scale numerical kernels and stresses the need for architectural analysis when choosing accelerators for high-performance computing applications.

Ευχαριστίες

Θα ήθελα να εκφράσω τις ειλικρινείς και θερμές μου ευχαριστίες στον επιβλέποντα καθηγητή μου, κ. Ιωάννη Παπαευσταθίου, για την εμπιστοσύνη, την καθοδήγηση και τη συνεχή υποστήριξή του σε όλη τη διάρκεια εκπόνησης της παρούσας διπλωματικής εργασίας.

Ιδιαίτερες ευχαριστίες απευθύνω στον υποψήφιο διδάκτορα ΄Αγγελο Αθανασιάδη, του οποίου η συνεχής διαθεσιμότητα και προθυμία για παροχή βοήθειας, σε συνδυασμό με την τεχνική του κατάρτιση και τις εύστοχες παρατηρήσεις του, υπήρξαν καθοριστικές σε κρίσιμα στάδια της παρούσας εργασίας.

Θα ήθελα επίσης να ευχαριστήσω τους φίλους και συμφοιτητές μου για τη στήριξη και την ενθάρρυνσή τους κατά τη διάρκεια των σπουδών μου.

Τέλος, εκφράζω τη βαθιά μου ευγνωμοσύνη στην οικογένειά μου, για τη διαρκή υποστήριξη, την κατανόηση και την πίστη τους σε εμένα.

Επιτάχυνση Συναρτήσεων του λογισμικού Seissol σε FPGA

Τσιτσάνου ΄Αννα
tsitsanou@ece.auth.gr

21 Ιανουαρίου 2026

Περιεχόμενα

1 Εισαγωγή **7**
1.1 Αντικείμενο και κίνητρο της εργασίας 7
1.2 Εισαγωγή στο SeisSol 7
1.3 Δομή της Διπλωματικής 8

2 Θεωρητικό Υπόβαθρο **10**
2.1 Υπολογιστικές Εφαρμογές Υψηλών Επιδόσεων 10
2.1.1 Εισαγωγή στις Εφαρμογές Υψηλών Επιδόσεων 10
2.1.2 Αρχιτεκτονική και Λειτουργία Συστημάτων HPC 11
2.1.3 Ο Ρόλος του HPC στις γεωεπιστήμες 11
2.2 Προηγούμενες έρευνες για το SeisSol 12
2.2.1 Αρχικές Βελτιστοποίηση σε CPU Αρχιτεκτονικές 12
2.2.2 Βελτιστοποίηση Εισόδου/Εξόδου (I/O) 12
2.2.3 Επιτάχυνση με Επιταχυντές GPU 13
2.3 Το λογισμικό Intel VTune Profiler 14
2.4 FPGA Αρχιτεκτονικές και Σχεδίαση με Vitis HLS και Vitis 14
2.4.1 Είσαγωγή στα FPGAs 14
2.4.2 Τα προγράμματα σχεδίασης Vitis και VitisHLS 16

3 Εγκατάσταση του SeisSol και χρονική ανάλυση επιδόσεων **18**
3.1 Εγκατάσταση του SeisSol και Επιλογή Παραδείγματος 18
3.2 Χρονική ανάλυση στο Vtune 19
3.3 Περιγραφή των συναρτήσεων 20
3.3.1 Γενική επισκόπηση των hotspots 20
3.3.2 Συνάρτηση derivativeTaylorExpansion::execute() 21
3.3.3 Assembly-based μικροπυρήνες πολλαπλασιασμού πινάκων (LIBXSMM kernels) 21
3.3.3.1 LIBXSMM kernel m12_n9_k20 22
3.3.3.2 LIBXSMM kernel m12_n9_k9 23
3.3.3.3 LIBXSMM kernel m20_n9_k10 24

4 Υλοποίηση **27**
4.1 Μεθοδολογία χαρτογράφησης των kernels σε FPGA 27
4.1.1 Υπολογιστική πλατφόρμα 27
4.1.2 Συνολική ροή ανάπτυξης 28
4.1.3 Προκλήσεις σχεδιασμού 28
4.2 Μετατροπή kernels και έλεγχος ορθότητας 28
4.2.1 Εντοπισμός των kernels μέσω VTune 29

4.2.2 Μετατροπή από AVX2 inline assembly σε C++ 29
4.2.3 Οργάνωση και υλοποίηση του testbench 29
4.2.4 Αποτελέσματα επαλήθευσης 30
4.3 Δημιουργία και ανάλυση project στο Vitis HLS 30
4.3.1 Δημιουργία και ρύθμιση του αρχικού project 30
4.3.2 Οργάνωση του testbench και δήλωση των διεπαφών 31
4.3.2.0.1 Δήλωση διεπαφών AXI 31
4.3.3 Αρχική σύνθεση των kernels και ανάλυση αποτελεσμάτων 32
4.3.3.1 Kernel libxsmm_m12_n9_k20 32
4.3.3.2 Kernel libxsmm_m12_n9_k9 32
4.3.3.3 Kernel libxsmm_m20_n9_k10 32
4.3.4 Συμπεράσματα αρχικής αξιολόγησης 33
4.4 Αντιμετώπιση των violations και βελτιστοποιήσεις 33
4.4.1 Βασικά HLS pragmas και η λειτουργία τους 33
4.4.2 Αντιμετώπιση των αρχικών violations 36
4.4.2.1 Πρώτη προσπάθεια: ενοποίηση των accumulators 37
4.4.2.2 Δεύτερη προσπάθεια: επανασχεδιασμός του εσωτερικού loop 37
4.4.2.3 Ανεπιτυχείς προσπάθειες: ping–pong accumulators 38
4.4.2.4 Καθοριστική λύση: read/write accumulators 38
4.4.2.5 Παράλληλη βελτιστοποίηση των memory accesses 39
4.4.3 Διερεύνηση επιπλέον τεχνικών και συμπεριφοράς των kernels 39
4.4.3.1 Αύξηση των διαστάσεων των πινάκων 39
4.4.3.2 Μεταβολή της χρήσης πόρων 40
4.4.3.3 Προσπάθεια γενίκευσης της συνάρτησης 40
4.4.3.4 Επανάληψη του kernel για μεγάλο αριθμό εκτελέσεων . 41
4.4.4 Χρήση Streams και Dataflow για Pipeline σε επίπεδο Kernel 41
4.4.4.1 Πρώτη εκδοχή: απλό dataflow pipeline τριών σταδίων 41
4.4.4.2 Δεύτερη εκδοχή: dataflow ανά chunk 42
4.4.4.3 Τρίτη εκδοχή: ξεχωριστά streams A και B με διαφορετικά AXI bundles 43
4.4.5 Βελτιστοποίηση μέσω χρήσης δεδομένων 512-bit (uint512_t) 44
4.4.5.1 Αναπροσαρμογή διαστάσεων και βοηθητικές συναρτήσεις 44
4.4.5.2 Πρώτη εκδοχή με uint512_t: packing και stream-based dataflow 45
4.4.5.3 Δεύτερη εκδοχή: διόρθωση της εξόδου και πλήρης συσχέτιση των accumulators 45
4.4.5.4 Περιορισμοί: αναποτελεσματική εκμετάλλευση των 512-bit vectors 46
4.4.5.5 Δοκιμή χρήσης float αντί για double 47
4.4.5.6 Συμπέρασμα και σύνδεση με την τελική βελτιστοποίηση . 47
4.4.6 Τελική βελτιστοποίηση των LIBXSMM πυρήνων 47
4.4.6.1 Πυρήνας m12_n9_k9: αναλυτική τελική αρχιτεκτονική . 48
4.4.6.2 Προσαρμογή της αρχιτεκτονικής στον πυρήνα m12_n9_k20 50
4.4.6.3 Πυρήνας m20_n9_k10: επέκταση σε μεγαλύτερη διάσταση m 51
4.5 Βελτιστοποίηση και Υλοποίηση του πυρήνα derivativeTaylorExpansion . 52
4.5.1 Αρχική Μεταφορά στο Vitis HLS και Προβλήματα II 52

4.5.2 Version 2: Υλοποίηση με Τοπικά Buffers και Ενιαίο Compute Loop 53
4.5.3 Version 3: Απόπειρα με hls::stream και DATAFLOW 54
4.5.4 Τελική Επιλογή: Η Βέλτιστη Buffer-Based Υλοποίηση 54
4.5.5 Συνολική Αξιολόγηση 55
4.6 Ενσωμάτωση των πυρήνων στο Vitis και εκτέλεση στο FPGA 55
4.6.1 Δημιουργία hardware kernel και αρχείου xclbin 55
4.6.2 Host πρόγραμμα και διαχείριση μνήμης 56
4.6.3 Πρώτες εκτελέσεις στο FPGA και επιβεβαίωση ορθότητας 57

5 Αποτελέσματα και Αξιολόγηση Επιδόσεων 59
5.1 Χρονικά αποτελέσματα software και hardware υλοποιήσεων 59
5.1.1 Μεθοδολογία μετρήσεων 59
5.1.2 Original συναρτήσεις 60
5.1.3 Πρώτες εκδοχές χωρίς violations (εκτελέσεις x1 και x1000 φορές) 60
5.1.4 Εκδοχές με `uint512` και dataflow (εκτελέσεις x1 και x1000 φορές) 60
5.1.5 Τελικές εκδοχές: επίδραση compiler optimizations στο hardware (×1000) 61
5.1.5.1 Function 1 (×1000) 61
5.1.5.2 Function 2 (×1000) 61
5.1.5.3 Function 3 (×1000) 61
5.1.6 Software timings εκτός Vitis: επίδραση optimizations 61
5.2 Αποτελέσματα σε επίπεδο συστήματος: Vitis project με τρεις kernels 62
5.2.1 Έκδοση `uint512` + dataflow: launch από ίδια/διαφορετικές ουρές (queues) 62
5.2.2 Τελικές εκδοχές project 3 kernels (×1000): optimizations O0/O1/O2 62
5.3 Συνάρτηση Taylor (Function 4): εκδοχές και optimizations 63
5.4 Συζήτηση Χρονισμών και Ανάλυση Απόδοσης 63
5.5 Ανάλυση τελικών εκδοχών μέσω Vitis Analyzer 64
5.5.1 Kernel 1 (libxsmm_m12_n9_k20) 64
5.5.2 Kernel 2 (libxsmm_m12_n9_k20) 66
5.5.3 Kernel 3 (libxsmm_m20_n9_k10) 70
5.5.4 Συνολικό project με τρεις kernels 72
5.5.5 Taylor kernel (derivativeTaylorExpansion) 75

6 Συμπεράσματα και Μελλοντική Εργασία 79
6.1 Συμπεράσματα 79
6.2 Προτάσεις για Μελλοντικές Βελτιστοποιήσεις 80

Α′ Ακρωνύμια και συντομογραφίες 81

Κατάλογος Σχημάτων

2.1 Τομείς χρήσεις του High Performance Computing 10
2.2 Cluster Αρχιτεκτονική του High Performance Computing 11
2.3 Η βασική δομή ενός FPGA 15
2.4 Η θέση των εργαλείων Vitis στη σχεδίαση 17

3.1 Αποτελέσματα της ανάλυσης Hotspots στο Intel Vtune Profiler 20

4.1 Εφαρμογή pipeline σε βρόχο 34
4.2 Εφαρμογή πλήρους unroll σε βρόχο 35
4.3 Χρήση της οδηγίας dataflow 36

5.1 System diagram για τον Kernel 1 από το Vitis Analyzer 65
5.2 Χρήση πόρων accelerator για τον Kernel 1: απόλυτες τιμές 65
5.3 Χρήση πόρων accelerator για τον Kernel 1: ποσοστά επί του User Budget 65
5.4 Συνολική χρήση πόρων συσκευής για τον Kernel 1 66
5.5 Post-placement SLR connectivity για τον Kernel 1 67
5.6 System diagram για τον Kernel 2 από το Vitis Analyzer 67
5.7 Χρήση πόρων accelerator για τον Kernel 2: απόλυτες τιμές 68
5.8 Χρήση πόρων accelerator για τον Kernel 2: ποσοστά επί του User Budget 68
5.9 Συνολική χρήση πόρων συσκευής για τον Kernel 2 69
5.10 Post-placement SLR connectivity για τον Kernel 2 69
5.11 System diagram για τον Kernel 3 από το Vitis Analyzer 70
5.12 Χρήση πόρων accelerator για τον Kernel 3: απόλυτες τιμές 70
5.13 Χρήση πόρων accelerator για τον Kernel 3: ποσοστά επί του User Budget 71
5.14 Συνολική χρήση πόρων συσκευής για τον Kernel 3 71
5.15 Post-placement SLR connectivity για τον Kernel 3 72
5.16 System diagram για το project με τους 3 Kernels από το Vitis Analyzer 73
5.17 Χρήση πόρων accelerator για το project με τους 3 Kernels: απόλυτες τιμές 73
5.18 Χρήση πόρων accelerator για το project με τους 3 Kernels: ποσοστά επί του User Budget 74
5.19 Συνολική χρήση πόρων συσκευής για το project με τους 3 Kernels 74
5.20 Post-placement SLR connectivity για το project με τους 3 Kernels 75
5.21 System diagram για τον Kernel Taylor από το Vitis Analyzer 76
5.22 Χρήση πόρων accelerator για τον Kernel Taylor: απόλυτες τιμές 76
5.23 Χρήση πόρων accelerator για τον Kernel Taylor: ποσοστά επί του User Budget 76
5.24 Συνολική χρήση πόρων συσκευής για τον Kernel Taylor 77
5.25 Post-placement SLR connectivity για τον Kernel Taylor 78

Κατάλογος Πινάκων

4.1 Αρχικά αποτελέσματα σύνθεσης των original LIBXSMM kernels στο Vitis HLS 33
4.2 Αποτελέσματα 1ης προσπάθειας βελτιστοποίησης για τον kernel m12_n9_k9 39
4.3 Αποτελέσματα dataflow υλοποίησης για τον kernel m12_n9_k9 43
4.4 Αποτελέσματα dataflow υλοποίησης για τον kernel m12_n9_k9 46
4.5 Τελικές εκδοχές των τριών kernels: επιδόσεις και εκτιμήσεις πόρων από Vitis HLS 52
4.6 Αποτελέσματα της σύνθεσης της συνάρτησης Taylor στο Vitis HLS 55
4.7 Ρυθμίσεις των compute units για τους τρεις kernels στο Vitis κατά το hardware build 56

5.1 Χρόνοι εκτέλεσης original συναρτήσεων σε software και hardware (1 εκτέλεση, ms) 60
5.2 Χρόνοι εκτέλεσης πρώτων εκδοχών χωρίς violations (1 και 1000 εκτελέσεις, ms) 60
5.3 Χρόνοι εκτέλεσης εκδοχών με `uint512` και dataflow (1 και 1000 εκτελέσεις, ms) 60
5.4 Τελικές εκδοχές Function 1 (×1000, ms) με hardware optimizations 61
5.5 Τελικές εκδοχές Function 2 (×1000, ms) με hardware optimizations 61
5.6 Τελικές εκδοχές Function 3 (×1000, ms) με hardware optimizations 61
5.7 Σοφτωαρε τιμινγς εκτός ίτισ: σύγκριση -O0 και -O3 (μς) 62
5.8 Project 3 kernels (v1): ίδια ουρά vs διαφορετικές ουρές (ms) 62
5.9 Προθεστ 3 κερνελς (φιναλ, ×1000): σύγκριση οπτιμιζατιονς (μς) 63
5.10 Function 4 (Taylor): σύγκριση εκδοχών και optimizations (ms) 63

Listings

4.1 Σύγκριση HLS και software υλοποίησης στο testbench. 31
4.2 Ενοποίηση των 9 accumulators σε έναν ενιαίο πίνακα 37
4.3 Μεταβολή βρόχου επανάληψης 37
4.4 Μετατροπή του εσωτερικού loop σε ενιαίο loop 36 επαναλήψεων 37
4.5 Read/write accumulators για άρση των RAW εξαρτήσεων 38
4.6 Αποστολή δεδομένων A και B σε streams (v1). 41
4.7 Ανάγνωση A και B κατά chunks (v2). 42
4.8 Ξεχωριστά AXI bundles για A, B και C (v3). 43
4.9 Ενοποιημένο initialization και update (v3). 43
4.10 Μετατροπείς double σε uint512_t και αντίστροφα 44
4.11 Ανάγνωση double από uint512_t. 44
4.12 Εγγραφή double σε uint512_t. 45
4.13 Κεντρικός βρόχος της πρώτης εκδοχής 45
4.14 Δεύτερη Εκδοχή: πλήρης accumulator 14 words. 46
4.15 Εγγραφή της εξόδου ως packed uint512. 46
4.16 Τοπικά buffers και αρχική φόρτωση δεδομένων για τον πυρήνα m12_n9_k9. 48
4.17 Ενημέρωση συσσωρευτών ανά στήλη j 48
4.18 Ορισμός περιοχής C που αντιστοιχεί σε κάθε chunk. 49
4.19 Ενημέρωση του C σε επίπεδο λέξης και lane για τον m12_n9_k9. 49
4.20 Ενδεικτικό mapping l σε δείκτες A και B για τον m12_n9_k20. 51
4.21 Mapping l σε A και B για τον m20_n9_k10. 51
4.22 Βρόχοι υπολογισμού της συνάρτησης derivativeTaylorExpansion 52
4.23 Ανάγνωση πινάκων σε buffers. 53
4.24 Νέο compute loop για τη συνάρτηση Taylor. 54

Κεφάλαιο 1

Εισαγωγή

1.1 Αντικείμενο και κίνητρο της εργασίας

Στον τομέα της γεωφυσικής, η κατανόηση και η πρόβλεψη φυσικών φαινομένων όπως οι σεισμοί αποτελούν μεγάλη πρόκληση, με επιπτώσεις στην ανθρώπινη ασφάλεια και στις υποδομές. Σε αυτό το πλαίσιο, οι σεισμικές προσομοιώσεις αποτελούν ένα κρίσιμο εργαλείο για την κατανόηση της διάδοσης σεισμικών κυμάτων και της δυναμικής των ρηγμάτων. Η ικανότητα πρόβλεψης και αναπαράστασης των σεισμικών γεγονότων με ακρίβεια μπορεί να συνεισφέρει σημαντικά τόσο στην επιστημονική έρευνα όσο και στην αντισεισμική προστασία και τη μηχανική ανάλυση κατασκευών.

Ωστόσο, οι σύνθετες μαθηματικές συναρτήσεις που διέπουν αυτά τα φαινόμενα απαιτούν σημαντικούς υπολογιστικούς πόρους και χρόνο για την εκτέλεσή τους, καθιστώντας απαραίτητη την αναζήτηση νέων μεθόδων επιτάχυνσης [1]. Πρόκειται για μη γραμμικά, υπερβολικά συστήματα με υψηλές χωρικές και χρονικές απαιτήσεις, συνεπώς καθίσταται απαραίτητη η χρήση υπολογιστικών συστημάτων υψηλών επιδόσεων και εξιδανικευμένων λογισμικών.

Για την αντιμετώπιση των παραπάνω προκλήσεων, αναπτύχθηκε το SeisSol, ένα από τα πλέον προηγμένα και ευρέως χρησιμοποιούμενα λογισμικά προσομοίωσης σεισμικών κυμάτων μεγάλης κλίμακας.

Η παρούσα διπλωματική εργασία επικεντρώνεται στη μελέτη και επιτάχυνση των κρίσιμων συναρτήσεων του SeisSol, μέσα από την υλοποίηση τους σε FPGA, χρησιμοποιώντας το λογισμικό Vitis, με απώτερο στόχο τη βελτίωση της αποδοτικότητας και η μείωση του απαιτούμενου χρόνου εκτέλεσης. Η επιτάχυνση λειτουργιών του Seissol σε Field-Programmable Gate Arrays μέσω του Vitis HLS αντιπροσωπεύει μια πολλά υποσχόμενη οδό για τη βελτίωση της υπολογιστικής απόδοσης και της ενεργειακής απόδοσης σε εφαρμογές υψηλής απόδοσης [2].

1.2 Εισαγωγή στο SeisSol

Το SeisSol είναι ένα λογισμικό ανοιχτού κώδικα, αναπτυγμένο στο Τεχνικό Πανεπιστήμιο του Μονάχου και στο Πανεπιστήμιο Ludwig-Maximilians. Αποτελεί ένα από τα κορυφαία εργαλεία για σενάρια σεισμών, ιδιαίτερα για την προσομοίωση δυναμικών

διαδικασιών ρήξης και προβλημάτων που απαιτούν διακριτοποίηση πολύπλοκων γεωμετριών [1]. Ειδικότερα, το SeisSol ξεχωρίζει για την ικανότητά του να μοντελοποιεί σύνθετες γεωμετρίες ρηγμάτων, να προσομοιώνει τη διάδοση υψηλών συχνοτήτων και να ενσωματώνει ρεαλιστικά, ετερογενή γεωλογικά μοντέλα [3].

Η αριθμητική του μέθοδος βασίζεται στον συνδυασμό της μεθόδου Discontinuous Galerkin(DG) για τη χωρική διακριτοποίηση και του σχήματος Arbitrary High Order DERivatives(ADER) για τη χρονική ολοκλήρωση [1]. Η μέθοδος DG είναι ιδιαίτερα σημαντική για την επίλυση υπερβολικών μερικών διαφορικών εξισώσεων, καθώς επιτρέπει τον αποτελεσματικό χειρισμό σύνθετων γεωμετριών και ασυνεχειών [3]. Το σχήμα ADER, βασισμένο στην επίλυση Riemann προβλημάτων, εξασφαλίζει υψηλή τάξη ακρίβειας σε όλες τις διαστάσεις, προσφέροντας ένα ρητό,σχήμα ολοκλήρωσης ενός βήματος [3]. Λόγω του υψηλού βαθμού τοπικότητας της, η μέθοδος ADER-DG επιδεικνύει σχεδόν βέλτιστες επιδόσεις (speedups) σε αρχιτεκτονικές υπερυπολογιστών [4] [1].

Ωστόσο, αυτές οι επιλογές μεθοδολογίας, ενώ προσφέρουν εξαιρετική ακρίβεια και ευελιξία, ταυτόχρονα παρουσιάζουν σημαντικές υπολογιστικές απαιτήσεις. Από πλευράς υπολογιστικής υλοποίησης, το SeisSol είναι σχεδιασμένο για πολυπύρηνους και πολυκόμβους επεξεργαστές, έχοντας αρχικά σχεδιαστεί και βελτιστοποιηθεί για παραδοσιακές πλατφόρμες x86 CPU πολλαπλών και πολλών πυρήνων [4]. Χρησιμοποιεί παράλληλη εκτέλεση MPI + OpenMP, γεγονός που του επιτρέπει να κλιμακώνεται σε χιλιάδες υπολογιστικούς κόμβους χωρίς σημαντική απώλεια απόδοσης [4]. Επίσης, αξιοποιεί βιβλιοθήκες βελτιστοποιημένων συναρτήσεων, όπως το LIBXSMM για τις πράξεις μικρών πολλαπλασιασμών πινάκων (small dense matrix multiplications), οι οποίες είναι κρίσιμες για την επίτευξη υψηλής απόδοσης [4]. Το μεγαλύτερο ποσοστό του χρόνου εκτέλεσης του SeisSol δαπανάται σε αυτές τις εσωτερικές "kernels" ρουτίνες του αλγορίθμου, οι οποίες αποτελούν τα κύρια σημεία απόδοσης (hotspots) και επηρεάζουν καθοριστικά τη συνολική απόδοση [1]. Αυτό θα αποτελέσει και το κύριο αντικείμενο της προσπάθειας επιτάχυνσης σε αυτή την εργασία.

1.3 Δομή της Διπλωματικής

Η παρούσα διπλωματική εργασία είναι δομημένη σε έξι βασικά κεφάλαια. Αρχικά, παρουσιάζεται το θεωρητικό υπόβαθρο και η ανάλυση επιδόσεων του λογισμικού SeisSol, ενώ συνεχίζει στην πρακτική υλοποίηση και στις βελτιστοποιήσεις του υλικού. Τέλος, γίνεται η αξιολόγηση των αποτελεσμάτων και η έκθεση των συμπερασμάτων. Πιο συγκεκριμένα, η εργασία διαρθρώνεται ως εξής:

- Στο **Κεφάλαιο 2** παρουσιάζονται οι θεμελιώδεις έννοιες που είναι απαραίτητες για την κατανόηση της εργασίας. Αρχικά, γίνεται μια εισαγωγή στις Εφαρμογές Υψηλών Επιδόσεων. Στη συνέχεια, εξετάζονται οι προηγούμενες έρευνες σχετικά με το SeisSol και οι βελτιστοποιήσεις του. Παρέχεται, επίσης, μια αναλυτική εισαγωγή στο εργαλείο Intel VTune Profiler για την ανάλυση επιδόσεων, καθώς και μια εκτενή επισκόπηση των FPGAs και της σχεδίασης υλικού με Vitis HLS και Vitis.

- Το **Κεφάλαιο 3** εστιάζει στην πρακτική προσέγγιση της εργασίας. Περιγράφεται η διαδικασία εκτέλεσης ενός παραδείγματος του SeisSol και η αρχική ανάλυση του χρόνου εκτέλεσης. Ακολουθεί η χρήση του VTune για τον εντοπισμό των

hotspots του κώδικα και η περιγραφή των κρίσιμων συναρτήσεων (assembly και C++) που αποτελούν τα σημεία συμφόρησης.

- Στο **Κεφάλαιο 4** αυτό αναλύεται η διαδικασία της υλοποίησης των βελτιστοποιήσεων. Ξεκινά με τη μετατροπή των συναρτήσεων assembly σε C++, προκειμένου να είναι δυνατή η περαιτέρω επεξεργασία τους. Στη συνέχεια, παρουσιάζεται η δημιουργία και ανάλυση του project στο Vitis HLS, οι μέθοδοι για την αντιμετώπιση των παραβιάσεων (violations) που προκύπτουν και οι τεχνικές βελτιστοποίησης που χρησιμοποιήθηκαν. Ολοκληρώνεται με την ενσωμάτωση των kernels στο Vitis project και την εκτέλεση στο FPGA.
- Το **Κεφάλαιο 5** παρουσιάζει και αναλύει τα πειραματικά αποτελέσματα. Συγκεκριμένα, παρουσιάζονται τα αποτελέσματα τόσο από την εκτέλεση του λογισμικού όσο και από την εκτέλεση στο FPGA.
- Το **Κεφάλαιο 6** συνοψίζει τα συμπεράσματα που προέκυψαν από την εργασία και παρέχει προτάσεις για μελλοντικές βελτιστοποιήσεις.
- Τέλος, ακολουθεί η Βιβλιογραφία με όλες τις πηγές που χρησιμοποιήθηκαν.

Κεφάλαιο 2

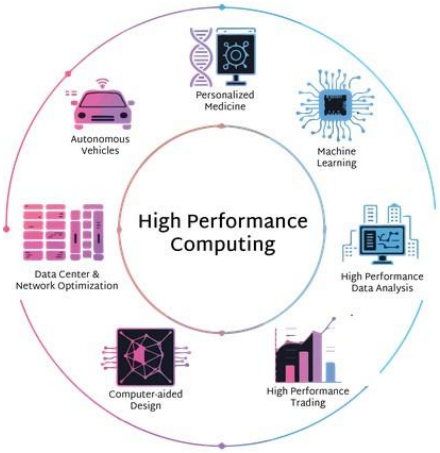
Θεωρητικό Υπόβαθρο

2.1 Υπολογιστικές Εφαρμογές Υψηλών Επιδόσεων

2.1.1 Εισαγωγή στις Εφαρμογές Υψηλών Επιδόσεων

Οι Υπολογιστικές Εφαρμογές Υψηλών Επιδόσεων (HPC) αποτελούν πλέον ένα θεμελιώδες εργαλείο στην επιστημονική έρευνα, αναγνωριζόμενες ως η ¨τρίτη επιστημονική πειθαρχία¨ μετά το πείραμα και τη θεωρία [5].

Η ικανότητά τους να εκτελούν περίπλοκους υπολογισμούς με πρωτοφανείς ταχύτητες και κλίμακα είναι Cωτικής σημασίας για την αντιμετώπιση πολυσύνθετων προβλημάτων σε πληθώρα επιστημονικών και μηχανικών πεδίων. Μέσω του HPC οι επιστήμονες μπορούν να αξιολογήσουν ποσότητες που διαφορετικά θα ήταν απρόσιτες, να βελτιώσουν πειράματα και να αποκτήσουν νέες γνώσεις για συστήματα που αναλύονται [5]. Οι σύγχρονες επιστημονικές εφαρμογές, όπως αυτές στον τομέα των Μεγάλων Δεδομένων, της Τεχνητής Νοημοσύνης, της χημείας, της φυσικής και της βιολογίας, είναι εξαιρετικά απαιτητικές σε υπολογιστικούς πόρους, καθιστώντας το HPC απαραίτητο για την πρόοδο [6].

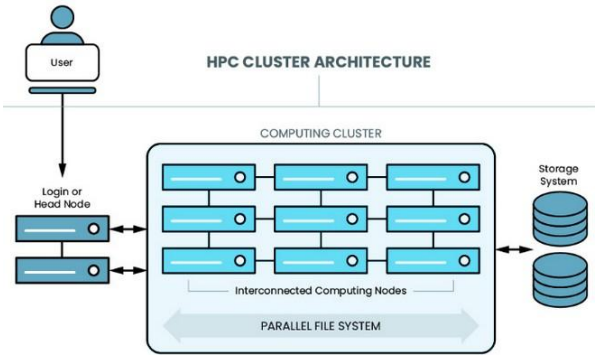


Σχήμα 2.1: Τομείς χρήσεις του High Performance Computing

2.1.2 Αρχιτεκτονική και Λειτουργία Συστημάτων HPC

Ένα σύστημα HPC αποτελείται από μια συλλογή πολλαπλών στοιχείων που συνδέονται μεταξύ τους και ελέγχεται από ένα σύστημα διαχείρισης πόρων. Οι εργασίες δεν εκτελούνται απαραίτητα σε μια συγκεκριμένη ομάδα στοιχείων, αλλά κατανέμονται σε διαφορετικές ομάδες εντός του cluster, λειτουργώντας ταυτόχρονα. Μια ομάδα πόρων, μέρος ενός συνόλου, ονομάζεται κόμβος (node), ενώ το σύνολο λέγεται cluster. Οι κόμβοι αυτοί μπορεί να περιλαμβάνουν κεντρικές μονάδες επεξεργασίας, τοπικούς δίσκους SSD, μνήμη RAM και επεξεργαστές γραφικών [7]. Οι HPC υποδομές αποτελούνται συχνά από χιλιάδες κόμβους, ικανούς να παράγουν εκατοντάδες PFLOPS (10^15 πράξεις κινητής υποδιαστολής ανά δευτερόλεπτο) στην κορυφαία τους απόδοση [8].

Η βασική αρχή λειτουργίας των συστημάτων HPC για την επιτάχυνση των εργασιών είναι ο παραλληλισμός [7]. Ένα σύνθετο πρόβλημα διαιρείται σε μικρότερες εργασίες που μπορούν να επιλυθούν ανεξάρτητα και ταυτόχρονα σε πολλούς επεξεργαστές ή πυρήνες σε ένα υπολογιστικό cluster [9]. Αυτή η στρατηγική επιτρέπει ταχύτερους χρόνους εκτέλεσης και καλύτερη απόδοση. Για να αξιοποιηθεί το δυναμικό των clusters, απαιτείται ένα κατάλληλο μοντέλο προγραμματισμού. Πολλές επιστημονικές εφαρμογές που εκμεταλλεύονται τις δυνατότητες των HPC clusters χρησιμοποιούν μοντέλα παράλληλου προγραμματισμού για την επικοινωνία σε κατανεμημένη μνήμη, όπως η βιβλιοθήκη Message Passing Interface(MPI) [10]. Επιπλέον, για παράλληλη επεξεργασία σε κοινόχρηστη μνήμη εντός ενός κόμβου, χρησιμοποιούνται λύσεις όπως το OpenMP [11]. Συχνά, τα συστήματα HPC χρησιμοποιούν ένα υβριδικό μοντέλο που συνδυάζει MPI και OpenMP για να επωφεληθούν από την παράλληλη επικοινωνία μεταξύ κόμβων και την παράλληλη εκτέλεση εντός κόμβων [11].



Σχήμα 2.2: Cluster Αρχιτεκτονική του High Performance Computing

2.1.3 Ο Ρόλος του HPC στις γεωεπιστήμες

Οι προσομοιώσεις σεισμικών κυμάτων και η μοντελοποίηση φαινομένων ρηγμάτων απαιτούν την επίλυση μη γραμμικών, υπερβολικών συστημάτων με τεράστιες χωρικές και χρονικές απαιτήσεις [12] [13]. Επομένως, το HPC κρίνεται καθοριστικής σημασίας στον τομέα της σεισμολογίας και γενικότερα των γεωεπιστημών. Για παράδειγμα, οι φυσικά βασισμένες προσομοιώσεις σεισμών μπορούν να απαιτήσουν δεκάδες έως εκατοντάδες χιλιάδες ισοδύναμες ώρες CPU, ακόμη και όταν χρησιμοποιούνται πλαίσια HPC [13]. Η ανάγκη για μοντελοποίηση σύνθετων γεωλογικών δομών, ετερογενών υλικών και η προσομοίωση σε διαφορετικές κλίμακες καθιστά απαραίτητη την αξιοποίηση υπερυπολογιστικών υποδομών για την επίτευξη υψηλής ακρίβειας και ρεαλισμού [12] [13].

Το SeisSol αποτελεί ένα χαρακτηριστικό παράδειγμα λογισμικού που αξιοποιεί την ισχύ του ΗΠΣ για την αντιμετώπιση αυτών των προκλήσεων.

2.2 Προηγούμενες έρευνες για το SeisSol

Το SeisSol έχει αποτελέσει αντικείμενο συνεχούς έρευνας με σκοπό τη βελτίωση της ταχύτητας και της αποδοτικότητάς του σε υπολογιστικά συστήματα υψηλών επιδόσεων. Οι βασικές περιοχές έρευνας περιλαμβάνουν την ανάπτυξη και βελτιστοποίηση των αριθμητικών μεθόδων και της εσωτερικής δομής των υπολογιστικών πυρήνων, τη διαχείριση της εισόδου/εξόδου και την προσαρμογή σε νέες αρχιτεκτονικές υλικού.

2.2.1 Αρχικές Βελτιστοποίηση σε CPU Αρχιτεκτονικές

Οι πρώτες προσπάθειες εστίασαν στην επιτάχυνση της αριθμητικής μεθόδου ADER-DG, η οποία αποτελεί τη βάση του SeisSol , όπως αναφέρθηκε στο προηγούμενο κεφάλαιο. Επίκεντρο των ερευνών αποτέλεσε η αναδιάρθρωση των πιο υπολογιστικά απαιτητικών τμημάτων, γνωστών ως τοπικοί πυρήνες (local kernels), όπου εκτελείται η κύρια αριθμητική επεξεργασία. Αυτοί οι πυρήνες περιλαμβάνουν αλυσίδες πράξεων πολλαπλασιασμού μικρών πινάκων, των οποίων η βελτιστοποίηση είναι κρίσιμη για τη συνολική ταχύτητα του λογισμικού.

Για τον σκοπό αυτό αναπτύχθηκε η βιβλιοθήκη YATeTo (Yet Another Tensor Toolbox), ένα σύστημα παραγωγής κώδικα (code generation), που μετατρέπει τις μαθηματικές εκφράσεις του σχήματος DG σε βελτιστοποιημένο κώδικα C++ κατά το στάδιο μεταγλώττισης. Η YATeTo αναδιατάσσει τους βρόχους και διασπά τους τανυστές σε μικρούς πίνακες(Loop-over-GEMM), ώστε να αξιοποιεί στο έπακρο τις δυνατότητες των κρυφών μνημών(cache) και των μονάδων επεξεργασίας των διανυσμάτων(vector units). Οι πολλαπλασιασμοί μικρών πινάκων εκτελούνται από τη βιβλιοθήκη LIBXSMM, η οποία παράγει εξειδικευμένο assembly code για συγκεκριμένα μεγέθη πινάκων, επιτυγχάνοντας αποδόσεις που φτάνουν το 60% του θεωρητικού peak performance των επεξεργαστών x86 [4].

Η υλοποίηση αυτή, σε συνδυασμό με SIMD vectorization (AVX/AVX512) και παράλληλο προγραμματισμό MPI+OpenMP, οδήγησε σε σημαντικές επιταχύνσεις. Σε προσομοιώσεις μεγάλης κλίμακας, όπως το μοντέλο του σεισμού της Σουμάτρας (2004), το SeisSol πέτυχε επιτάχυνση 13,6× και μείωση του χρόνου εκτέλεσης σε 13,9 ώρες σε 86.016 πυρήνες του υπερυπολογιστή SuperMUC [1].

Αυτές οι έρευνες απέδειξαν ότι η αποδοτική παραγωγή εξειδικευμένου κώδικα σε επίπεδο πυρήνα μπορεί να αποδώσει καλύτερα αποτελέσματα από γενικού σκοπού βελτιστοποιήσεις, κάνοντας το SeisSol ένα από τα πιο αποδοτικά λογισμικά σεισμικής προσομοίωσης για CPU αρχιτεκτονικές.

2.2.2 Βελτιστοποίηση Εισόδου/Εξόδου (I/O)

Ένα κρίσιμο εμπόδιο για την περαιτέρω επιτάχυνση του SeisSol ήταν η χαμηλή ταχύτητα εισόδου/εξόδου. Καθώς το SeisSol εκτελεί προσομοιώσεις με δισεκατομμύρια στοιχεία, κάθε βήμα απαιτεί την ανάγνωση και εγγραφή τεράστιων όγκων δεδομένων —

τόσο για την περιγραφή του πλέγματος όσο και για την αποθήκευση των ενδιάμεσων αποτελεσμάτων (output snapshots).

Για την αντιμετώπιση αυτού του προβλήματος, αναπτύχθηκε ένα καινοτόμο παράλληλο σύστημα I/O ειδικά προσαρμοσμένο στις ανάγκες προσομοιώσεων μη δομημένων πλεγμάτων (unstructured meshes).

Στη νέα αρχιτεκτονική, αντικαταστάθηκε το σειριακό NetCDF3 με το παράλληλο netCDF4/HDF5, το οποίο υποστηρίζει ταυτόχρονη πρόσβαση μέσω MPI-IO. Αυτό επέτρεψε σε πολλούς υπολογιστικούς κόμβους να γράφουν τα δεδομένα τους απευθείας σε ένα κοινό αρχείο χωρίς σύγκρουση.

Επιπρόσθετα, υιοθετήθηκε η τεχνική συσσώρευσης(aggregation), η οποία επέτρεπε σε ομάδες διεργασιών να συγχωνεύουν τα δεδομένα τους και να τα εγγράφουν μέσω ενός writer process σε μεγαλύτερα, ευθυγραμμισμένα blocks, αυξάνοντας την αποδοτικότητα του δίσκου.

Η νέα αυτή προσέγγιση αύξησε το bandwidth εγγραφής από 832 MiB/s σε 6,7 GiB/s σε 2048 κόμβους του SuperMUC, επιτρέποντας ταχύτερη αποθήκευση και φόρτωση δεδομένων [14]. Παράλληλα, εισήχθη ένα νέο μορφότυπο πλέγματος netCDF4 με πληροφορίες για ghost layers και domain decomposition, μειώνοντας τον χρόνο αρχικοποίησης τεράστιων πλεγμάτων σε λιγότερο από 35 δευτερόλεπτα για περισσότερα από 1 δισεκατομμύριο στοιχεία.

΄Ετσι, το bottleneck του I/O περιορίστηκε σημαντικά, επιτρέποντας στο SeisSol να εκτελεί προσομοιώσεις petascale χωρίς σημαντικές καθυστερήσεις φόρτωσης ή αποθήκευσης [14].

2.2.3 Επιτάχυνση με Επιταχυντές GPU

Η επόμενη σημαντική κατεύθυνση έρευνας για την επιτάχυνση του SeisSol αφορά τη μεταφορά και βελτιστοποίηση του κώδικα σε GPU συστήματα. Αρχικά, παρατηρήθηκε ότι το SeisSol, λόγω της έντονης τοπικότητας της μεθόδου ADER-DG, είναι ιδιαίτερα κατάλληλο για εκτέλεση σε επιταχυντές GPU, όπου κάθε στοιχείο του πλέγματος μπορεί να υπολογιστεί ανεξάρτητα. Ωστόσο, η αρχική δομή του κώδικα ήταν βελτιστοποιημένη αποκλειστικά για CPUs, με αποτέλεσμα η άμεση μεταφορά σε GPU να είναι μη αποδοτική [4].

Για την αντιμετώπιση αυτού του προβλήματος, ανασχεδιάστηκε η αρχιτεκτονική του SeisSol, που βασίζεται στην παραγωγή κώδικα, ώστε να μπορεί να παράγει αυτόματα CUDA kernels για τις πιο κρίσιμες υπολογιστικές πράξεις. Για τον σκοπό αυτό χρησιμοποιήθηκε η βιβλιοθήκη GemmForge, που δημιουργεί βελτιστοποιημένους CUDA kernels για batched πολλαπλασιασμούς μικρών πινάκων, μια πράξη που κυριαρχεί στον χρόνο εκτέλεσης του ADER-DG σχήματος, όπως έχει αναφερθεί νωρίτερα.

Επιπλέον, το μοντέλο παραλληλισμού επεκτάθηκε από MPI+X σε MPI+CUDA, με αποτέλεσμα το SeisSol να μπορεί να υποστηρίξει distributed multi-GPU εκτέλεση, με κάθε MPI διεργασία να διαχειρίζεται ένα ή περισσότερα GPUs και με τοπική επικοινωνία μέσω NVLink ή PCIe.

Η GPU έκδοση του SeisSol πέτυχε 2,5× υψηλότερη απόδοση σε σχέση με τις αντίστοιχες ρουτίνες της cuBLAS και έδειξε καλή κλιμακωσιμότητα έως 256 GPUs στον υπερυπο-

λογιστή Marconi100 [4].

Ακόμη, αξιοποιήθηκε η μέθοδος Local Time Stepping (LTS), πού μειώνει τον αριθμό των χρονικών βημάτων για περιοχές με μεγαλύτερα στοιχεία πλέγματος, εξοικονομώντας χρόνο εκτέλεσης. Αν και η τεχνική αυτή οδηγεί σε χαμηλότερη χρησιμοποίηση υλικού λόγω μικρότερων clusters, εξακολουθεί να προσφέρει μέση μείωση του συνολικού χρόνου επίλυσης κατά 30% [4].

Αυτή η εξέλιξη καθιστά το SeisSol ένα φορητό λογισμικό υψηλής απόδοσης (performance-portable), ικανό να εκμεταλλεύεται τόσο CPU όσο και GPU αρχιτεκτονικές.

2.3 Το λογισμικό Intel VTune Profiler

Το Intel VTune Profiler αποτελεί ένα προηγμένο εργαλείο ανάλυσης απόδοσης (performance analysis), που αναπτύχθηκε από την Intel και χρησιμοποιείται ευρέως για τον εντοπισμό σημείων συμφόρησης (bottlenecks) σε εφαρμογές υψηλών επιδόσεων. Το εργαλείο αυτό συλλέγει μετρήσεις χαμηλού επιπέδου από το υλικό (όπως CPU cycles, cache misses, branch mispredictions και memory bandwidth) και τις αντιστοιχεί σε τμήματα του πηγαίου κώδικα, επιτρέποντας στον προγραμματιστή να εντοπίζει τις συναρτήσεις, τους βρόχους ή τα νήματα που καταναλώνουν τον περισσότερο χρόνο εκτέλεσης. [15] [16]

Η ανάλυση πραγματοποιείται μέσω δειγματοληψίας γεγονότων του επεξεργαστή (event-based sampling) και ανάλυσης των stack traces, προσφέροντας λεπτομερή εικόνα της συμπεριφοράς της εφαρμογής σε επίπεδο πυρήνα, μνήμης και παραλληλισμού. Επιπλέον, το VTune υποστηρίζει πολλαπλά μοντέλα προγραμματισμού όπως OpenMP, MPI, TBB και OpenCL, καθιστώντας το κατάλληλο για εφαρμογές που εκτελούνται σε ετερογενή περιβάλλοντα CPU– GPU [15]. Ο χρήστης μπορεί να πραγματοποιήσει αναλύσεις “Hotspot”, “Memory Access”, “Threading” ή “Microarchitecture Exploration”, ώστε να αξιολογήσει τη χρήση των διαθέσιμων πόρων και να κατευθύνει στοχευμένες βελτιστοποιήσεις [16] [15].

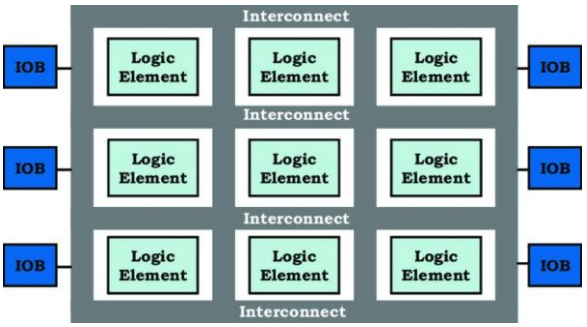
Συνοψίζοντας, το Intel VTune Profiler αποτελεί ένα ισχυρό μέσο για τη εύρεση των αιτίων χαμηλής απόδοσης, συμβάλλοντας στην ανάπτυξη ταχύτερων και πιο αποδοτικών εφαρμογών υψηλών επιδόσεων. Επομένως, η χρήση του VTune κρίνεται απαραίτητη για την ακριβή αξιολόγηση και βελτιστοποίηση της απόδοσης του SeisSol σε FPGA.

2.4 FPGA Αρχιτεκτονικές και Σχεδίαση με Vitis HLS και Vitis

2.4.1 Εισαγωγή στα FPGAs

Η αυξανόμενη Cήτηση για υπολογιστική ισχύ στην επίλυση πολύπλοκων επιστημονικών, τεχνολογικών και βιομηχανικών προβλημάτων έχει οδηγήσει στην αναζήτηση εναλλακτικών αρχιτεκτονικών πέραν των παραδοσιακών επεξεργαστών γενικού σκοπού. Σε αυτό το πλαίσιο, τα Field Programmable Gate Arrays (FPGAs) έχουν αναδειχθεί ως μία εξαιρετικά ευέλικτη και αποδοτική λύση για την επιτάχυνση υπολογιστικών διεργασιών και την ανάπτυξη προσαρμοσμένων αρχιτεκτονικών υψηλής απόδοσης.

΄Ενα FPGA αποτελεί μία προγραμματιζόμενη λογική συσκευή γενικού σκοπού, η οποία περιλαμβάνει λογικά blocks, διασυνδέσεις και μπλοκ εισόδου-εξόδου των οποίων η διαμόρφωση μπορεί να πραγματοποιηθεί μετά την κατασκευή, είτε από τον σχεδιαστή είτε από τον χρήστη [17]. Αυτή η δυνατότητα επαναπρογραμματισμού επιτρέπει στο ίδιο υλικό να προσαρμόζεται δυναμικά, από μία απλή λογική πύλη έως ένα πολύπλοκο σύστημα-σε-τσιπ ή ακόμη και έναν επιταχυντή τεχνητής νοημοσύνης, χωρίς να απαιτείται επανασχεδίαση του πυριτίου [17]. Η ιδιαιτερότητα αυτή προσδίδει στα FPGAs τον χαρακτήρα μιας τεχνολογίας που συνδυάζει το λογισμικό με το υλικό, καθιστώντας τα ιδανικά για εφαρμογές με υψηλό βαθμό παραλληλισμού, εξειδικευμένες απαιτήσεις υλικού και μεταβαλλόμενα πεδία δοκιμής, όπου οι σταθερές αρχιτεκτονικές CPU ή GPU ενδέχεται να μην επιδεικνύουν βέλτιστη απόδοση.



Σχήμα 2.3: Η βασική δομή ενός FPGA

Ειδικότερα, η δυνατότητα προσαρμογής του υλικού σε συγκεκριμένους αλγορίθμους επιτρέπει τη βελτιστοποίηση της ενεργειακής αποδοτικότητας και της υπολογιστικής ταχύτητας, καθιστώντας τα FPGAs ιδιαίτερα χρήσιμα σε εφαρμογές υψηλής απόδοσης, όπως η επιτάχυνση αριθμητικών πυρήνων του SeisSol [2]. Η χρήση των FPGAs ως επιταχυντών στην υπολογιστική υψηλής απόδοσης έχει αυξηθεί σημαντικά, προσφέροντας βελτιώσεις στην απόδοση λόγω της χωρικής υπολογιστικής τους ικανότητας και της εξαιρετικής ενεργειακής τους αποδοτικότητας [18].

Η ιστορική εξέλιξη των FPGA ξεκινά από τις πρώιμες συσκευές προγραμματιζόμενης λογικής και τη μετάβαση στα field-programmable logic arrays στα μέσα της δεκαετίας του 1980 [19]. Με την ωρίμανση της τεχνολογίας και την πρόοδο της μικροηλεκτρονικής, οι FPGA αρχιτεκτονικές εξελίχθηκαν ραγδαία, με αποτέλεσμα τη διεύρυνση των εφαρμογών τους σε πλήθος πεδίων, από τη βιοϊατρική και τα κέντρα δεδομένων έως τα συστήματα υψηλής απόδοσης [17].

Τα τελευταία χρόνια, οι FPGA αρχιτεκτονικές έχουν εμπλουτιστεί με πολύπλοκους πόρους, όπως block DSPs [2], προηγμένες ενσωματωμένες μνήμες (on-chip memories) και διεπαφές υψηλής ταχύτητας, συμπεριλαμβανομένων των multi-die και 3D packaging διασυνδέσεων [20]. Οι διεπαφές μνήμης έχουν επίσης εξελιχθεί σημαντικά, περνώντας από τεχνολογίες όπως SDRAM, DDR, DDR2, DDR3, DDR4 στην υιοθέτηση High Bandwidth Memory (HBM) μέσω interposer διεπαφών, οι οποίες προσφέρουν αυξημένη χωρητικότητα και εύρος Cώνης.

Παράλληλα, ανασκοπήσεις που επικεντρώνονται στις σύγχρονες τάσεις αναφέρουν πως η μετάβαση των FPGAs σε τεχνολογίες μικρότερου κόμβου, η ενσωμάτωση διεπαφών υψηλού ρυθμού και η υιοθέτηση τεχνικών όπως η μερική αναδιαμόρφωση (partial reconfiguration) έχουν συμβάλει στη χρήση τους όχι μόνο σε ενσωματωμένα συστήματα αλλά

και σε επιταχυντές μεγάλης κλίμακας. Η μερική αναδιαμόρφωση, συγκεκριμένα, έχει μελετηθεί εκτενώς, βελτιώνοντας την ευελιξία και την απόδοση των FPGAs [17].

Αυτές οι τεχνολογικές εξελίξεις — από τις προηγμένες διεπαφές μνήμης έως την αυξανόμενη πολυπλοκότητα και πυκνότητα των λογικών blocks — έχουν καταστήσει τα FPGAs ικανά να διαχειρίζονται ολοένα και πιο απαιτητικές υπολογιστικές εργασίες, ενώ παράλληλα υποστηρίζουν εργαλεία συνθεσης υψηλού επιπέδου (High-Level Synthesis – HLS), όπως το Vitis HLS, που επιτρέπουν την περιγραφή υλικού με γλώσσες υψηλού επιπέδου.

Συνοψίζοντας, τα FPGAs αποτελούν πλέον βασικό δομικό στοιχείο της σύγχρονης υπολογιστικής υποδομής, συνδυάζοντας προγραμματιζόμενη ευελιξία, ενεργειακή αποδοτικότητα και υψηλή απόδοση. Αυτά τα χαρακτηριστικά τα καθιστούν ιδανικά για εφαρμογές που απαιτούν ταυτόχρονα ευελιξία και υπολογιστική ισχύ, όπως η επιτάχυνση υπολογισμών, η ροή δεδομένων σε πραγματικό χρόνο και οι επιστημονικές προσομοιώσεις [17]. Η αυξανόμενη παρουσία των FPGAs στην έρευνα και ανάπτυξη συμβάλλει στην ενίσχυση του οικοσυστήματος HLS, επηρεάζοντας έτσι τον τρόπο σχεδιασμού και αξιοποίησης λογισμικού όπως το SeisSol.

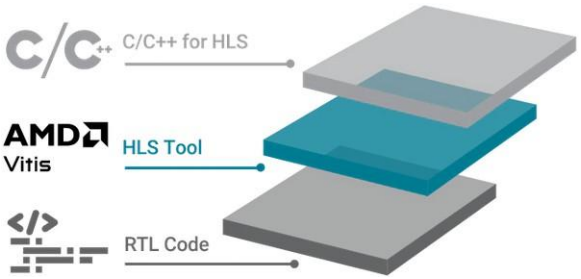
2.4.2 Τα προγράμματα σχεδίασης Vitis και VitisHLS

Το οικοσύστημα Vitis της AMD Xilinx αποτελεί ένα ολοκληρωμένο πλαίσιο εργαλείων για την ανάπτυξη, προσομοίωση και υλοποίηση επιταχυντών και εφαρμογών σε πλατφόρμες FPGA και SoC. Στόχος του είναι να γεφυρώσει το χάσμα μεταξύ λογισμικού και υλικού, παρέχοντας ένα ενιαίο περιβάλλον που επιτρέπει σε προγραμματιστές να αξιοποιούν τις δυνατότητες του υλικού χωρίς να απαιτείται εξειδικευμένη γνώση σε γλώσσες περιγραφής υλικού (HDLs) [21, 22].

Η εργαλειοθήκη Vitis συνδυάζει εργαλεία για τη σύνθεση (synthesis), προσομοίωση (simulation), ανάλυση απόδοσης (profiling) και υλοποίηση (implementation) εφαρμογών που εκτελούνται σε FPGA ή ετερογενείς αρχιτεκτονικές (heterogeneous computing). Μέσω του Vitis Unified Software Platform, οι προγραμματιστές μπορούν να αναπτύσσουν εφαρμογές σε γλώσσες υψηλού επιπέδου όπως C, C++, OpenCL ή Python, οι οποίες στη συνέχεια μετατρέπονται σε επιταχυντές υλικού μέσω της διαδικασίας σύνθεσης υψηλού επιπέδου (High-Level Synthesis – HLS) [21].

Το Vitis HLS αποτελεί το βασικό εργαλείο του οικοσυστήματος για τη μετάφραση κώδικα υψηλού επιπέδου (C, C++, SystemC) σε βελτιστοποιημένο κύκλωμα υλικού (RTL – Register Transfer Level). Κατά τη διαδικασία αυτή, ο προγραμματιστής μπορεί να αξιοποιήσει ειδικές οδηγίες (pragmas) ή κατευθυντήριες εντολές (directives) για να ελέγξει την παράλληλη εκτέλεση, τον χρονισμό, την κατανομή μνήμης και την αξιοποίηση των πόρων του FPGA. Οι παράμετροι αυτοί επιτρέπουν την επίτευξη του επιθυμητού συμβιβασμού μεταξύ ταχύτητας, κατανάλωσης ισχύος και χρήσης λογικών στοιχείων [22].

΄Ενα βασικό πλεονέκτημα του Vitis HLS είναι η δυνατότητα συν-σχεδίασης λογισμικού και υλικού (hardware–software co-design). Ο χρήστης μπορεί να αναπτύξει μια εφαρμογή όπου οι πιο κρίσιμες υπολογιστικές διεργασίες εκτελούνται σε επιταχυντές υλικού, ενώ ο έλεγχος και η ροή της εφαρμογής διαχειρίζονται από την CPU. Το περιβάλλον Vitis επιτρέπει την ενσωμάτωση αυτών των επιταχυντών με το σύστημα εκτέλεσης, τη διαχείριση μεταφοράς δεδομένων μεταξύ μνήμης και υλικού και την εκτέλεση



Σχήμα 2.4: Η θέση των εργαλείων Vitis στη σχεδίαση

προσομοιώσεων πριν την υλοποίηση.

Συνολικά, τα εργαλεία Vitis και Vitis HLS προσφέρουν έναν αποδοτικό και ευέλικτο τρόπο ανάπτυξης εφαρμογών για FPGA, επιτρέποντας τη γρήγορη δημιουργία, ανάλυση και βελτιστοποίηση επιταχυντών χωρίς την ανάγκη χειροκίνητης σχεδίασης σε επίπεδο RTL.

Κεφάλαιο 3

Εγκατάσταση του SeisSol και χρονική ανάλυση επιδόσεων

3.1 Εγκατάσταση του SeisSol και Επιλογή Παραδείγματος

Πρώτο βήμα για την εκπόνηση αυτής της εργασίας αποτέλεσε η εγκατάσταση του προγράμματος SeisSol. Αρχικά, πραγματοποιήθηκε προσπάθεια εγκατάστασης του σε προσωπικό υπολογιστή με λειτουργικό Windows. Κατά τη διαδικασία αυτή προέκυψαν σημαντικές δυσκολίες που σχετίζονταν με τη διαχείριση και τη συμβατότητα των απαιτούμενων βιβλιοθηκών. Δοκιμάστηκαν εναλλακτικές λύσεις, όπως η εκτέλεση σε περιβάλλον Windows Subsystem for Linux (WSL) και μέσω εικονικής μηχανής (VirtualBox), ωστόσο οι περιορισμοί των συγκεκριμένων περιβαλλόντων δεν επέτρεψαν τη σταθερή και αποδοτική λειτουργία του λογισμικού. Για τον λόγο αυτό κρίθηκε αναγκαία η εγκατάσταση λειτουργικού συστήματος Linux σε dual boot, ώστε το SeisSol να εκτελείται σε εγγενές περιβάλλον Linux, εξασφαλίζοντας τη σωστή λειτουργία των εξαρτήσεων και τη δυνατότητα αξιόπιστης ανάλυσης απόδοσης. Ωστόσο, εμφανίστηκαν προβλήματα υπολογιστικής ισχύος, με αποτέλεσμα το λογισμικό να μη λειτουργεί αποδοτικά.

Η λύση βρέθηκε με την εκ νέου εγκατάσταση του SeisSol στον server του εργαστηρίου, ο οποίος διέθετε Ubuntu 18.04.6 LTS, επεξεργαστή Intel Xeon E5-2620 v4 με 16 πυρήνες στα 2.10 GHz και 78 GB μνήμης RAM. Η συγκεκριμένη πλατφόρμα παρείχε τη δυνατότητα αξιοποίησης παραλληλισμού μέσω OpenMP, προσφέροντας σταθερό περιβάλλον για εκτελέσεις μεγάλης διάρκειας και αξιόπιστη συλλογή μετρικών απόδοσης. Η μεταγλώττιση του SeisSol πραγματοποιήθηκε με τον μεταγλωττιστή GCC 9.4.0, χρησιμοποιώντας CMake για τη ρύθμιση της διαδικασίας κατασκευής. Οι κύριες επιλογές σύνθεσης περιλάμβαναν τη ρύθμιση της αρχιτεκτονικής σε Haswell (hsw), ακρίβεια double, και τάξη μεθόδου 4 (ORDER=4). Ενεργοποιήθηκε η υποστήριξη για NUMA-aware pinning και ASAGI για τη διαχείριση δεδομένων πλέγματος, ενώ για τη βελτιστοποίηση των υπολογιστικών πυρήνων ενεργοποιήθηκαν οι βιβλιοθήκες LIBXSMM και PSpaMM. Το τελικό εκτελέσιμο παρήχθη σε λειτουργία Release, με ενεργοποιημένη τη σημαία -g για ενσωμάτωση συμβόλων debug, διευκολύνοντας τη μετέπειτα ανάλυση με το Intel VTune Profiler.

Οι απαιτούμενες εξαρτήσεις εγκαταστάθηκαν μέσω του διαχειριστή πακέτων Spack, συμπεριλαμβανομένων των HDF5, NetCDF, ParMETIS και PUMGen. Η βιβλιοθήκη HDF5 εγκαταστάθηκε τόσο μέσω Spack όσο και με χειροκίνητη μεταγλώττιση, προκειμένου να εξασφαλιστεί η σωστή υποστήριξη παραλληλίας (–enable-parallel) για τις ανάγκες εισόδου/εξόδου δεδομένων του SeisSol. Παράλληλα, το εργαλείο PUMGen χρησιμοποιήθηκε για τη δημιουργία των αρχείων πλέγματος που απαιτούνται από τα παραδείγματα του λογισμικού.

Για την εκτέλεση του προγράμματος απαιτείται ένα αρχείο παραμέτρων .par. Αρχικά, επιλέχθηκε το παράδειγμα tpv33, η δοκιμή του οποίου περιλαμβανόταν στις οδηγίες εγκατάστασης του SeisSol [23]. Ωστόσο, η εκτέλεση του συγκεκριμένου παραδείγματος αποδείχθηκε ιδιαίτερα χρονοβόρα(περίπου 10 ώρες) και αποφασίστηκε η διερεύνηση των επιλογών για την εύρεση ενός μικρότερου σεναρίου. Έπειτα από τη δοκίμη 6 διαθέσιμων παραδειγμάτων από το repository του SeisSol, επιλέχθηκε το benchmark tpv24. Ο συνολικός χρόνος εκτέλεσης αυτού του παραδείγματος ανήλθε σε περίπου 2 ώρες και 21 λεπτά, χρόνος αισθητά μικρότερος από τα υπόλοιπα, καθιστώντας το ιδανικό για την πραγματοποίηση αναλυτικής μελέτης απόδοσης με το Intel VTune Profiler. Το συγκεκριμένο παράδειγμα βασίζεται σε προσομοίωση δυναμικής διάρρηξης με δεδομένα εισόδου υψηλής ανάλυσης. Πριν από την εκτέλεση, πραγματοποιήθηκε η δημιουργία του πλέγματος με το σενάριο generating_the_mesh.sh, το οποίο καλεί το Gmsh για τη δημιουργία τρισδιάστατου μη συμβατού πλέγματος (non-conforming hybrid mesh). Η εκτέλεση του προγράμματος πραγματοποιήθηκε σε OpenMP λειτουργία, αξιοποιώντας 16 νήματα (OMP_NUM_THREADS=16). Για την εξισορρόπηση της κατανομής των νημάτων χρησιμοποιήθηκε η μεταβλητή OMP_PLACES="cores(8)", εξασφαλίζοντας βέλτιστη χρήση των διαθέσιμων πυρήνων.

Η ολοκλήρωση της εγκατάστασης και εκτέλεσης του παραδείγματος επιβεβαίωσε τη σωστή λειτουργία του λογισμικού και επέτρεψε την επόμενη φάση της έρευνας, που αφορούσε την εντοπισμό κρίσιμων περιοχών υπολογιστικής επιβάρυνσης (hotspots) και τη μετέπειτα βελτιστοποίηση επιλεγμένων συναρτήσεων.

3.2 Χρονική ανάλυση στο Vtune

Επόμενο στάδιο της εργασίας αποτέλεσε η χρονική ανάλυση των επιδόσεων του SeisSol, με τη χρήση του εργαλείου Intel VTune Profiler, προκειμένου να εντοπιστούν οι λειτουργίες που καταναλώνουν το μεγαλύτερο ποσοστό του συνολικού χρόνου εκτέλεσης.

Για την πραγματοποίηση της ανάλυσης αυτής, δημιουργήθηκε ένα νέο Project μέσω του γραφικού περιβάλλοντος (GUI) του Intel VTune Profiler. Στη διαμόρφωση του Project επιλέχθηκε η εκτέλεση στην τοπική μηχανή (local host), καθορίστηκε το μονοπάτι προς το εκτελέσιμο αρχείο του παραδείγματος tpv24, και επιλέχθηκαν ως τύπος ανάλυσης τα *Hotspots*.

Η συγκεκριμένη ανάλυση επιτρέπει τον εντοπισμό των πιο χρονοβόρων συναρτήσεων, παρουσιάζοντας για καθεμία τον συνολικό χρόνο εκτέλεσης, το ποσοστό κατανάλωσης πόρων και τη σχετική συμβολή της στη συνολική απόδοση του προγράμματος.

Η ανάλυση *Hotspots* αποκάλυψε τις συναρτήσεις που καταναλώνουν το μεγαλύτερο ποσοστό του συνολικού χρόνου εκτέλεσης. Στην Εικόνα 3.1 παρουσιάζονται οι πέντε

πιο χρονοβόρες συναρτήσεις (*top hotspots*) του προγράμματος, όπως καταγράφηκαν από το Intel VTune Profiler.

Top Hotspots

This section lists the most active functions in your application. Optimizing these hotspot functions typically results in improving overall application performance.

Function	Module	CPU Time	% of CPU Time
libxsmm_dense_double_m20_n9_k10_ldA20_ldB12_ldC20_alpha1_beta1_alignedA1_alignedC1_transAFalse_transBFalse_BL2viaC	SeisSol_Release_dhsw_4_elastic	23678.849 s	18.2%
seissol::kernel::derivativeTaylorExpansion::execute	SeisSol_Release_dhsw_4_elastic	16108.335 s	12.4%
libxsmm_dense_double_m12_n9_k20_ldA12_ldB20_ldC12_alpha1_beta0_alignedA1_alignedC1_transAFalse_transBFalse_nopf	SeisSol_Release_dhsw_4_elastic	13740.170 s	10.5%
libxsmm_dense_double_m12_n9_k9_ldA12_ldB9_ldC12_alpha1_beta0_alignedA1_alignedC1_transAFalse_transBFalse_nopf	SeisSol_Release_dhsw_4_elastic	13404.677 s	10.3%
libxsmm_dense_double_m20_n9_k10_ldA20_ldB12_ldC20_alpha1_beta1_alignedA1_alignedC1_transAFalse_transBFalse_nopf	SeisSol_Release_dhsw_4_elastic	7322.281s	5.6%

Σχήμα 3.1: Αποτελέσματα της ανάλυσης Hotspots στο Intel Vtune Profiler

Οι πέντε αυτές συναρτήσεις αντιπροσώπευαν συνολικά πάνω από το 50% του χρόνου χρήσης της CPU, γεγονός που καθιστά εμφανή οποιαδήποτε πιθανή βελτιστοποίηση στην απόδοση του προγράμματος. Για τον λόγο αυτό, αποφασίστηκε να μελετηθούν και να βελτιστοποιηθούν όλα τα παραπάνω hotspots, ώστε να επιτευχθεί ουσιαστική μείωση του συνολικού χρόνου εκτέλεσης, σε αντίθεση με την απομόνωση μόνο μίας συνάρτησης, όπου η όποια βελτίωση θα είχε περιορισμένο αντίκτυπο.

Ακόμη, παρατηρήθηκε ότι η πρώτη και η πέμπτη συνάρτηση είχαν ουσιαστικά τον ίδιο κώδικα, με μοναδική διαφορά την προσθήκη εντολών prefetch στην πρώτη περίπτωση. Η ύπαρξη αυτών των εντολών υποδεικνύει πως η πρώτη εκδοχή αποτελεί ήδη βελτιστοποιημένη μορφή της πέμπτης, καθώς το prefetching χρησιμοποιείται για την προφόρτωση δεδομένων στη μνήμη cache, με στόχο τη μείωση καθυστερήσεων πρόσβασης. Ως εκ τούτου, θεωρήθηκε σκόπιμο να επικεντρωθεί η προσπάθεια βελτιστοποίησης στα hotspots 2, 3, 4 και 5.

3.3 Περιγραφή των συναρτήσεων

3.3.1 Γενική επισκόπηση των hotspots

Με βάση την ανάλυση που πραγματοποιήθηκε με το Intel VTune Profiler, εντοπίστηκαν πέντε κύριες συναρτήσεις που συγκεντρώνουν σημαντικό ποσοστό του συνολικού CPU time στο παράδειγμα tpv24. Οι συναρτήσεις αυτές αποτελούν υπολογιστικά εντατικά τμήματα του SeisSol και συνιστούν τους βασικούς στόχους για περαιτέρω διερεύνηση.

Το δεύτερο hotspot αντιστοιχεί σε μία συνάρτηση, υλοποιημένη σε C++, που αναφέρεται στη μαθηματική τεχνική "Taylor Expansion" και χρησιμοποιείται εκτεταμένα κατά την εξέλιξη του αριθμητικού σχήματος. Τα υπόλοιπα hotspots προέρχονται από μικρού μεγέθους πυρήνες που πραγματοποιούν πολλαπλασιασμό πινάκων, παραγόμενους από τη βιβλιοθήκη LIBXSMM, σε γλώσσα assembly. Παρά το μικρό χρόνο εκτέλεσης κάθε μεμονωμένης κλήσης, η εξαιρετικά υψηλή συχνότητα ενεργοποίησης αυτών των πυρήνων οδηγεί σε σημαντική χρονική επιβάρυνση.

Στις ακόλουθες ενότητες, θα πραγματοποιηθεί η εξέταση των συναρτήσεων αυτών, εστιάζοντας στη λειτουργία και την υπολογιστική τους αρχιτεκτονική.

3.3.2 Συνάρτηση derivativeTaylorExpansion::execute()

Το δεύτερο σημείο επιβάρυνσης που εντοπίστηκε στην ανάλυση αποτελεί η συνάρτηση kernel::derivativeTaylorExpansion::execute(), η οποία είναι υλοποιημένη σε C++ και ανήκει στο υποσύστημα των πυρήνων του SeisSol. Ο ρόλος της συνάρτησης είναι ο υπολογισμός ενός ενδιάμεσου όρου ανάπτυξης Taylor της λύσης, μέσω γραμμικού συνδυασμού προϋπολογισμένων παραγώγων της κατάστασης dQ με κατάλληλους συντελεστές power. Το αποτέλεσμα του συνδυασμού αποθηκεύεται στον πίνακα I, ο οποίος χρησιμοποιείται σε επόμενα στάδια του αριθμητικού σχήματος.

Στην αρχή της συνάρτησης πραγματοποιούνται έλεγχοι ορθότητας με assert, ώστε να διασφαλιστεί ότι οι συντελεστές power δεν λαμβάνουν μη έγκυρες τιμές(NaN) και ότι οι δείκτες προς τους πίνακες I και dQ είναι έγκυροι.

Η συνάρτηση οργανώνεται σε τέσσερις διαδοχικούς διπλούς βρόχους. Ο εξωτερικός βρόχος b διατρέχει 9 «στήλες», ενώ ο εσωτερικός βρόχος a διατρέχει επιμέρους υποσύνολα της πρώτης διάστασης, με διαφορετικό άνω όριο κάθε φορά (20, 10, 4 και 1 στοιχεία αντίστοιχα).

Συνολικά, για κάθε τιμή της b η συνάρτηση εκτελεί έναν γραμμικό συνδυασμό στοιχείων από τα τέσσερα σύνολα παραγώγων dQ(0)–dQ(3), ενημερώνοντας τμηματικά τον πίνακα I. ΄Ολες οι δομές δεδομένων είναι αποθηκευμένες σε μονοδιάστατους πίνακες, με δείκτη της μορφής a + stride * b, ώστε η εσωτερική διάσταση a να αντιστοιχεί σε συνεχόμενες θέσεις μνήμης.

Η χρήση της οδηγίας #pragma omp simd στον εσωτερικό βρόχο υποδεικνύει στον μεταγλωττιστή ότι οι επαναλήψεις μπορούν να διανυσματοποιηθούν (vectorization) κατά μήκος της διάστασης a, εκμεταλλευόμενες τις SIMD μονάδες της CPU.

3.3.3 Assembly-based μικροπυρήνες πολλαπλασιασμού πινάκων (LIBXSMM kernels)

Τα hotspots 3, 4 και 5 που εντοπίστηκαν από το Intel VTune Profiler αντιστοιχούν σε μικρού μεγέθους, πλήρως βελτιστοποιημένους μικροπυρήνες πολλαπλασιασμού πινάκων, παραγόμενους από τη βιβλιοθήκη LIBXSMM. Οι πυρήνες αυτοί υλοποιούν πράξεις τύπου GEMM με συγκεκριμένες διαστάσεις, οι οποίες εμφανίζονται επαναλαμβανόμενα στο αριθμητικό σχήμα ADER-DG. Παρόλο που ο χρόνος εκτέλεσης κάθε μεμονωμένης κλήσης είναι πολύ μικρός, η υψηλή συχνότητα ενεργοποίησής τους οδηγεί σε σημαντική συσσωρευτική συμβολή στον συνολικό CPU time.

Οι αρχικές υλοποιήσεις σε assembly αξιοποιούν εντολές AVX2 και FMA (Fused Multiply-Add), με χειροκίνητη διαχείριση διανυσματικών καταχωρητών(registers) και συσσωρευτών(accumulators). Για τις ανάγκες της παρούσας εργασίας και της μετέπειτα επεξεργασίας στο Vitis HLS, οι συναρτήσεις μετατράπηκαν σε απλές, κατανοητές C++ υλοποιήσεις, οι οποίες διατηρούν την ίδια αριθμητική λειτουργία. Η επιλογή αυτής της μορφής υλοποίησης ήταν συνειδητή: μια *καθαρή* C++ περιγραφή σε επίπεδο βρόχων και δεικτών διευκολύνει τη χρήση HLS pragmas και την εξερεύνηση διαφορετικών χαρτογραφήσεων σε υλικό, χωρίς να απαιτείται χειροκίνητος χειρισμός καταχωρητών ή

vector intrinsics. Με αυτόν τον τρόπο, η βελτιστοποίηση μεταφέρεται από το επίπεδο της assembly στο επίπεδο της συν-σχεδίασης λογισμικού–υλικού.

3.3.3.1 LIBXSMM kernel m12_n9_k20

Ο πυρήνας m12_n9_k20 υλοποιεί τον πολλαπλασιασμό πινάκων με διαστάσεις A : 12×20, B : 20×9, C : 12×9. Η πράξη ακολουθεί μια τριπλή δομή βρόχων, όπου ο εξωτερικός βρόχος i χωρίζει τις γραμμές του C σε τρία blocks των τεσσάρων γραμμών ($i = 0, 1, 2$), ενώ οι εσωτερικοί βρόχοι διατρέχουν την ενδιάμεση διάσταση k=20 και τις γραμμές μέσα στο block.

Η πρόσβαση στον πίνακα B έχει τη μορφή:

$$temp1B = B[j + 60 \cdot i], \quad temp2B = B[j + 20 + 60 \cdot i], \quad temp3B = B[j + 40 + 60 \cdot i],$$

όπου ο δείκτης j ($0 \leq j < 20$) διατρέχει την ενδιάμεση διάσταση k. Κάθε επανάληψη του εξωτερικού βρόχου i επεξεργάζεται ταυτόχρονα τρεις στήλες του πίνακα B: οι τριάδες $(j + 60i)$, $(j + 20 + 60i)$ και $(j + 40 + 60i)$αντιστοιχούν σε τρεις διαδοχικές στήλες, οργανωμένες σε blocks των 20 στοιχείων. Στο εσωτερικό, για κάθε j φορτώνονται τα τρία στοιχεία temp1B, temp2B, temp3B και πολλαπλασιάζονται με μία ολόκληρη στήλη του A, η οποία είναι οργανωμένη σε 3 blocks. Τα αποτελέσματα συσσωρεύονται σε εννέα πίνακες accum1 έως accum9 των τεσσάρων στοιχείων, καθένας από τους οποίους αντιστοιχεί σε μία στήλη του C.

Η αποθήκευση των αποτελεσμάτων στον πίνακα C γίνεται μέσα στον εξωτερικό βρόχο i, όταν ολοκληρωθούν όλες οι επαναλήψεις ως προς το j. Για κάθε block τεσσάρων γραμμών, τα συσσωρευμένα αποτελέσματα γράφονται ως εξής:

$$\begin{aligned} C[l + i \cdot 36] &= accum1[l], \\ C[l + 4 + i \cdot 36] &= accum2[l], \\ &\quad l = 0, \ldots, 3. \\ &\cdot \\ C[l + 32 + i \cdot 36] &= accum9[l], \end{aligned}$$

Με αυτόν τον τρόπο, κάθε επανάληψη του i γράφει ένα πλήρες block 4×9 του πίνακα C σε μια συνεχόμενη περιοχή μνήμης μήκους 36 στοιχείων. Η διάταξη αυτή είναι ιδιαίτερα φιλική προς την cache και τη διανυσματοποίηση, καθώς επιτρέπει την ομαδοποιημένη αποθήκευση τεσσάρων στοιχείων ανά στήλη.

Αξίζει να σημειωθεί ότι η αρχική υλοποίηση σε assembly ήταν ιδιαίτερα βελτιστοποιημένη και αξιοποιούσε στο μέγιστο τις εντολές AVX2/ FMA. Η βιβλιοθήκη LIBXSMM παράγει κώδικα με πλήρη αποβροχοποίηση (full unrolling) των τριών κλασικών βρόχων του GEMM και χρησιμοποιεί ολόκληρους διανυσματικούς καταχωρητές (YMM) ως ανεξάρτητους συσσωρευτές. Καθέ ένας από αυτούς τους συσσωρευτές περιέχει 4 στοιχεία. Αυτός είναι και ο λόγος που επιλέχθηκε η επεξεργασία των στοιχείων σε τετράδες στον κώδικα C++.

Κάθε εντολή vfmadd231pd εκτελεί τρείς πράξεις (acc = acc + A * B) σε μια κίνηση, μειώνοντας το instruction overhead και αξιοποιώντας το εύρος Cώνης των καταχω-

ρητών. Επιπλέον, η assembly εκδοχή περιλαμβάνει χειροκίνητο scheduling φόρτωσης(vmovapd) ώστε να αποφεύγονται pipeline stalls, καθώς και χρήση συγκεκριμένων καταχωρητών για τα βήματα(strides) των πινάκων, περιορίζοντας τη συχνότητα υπολογισμού διευθύνσεων.

Παρά τη σημαντική της ταχύτητα, η συγκεκριμένη μορφή κώδικα δεν είναι άμεσα αξιοποιήσιμη σε εργαλεία όπως το Vitis HLS, καθώς δένει τη λειτουργικότητα σε μια συγκεκριμένη CPU ISA. Η μετατροπή σε καθαρή C++ μας επιτρέπει να αποτυπώσουμε την αριθμητική δομή του αλγορίθμου σε επίπεδο βρόχων, ανοίγοντας τον δρόμο για βελτιστοποίηση σε FPGA με χρήση HLS pragmas, παράγοντας τελικά έναν φορητό, επεκτάσιμο και ευκολότερο στη συντήρηση κώδικα.

3.3.3.2 LIBXSMM kernel m12_n9_k9

Ο πυρήνας m12_n9_k9 έχει παρόμοια δομή με τον m12_n9_k20, αλλά με ενδιάμεση διάσταση k = 9. Οι διαστάσεις των πινάκων είναι $A : 12 \times 9, B : 9 \times 9,$ C : 12 × 9. Η C++ υλοποίηση οργανώνεται σε τρεις βασικούς βρόχους:

- εξωτερικός βρόχος i = 0..2, ο οποίος χωρίζει τον πίνακα C σε τρία blocks τεσσάρων γραμμών,
- ενδιάμεσος βρόχος j = 0..8, που διατρέχει την ενδιάμεση διάσταση k,
- εσωτερικός βρόχος l = 0..3, που διατρέχει τις τέσσερις γραμμές του εκάστοτε block.

Στον βρόχο ως προς j, ο κώδικας φορτώνει τρία στοιχεία από τον πίνακα B:

$$temp1B = B[j + 27 \cdot i], \quad temp2B = B[j + 9 + 27 \cdot i], \quad temp3B = B[j + 18 + 27 \cdot i].$$

Για κάθε τιμή του i, τα μεγέθη αυτά αντιστοιχούν σε τρεις διαδοχικές στήλες του B. Πιο συγκεκριμένα, για i = 0 επεξεργάζονται οι στήλες 0, 1 και 2, για i = 1 οι στήλες 3, 4, 5, και για i = 2 οι στήλες 6, 7, 8. Το j διατρέχει τις γραμμές αυτών των στηλών ($0 \leq j < 9$), οπότε σε κάθε επανάληψη του εξωτερικού βρόχου i επεξεργαζόμαστε ένα block από τρεις στήλες του B.

Μέσα στον εσωτερικό βρόχο l, τα στοιχεία αυτά του B πολλαπλασιάζονται με τα αντίστοιχα στοιχεία του A (για τις τέσσερις γραμμές του block) και τα αποτελέσματα προστίθενται σε εννέα συσσωρευτές accum1 έως accum9 των τεσσάρων στοιχείων. Κάθε accumX[l] αντιστοιχεί σε ένα στοιχείο του C στη γραμμή l του εκάστοτε block και σε μια συγκεκριμένη στήλη: οι τρεις πρώτοι συσσωρευτές αντιστοιχούν στις τρεις στήλες του B που επεξεργάζεται το συγκεκριμένο i, οι επόμενοι τρεις στις επόμενες στήλες, κ.ο.κ.

΄Οταν ολοκληρωθούν όλες οι επαναλήψεις ως προς j, τα συσσωρευμένα αποτελέσματα γράφονται στον πίνακα C. Η αποθήκευση πραγματοποιείται και πάλι μέσα στον εξωτερικό βρόχο i και έχει τη μορφή:

$$\begin{aligned} C[l + i \cdot 36] &= accum1[l], \\ C[l + 4 + i \cdot 36] &= accum2[l], \\ &\qquad\qquad l = 0, \ldots, 3. \\ &\vdots \\ C[l + 32 + i \cdot 36] &= accum9[l], \end{aligned}$$

Για κάθε i, η έκφραση C[. . . + i * 36] γράφει ένα συνεχές block 36 στοιχείων, που αντιστοιχεί σε έναν πίνακα 4 × 9 : τέσσερις γραμμές (τιμές του l) και εννέα στήλες (εννέα συσσωρευτές). Η διάταξη αυτή είναι ιδιαίτερα σημαντική, καθώς καθορίζει τόσο τον τρόπο που γίνεται η αποθήκευση στη μνήμη όσο και τις δυνατότητες μελλοντικής βελτιστοποίησης σε επίπεδο cache και παραλληλισμού.

Στην αρχική υλοποίηση της LIBXSMM, ο πυρήνας m12_n9_k9 είναι ακόμη πιο επιθετικά αποβροχοποιημένος (σε σχέση με το k=9) και αξιοποιεί ένα πολύ συγκεκριμένο μοτίβο microkernel layout: τριάδες στηλών του B φορτώνονται σε YMM registers, ενώ κάθε ομάδα τεσσάρων γραμμών του A φορτώνεται επίσης σε ειδικούς registers. Τα εννέα διανυσματικά accumulators του C παραμένουν συνεχώς σε registers καθ΄ όλη τη διάρκεια του υπολογισμού, χωρίς καμία ενδιάμεση πρόσβαση στη μνήμη. Αυτό σημαίνει ότι ολόκληρη η πράξη C += A × B γίνεται αποκλειστικά σε registers και μόνο στο τέλος γράφεται πίσω ο πίνακας C. Η τεχνική αυτή μειώνει δραστικά τον αριθμό φορτώσεων/αποθηκεύσεων και αυξάνει την υπολογιστική πυκνότητα.

Η assembly εκδοχή χρησιμοποιεί επίσης προσεκτικό instruction interleaving για τις vfmadd231pd, ώστε να διασκορπίζονται μεταξύ φορτώσεων και να ελαχιστοποιούνται τα stalls στο pipeline της εκτέλεσης. Ωστόσο, αυτή η μορφή μικροβελτιστοποίησης απευθύνεται αποκλειστικά σε αρχιτεκτονικές x86_64 και δεν μπορεί να μεταφερθεί σε FPGA. Η μετατροπή σε καθαρή C++ απλοποιεί τον αλγόριθμο στο δομικό του επίπεδο, χρησιμοποιώντας loops, blocks και συσσωρευτές, και επιτρέπει την εφαρμογή HLS τεχνικών όπως loop unrolling, pipelining και array partitioning σε επίπεδο hardware, διατηρώντας παράλληλα την ίδια αριθμητική συμπεριφορά.

3.3.3.3 LIBXSMM kernel m20_n9_k10

Η ιδιαιτερότητά του πυρήνα m20_n9_k10 σε σχέση με τους προηγούμενους δύο πυρήνες (m12_n9_k20 και m12_n9_k9) έγκειται στο μεγαλύτερο πλήθος γραμμών του πίνακα A και στη διαφορετική οργάνωση των στηλών του C, οι οποίες δεν αντιστοιχούν πλέον σε τριάδες στηλών όπως στους άλλους πυρήνες, αλλά απαιτούν δύο διακριτά στάδια υπολογισμού.

Σε αντίθεση με τα προηγούμενα hotspots όπου ολόκληρος ο πίνακας C υπολογιζόταν σε μία ενιαία φάση, στον πυρήνα m20_n9_k10 ο αλγόριθμος χωρίζεται υποχρεωτικά σε δύο στάδια. Αυτό οφείλεται στη δομή του αρχικού μικροπυρήνα της LIBXSMM, ο οποίος στο πρώτο στάδιο χρησιμοποιεί δώδεκα συσσωρευτές (accum1–accum12) για να ενημερώσει τα βασικά τμήματα του block των 60 στοιχείων του πίνακα C. Οι δώδεκα αυτοί accumulators καλύπτουν 48 από τα 60 στοιχεία, σύμφωνα με τη διάταξη του A σε blocks τεσσάρων γραμμών. Στη συνέχεια, ο μικροπυρήνας εκτελεί ένα δεύτερο στάδιο όπου χρησιμοποιεί τρεις επιπλέον συσσωρευτές (accum10, accum11, accum12 στη μεταγραφή) για να ενημερώσει συγκεκριμένες θέσεις μέσα στο ίδιο block, στις θέσεις offsets 16, 36 και 56.

Στην C++ υλοποίηση, η διαδικασία αυτή αποτυπώνεται ρητά. Αρχικά, οι συσσωρευτές accum1 έως accum12 αρχικοποιούνται με τις υπάρχουσες τιμές του C μέσω του ενδιάμεσου πίνακα local_C, ώστε να επιδράσει σωστά ο παράγοντας beta = 1 του μι-

κροπυρήνα(εξαιρούνται τα offsets 16, 36 και 56):

$$\begin{aligned}
\text{accum1}[l] &\leftarrow C[l + i \cdot 60],\\
\text{accum2}[l] &\leftarrow C[l + 4 + i \cdot 60],\\
\text{accum3}[l] &\leftarrow C[l + 8 + i \cdot 60],\\
&\vdots\\
\text{accum12}[l] &\leftarrow C[l + 52 + i \cdot 60].
\end{aligned}$$

Στο πρώτο στάδιο υπολογίζονται οι εννέα πρώτες στήλες του C, αντιστοιχώντας ακριβώς στην οργάνωση των ymm4 έως ymm12 στην assembly έκδοση. Για κάθε έναν από τους τρεις πίνακες στηλών του B (με offsets 0, 12 και 24), εκτελείται ο βρόχος ως προς j, ενημερώνοντας τους συσσωρευτές ως προς τις τέσσερις γραμμές κάθε block:

$$\text{accum1}[l] \leftarrow \text{accum1}[l] + B(j) \cdot A(j, l), \qquad l = 0, \ldots, 3.$$

Με την ολοκλήρωση του πρώτου σταδίου, οι τιμές αυτές γράφονται πίσω στον local_C(εξαιρούνται τα offsets 16, 36 και 56):

$$\begin{aligned}
\text{local_C}[l + i \cdot 60] &= \text{accum1}[l],\\
\text{local_C}[l + 4 + i \cdot 60] &= \text{accum2}[l],\\
\text{local_C}[l + 8 + i \cdot 60] &= \text{accum3}[l],\\
&\vdots\\
\text{local_C}[l + 52 + i \cdot 60] &= \text{accum12}[l].
\end{aligned}$$

Στο δεύτερο στάδιο εκτελείται ένας νέος κύκλος υπολογισμών που αφορά αποκλειστικά τις επιπλέον τρεις στήλες του C, εκείνες που στην ασσεμβλψ αποθηκεύονται σε ymm13, ymm14 και ymm15. Αυτές οι στήλες βρίσκονται στα offsets 16, 36 και 56 στοιχείων μέσα στο block των 60 στοιχείων, οπότε προηγουμένως πρέπει να φορτωθούν τα αρχικά τους δεδομένα στους συσσωρευτές:

$$\text{accum10}[l] \leftarrow C[l{+}16{+}i{\cdot}60], \quad \text{accum11}[l] \leftarrow C[l{+}36{+}i{\cdot}60], \quad \text{accum12}[l] \leftarrow C[l{+}56{+}i{\cdot}60].$$

Στη συνέχεια, εκτελείται νέος βρόχος ως προς j, στον οποίο οι πράξεις πολλαπλασιασμού και πρόσθεσης γίνονται για τις στήλες που αντιστοιχούν στα δεύτερα τμήματα γραμμών του πίνακα A (δηλαδή τους δείκτες $l+16$). Αυτό είναι απαραίτητο επειδή η δομή μνήμης του A είναι τέτοια που οι γραμμές 16–19 αποτελούν ξεχωριστό block των 4 στοιχείων, όπως και στην assembly εκδοχή.

Τέλος, οι ενημερωμένοι συσσωρευτές αποθηκεύονται πίσω στον local_C στα σωστά offsets:

$$\begin{aligned}
\text{local_C}[l + 16 + i \cdot 60] &= \text{accum10}[l],\\
\text{local_C}[l + 36 + i \cdot 60] &= \text{accum11}[l],\\
\text{local_C}[l + 56 + i \cdot 60] &= \text{accum12}[l].
\end{aligned}$$

Η επιλογή διάσπασης του πυρήνα σε δύο στάδια δεν είναι αυθαίρετη. Αντιγράφει με ακρίβεια τη δομή του αρχικού assembly μικροπυρήνα της LIBXSMM, η οποία είναι από

τις πιο περίπλοκες, καθώς ο αριθμός γραμμών (m = 20) δεν ταιριάζει ομοιόμορφα στο πλάτος των AVX καταχωρητών. Η LIBXSMM αντιμετωπίζει το πρόβλημα αυτό με έναν υβριδικό μικροπυρήνα: ο πίνακας A μπλοκάρεται σε ομάδες των τεσσάρων γραμμών, οι οποίες μπορούν να επεξεργαστούν ταυτόχρονα εντός ενός YMM register group, ενώ οι υπόλοιπες γραμμές υλοποιούνται σε δεύτερο χρόνο. Παράλληλα, τα στοιχεία του πίνακα C κατανέμονται σε περισσότερους από δώδεκα συσσωρευτές που αντιστοιχούν σε YMM καταχωρητές, με μη ομοιόμορφα offsets και strides. Ο κώδικας χρησιμοποιεί χειροποίητο scheduling εντολών FMA, ώστε να επιτευχθεί υψηλή πληρότητα του execution pipeline και μέγιστη απόδοση στις AVX2 μονάδες. Με τον τρόπο αυτό, η μετατροπή σε C++ διατηρεί πλήρη ισοδυναμία με την αριθμητική λογική του αρχικού κώδικα ενώ ταυτόχρονα αποδίδεται σε μορφή κατάλληλη για βελτιστοποίηση μέσω Vitis HLS.

Κεφάλαιο 4

Υλοποίηση

4.1 Μεθοδολογία χαρτογράφησης των kernels σε FPGA

Η υλοποίηση των μικροπυρήνων του SeisSol σε FPGA βασίστηκε σε μια σαφώς ορισμένη διαδικασία ανάπτυξης, η οποία περιλαμβάνει όλα τα στάδια από τη μετατροπή των αρχικών συναρτήσεων σε μορφή κατάλληλη για High-Level Synthesis έως την τελική βελτιστοποίηση και αξιολόγηση της απόδοσης στο υλικό. Στόχος είναι η δημιουργία εξειδικευμένων επιταχυντών που παρουσιάζουν μικρότερο χρόνο εκτέλεσης σε σχέση με τις αντίστοιχες υλοποιήσεις software.

4.1.1 Υπολογιστική πλατφόρμα

Η χαρτογράφηση και αξιολόγηση των kernels πραγματοποιήθηκε στον επιταχυντή Xilinx Alveo U200, μία PCIe επιταχυνόμενη κάρτα βασισμένη στο FPGA Xilinx UltraScale+ VU9P. Η κάρτα στοχεύει εφαρμογές υψηλής υπολογιστικής πυκνότητας και bandwidth, προσφέροντας τα εξής χαρακτηριστικά [24, 25]:

- 64 GB HBM/DDR4 συνολικής χωρητικότητας, με δυνατότητα ταχύτατου PCIe Gen3x16/Gen4x8 interface,
- 902k LUTs και 2.8M flip-flops,
- 6840 DSP slices για εφαρμογές έντονων αριθμητικών υπολογισμών,
- θεωρητικό memory bandwidth έως και 77.8 GB/s,
- υποστήριξη clock frequency για FPGA kernels έως και 300 MHz.

Η ανάπτυξη του κώδικα πραγματοποιήθηκε στο Vitis HLS 2022.2 και η τελική ολοκλήρωση στο Vitis 2022.2, σε περιβάλλον Ubuntu 18.04.6 LTS. Για την εκτέλεση των kernels στο FPGA χρησιμοποιήθηκε η βιβλιοθήκη Xilinx Runtime (XRT). Αυτή η βιβλιοθήκη παρέχει ένα πρότυπο προγραμματισμού και διαχείρισης συσκευών για επικοινωνία μεταξύ του κεντρικού υπολογιστή (host) και του FPGA, επιτρέποντας την απρόσκοπτη εκτέλεση επιταχυνόμενων εφαρμογών [26].

4.1.2 Συνολική ροή ανάπτυξης

Η διαδικασία ανάπτυξης ακολούθησε τα παρακάτω στάδια:

1. Επιλογή των τριών LIBXSMM kernels ως αρχικό πεδίο μελέτης.
2. Μετατροπή των αρχικών assembly υλοποιήσεων σε C++ reference κώδικα.
3. Δημιουργία project στο Vitis HLS και εισαγωγή των ανακατασκευασμένων C++ συναρτήσεων.
4. Ανάπτυξη κοινού testbench για όλους τους kernels.
5. Προκαταρκτική διερεύνηση βελτιστοποιήσεων και αξιολόγηση του synthesis report.
6. Δημιουργία project στο Vitis.
7. Ανάπτυξη host εφαρμογής και ενσωμάτωση των kernels.
8. Εκτέλεση software emulation, hardware emulation και hardware runs.
9. Δοκιμή επιπρόσθετων βελτιστοποιήσεων στο Vitis HLS και επανάληψη των επόμενων βημάτων.
10. Επαναληπτική διαδικασία για τη συνάρτηση Taylor.

Σημειώνεται ότι, παρότι αρχικά οι τρεις kernels αντιμετωπίστηκαν ενιαία, σε μεταγενέστερο στάδιο η ανάπτυξη πραγματοποιήθηκε σε ξεχωριστά έργα, ώστε να διευκολυνθεί ο εντοπισμός και η αντιμετώπιση προβλημάτων synthesis, ενώ αποφασίστηκε η εκ νέου ενσωμάτωση των τελικών εκδοχών τους σε κοινό project για τη συνολική αξιολόγηση των βελτιώσεων.

4.1.3 Προκλήσεις σχεδιασμού

Παρότι οι kernels δεν παρουσιάζουν υψηλή κατανάλωση πόρων, η κυριότερη δυσκολία εντοπίστηκε στη διαχείριση:

- loop dependencies, οι οποίες περιόριζαν την επίτευξη επιθυμητών initiation intervals,
- violations κατά τη σύνθεση που απαιτούσαν αναδιοργάνωση του κώδικα.

Οι αλλαγές αυτές περιλαμβάνουν προσεκτική χρήση pragmas, αναδιάταξη loops, δημιουργία τοπικών buffers και άλλες τεχνικές που αναλύονται λεπτομερώς στις επόμενες ενότητες.

4.2 Μετατροπή kernels και έλεγχος ορθότητας

Η μετατροπή των μικροπυρήνων του SeisSol από AVX2 inline assembly σε καθαρό C++ αποτέλεσε το πρώτο κρίσιμο βήμα της διαδικασίας χαρτογράφησης στο Vitis HLS. Στόχος του σταδίου αυτού ήταν η δημιουργία μιας ισοδύναμης, αλλά απλοποιημένης υλοποίησης, η οποία θα ήταν κατάλληλη για High-Level Synthesis (HLS). Παράλληλα, αναπτύχθηκε ένα ολοκληρωμένο software reference testbench, μέσω του οποίου έγινε η επαλήθευση της αριθμητικής ορθότητας όλων των νέων υλοποιήσεων.

4.2.1 Εντοπισμός των kernels μέσω VTune

Η ανάλυση απόδοσης του πλήρους εκτελέσιμου SeisSol, που πραγματοποιήθηκε με το εργαλείο Intel VTune Profiler, αποκάλυψε ότι οι υπολογιστικά κρίσιμες συναρτήσεις που ευθύνονται για μεγάλο μέρος του χρόνου εκτέλεσης βρίσκονταν εντός του αρχείου subroutine.cpp, ως τμήμα του κώδικα που παράγεται αυτόματα κατά τη διαδικασία build του προγράμματος. Οι συναρτήσεις αυτές ήταν ορισμένες ως C++, αλλά περιείχαν εκτεταμένα τμήματα κώδικα AVX2 μέσω της εντολής __asm__ __volatile__, βασισμένα στη βιβλιοθήκη LIBXSMM. Οι συναρτήσεις αυτές εισάγονται από το SeisSol ως εξειδικευμένοι μικροπυρήνες matrix multiplication μικρού μεγέθους.

΄Οπως αναφέρθηκε και στο προηγούμενο κεφάλαιο, οι τρεις μικροπυρήνες LIBXSMM που εξετάζονται στην παρούσα εργασία είναι οι εξής:

- libxsmm_m12_n9_k20,
- libxsmm_m12_n9_k9,
- libxsmm_m20_n9_k10.

4.2.2 Μετατροπή από AVX2 inline assembly σε C++

Μετά την αναγνώριση των assembly kernels, ακολούθησε η λεπτομερής μελέτη της λειτουργίας τους. Η αρχική λογική των συναρτήσεων περιλαμβάνει:

- φόρτωση blocks του A και του B σε καταχωρητές,
- αναπαραγωγή του blocking σε ομάδες των 12 ή 20 στοιχείων,
- χρήση πολλαπλών συστοιχιών accumulators,
- unrolling βρόχων σε επίπεδο assembly.

Οι C++ εκδόσεις που αναπτύχθηκαν διατηρούν:

- τον ίδιο αριθμό και δομή accumulators,
- την ίδια διάταξη των loops,
- τους ίδιους δείκτες πρόσβασης στη μνήμη,
- και το ίδιο blocking του πίνακα C.

Στόχος ήταν η παραγωγή κώδικα όσο το δυνατόν πιο «καθαρού» ώστε να διευκολύνεται η βελτιστοποίηση μέσω HLS, χωρίς όμως να θυσιαστεί η αριθμητική ισοδυναμία σε σχέση με την αρχική AVX2 υλοποίηση.

4.2.3 Οργάνωση και υλοποίηση του testbench

Κατά τη ανάγνωση του αρχείου kernel.cpp, που παράχθηκε αυτόματα από το build του SeisSol, διαπιστώθηκε ότι οι τρεις μικροπυρήνες καλούνται από μεγάλο αριθμό συναρτήσεων. Ωστόσο, εντοπίστηκε μία συνάρτηση που καλούσε και τους τρεις εκτελούνται σειριακά ως εξής:

1. Ο πρώτος kernel υπολογίζει ενδιάμεσα αποτελέσματα.
2. Τα αποτελέσματα αυτά χρησιμοποιούνται ως είσοδος της συνάρτησης libxsmm_m12_n9_k9.

3. Ο τρίτος kernel (m20_n9_k10) χρησιμοποιεί την έξοδο του δεύτερου και ενημερώνει τον πίνακα αποτελέσματος.

Η «ροή δεδομένων» αυτή ήταν κρίσιμη στη διαδικασία δημιουργίας του testbench, καθώς η αριθμητική ορθότητα έπρεπε να επαληθευτεί σε κάθε στάδιο, αλλά και στο συνολικό αποτέλεσμα.

Για την επαλήθευση της ορθότητας δημιουργήθηκε νέο αρχείο C++ στον ίδιο φάκελο με τα subroutine.cpp και kernel.cpp. Το testbench περιλάμβανε τα ακόλουθα στοιχεία:

- εισαγωγή των αρχικών LIBXSMM assembly kernels,
- εισαγωγή των νέων C++ εκδόσεων,
- δημιουργία και αρχικοποίηση όλων των εισόδων των συναρτήσεων με πραγματικά δεδομένα από το SeisSol,
- συνάρτηση σύγκρισης πινάκων με tolerance 10^{-9},
- εκτέλεση και των δύο εκδόσεων κάθε kernel,
- εκτύπωση αποτελεσμάτων και επιβεβαίωση αριθμητικής ισοδυναμίας.

4.2.4 Αποτελέσματα επαλήθευσης

Μετά από σειρά δοκιμών με διαφορετικά σύνολα δεδομένων εισόδου, και οι τρεις C++ υλοποιήσεις παρήγαγαν αποτελέσματα ταυτόσημα (εντός του προκαθορισμένου tolerance) με τις αντίστοιχες LIBXSMM assembly εκδόσεις για όλους τους πίνακες εξόδου.

Τα αποτελέσματα αυτά διασφαλίζουν ότι οι C++ εκδόσεις μπορούν να χρησιμοποιηθούν με ασφάλεια ως reference υλοποιήσεις για την περιβάλλουσα διαδικασία HLS synthesis, όπως περιγράφεται στις επόμενες ενότητες.

4.3 Δημιουργία και ανάλυση project στο Vitis HLS

Το στάδιο αυτό είχε ως στόχο τη δημιουργία ενός ενοποιημένου περιβάλλοντος στο Vitis HLS για την αρχική υλοποίηση, σύνθεση και επαλήθευση των τριών μικροπυρήνων LIBXSMM που μελετήθηκαν. Η διαδικασία περιλάμβανε την προετοιμασία των C++ kernels, τη δημιουργία ενός κοινού testbench και την εκτέλεση των πρώτων συνθέσεων για τον εντοπισμό περιορισμών και πιθανών προβλημάτων.

4.3.1 Δημιουργία και ρύθμιση του αρχικού project

Το πρώτο project στο Vitis HLS δημιουργήθηκε με την ονομασία SeisSol_Functions και περιλάμβανε και τους τρεις μικροπυρήνες: libxsmm_m12_n9_k20, libxsmm_m12_n9_k9 και libxsmm_m20_n9_k10. Οι βασικές ρυθμίσεις που επιλέχθηκαν ήταν οι εξής:

- **Clock period:** 3 ns (στόχος λειτουργίας 300 MHz),
- **Target device:** xcu200-fsgd2104-2-e,
- **Part selection:** Alveo U200 data center accelerator card

Κάθε kernel ενσωματώθηκε στο project ως ανεξάρτητη C++ συνάρτηση. Τα δεδομένα εισόδου αντιγράφονται από τη global memory σε τοπικούς buffers (πίνακεςA_buffer, B_buffer, C_buffer), στους οποίους εκτελείται η βασική λογική υπολογισμού. Ο διαχωρισμός αυτός της φάσης ανάγνωσης/γραφής από τη φάση υπολογισμού απλοποιεί το μοντέλο μνήμης και διευκολύνει την μετέπειτα εφαρμογή βελτιστοποιήσεων (pipelining, unrolling κ.λπ.).

Δεδομένου ότι το Vitis HLS υποστηρίζει μόνο μία *top-level* συνάρτηση ανά σύνθεση, πραγματοποιήθηκαν τρεις αυτόνομες συνθέσεις, με κάθε μία από τις συναρτήσεις libxsmm_m12_n9_k20, libxsmm_m12_n9_k9 και libxsmm_m20_n9_k10 να ορίζεται διαδοχικά ως top function.

4.3.2 Οργάνωση του testbench και δήλωση των διεπαφών

Για την επαλήθευση των kernels δημιουργήθηκε ένα κοινό testbench, το οποίο αναπαρήγαγε τα δεδομένα εισόδου που χρησιμοποιήθηκαν κατά την αρχική software αξιολόγηση.

Το testbench καλεί διαδοχικά τους τρεις kernels αναπαράγοντας την πραγματική αλυσίδα υπολογισμών του SeisSol:

1. libxsmm_m12_n9_k20(fMrT_3, I, _buffer0),
2. libxsmm_m12_n9_k9(_buffer0, AplusT, _buffer1),
3. libxsmm_m20_n9_k10(rDivM_3, _buffer1, Q).

Παράλληλα, στο ίδιο αρχείο ορίστηκαν και οι πλήρως software εκδόσεις sw_libxsmm_m12_n9_k20, sw_libxsmm_m12_n9_k9 και sw_libxsmm_m20_n9_k10, οι οποίες χρησιμοποιούνται ως reference implementation. Μετά από κάθε κλήση του HLS kernel, το testbench εκτελεί την αντίστοιχη software υλοποίηση και συγκρίνει τα αποτελέσματα μέσω της συνάρτησης compare_arrays, με tolerance 10^{-9} σε double precision. Με αυτό τον τρόπο, εξασφαλίζεται ο συνεχής έλεγχος της ορθής αριθμητικής λειτουργίας των συναρτήσεων παρά τις μεταβολές στα αρχεία των kernels.

Κώδικας 4.1: Σύγκριση HLS και software υλοποίησης στο testbench.

```
size_t size = A2_SIZE;
libxsmm_m12_n9_k20(fMrT_3, I, _tmp0);
sw_libxsmm_m12_n9_k20(fMrT_3, I, _tmp2);

if (compare_arrays(_tmp0, _tmp2, size)) {
  std::cout << "libxsmm_m12_n9_k20 is CORRECT!" << std::endl;
} else {
  std::cout << "libxsmm_m12_n9_k20 is WRONG!" << std::endl;
}
```

4.3.2.0.1 Δήλωση διεπαφών AXI Για όλους τους kernels χρησιμοποιήθηκε ενιαίο σύστημα διεπαφών AXI:

- #pragma HLS INTERFACE m_axi για τους πίνακες εισόδου/εξόδου (A, B, C) με bundle = gmem,
- #pragma HLS INTERFACE s_axilite για τα control registers των A, B, C και της return τιμής με bundle = control.

Η διάταξη αυτή επιτρέπει στους kernels να χαρτογραφηθούν σε AXI master interfaces στο τελικό Vitis project, χωρίς να απαιτούνται αλλαγές στον κώδικα κατά τη μετάβαση από το στάδιο HLS στο στάδιο ολοκλήρωσης στο FPGA.

4.3.3 Αρχική σύνθεση των kernels και ανάλυση αποτελεσμάτων

Μετά την επιτυχή εκτέλεση της C Simulation, πραγματοποιήθηκε C/RTL synthesis για κάθε kernel ξεχωριστά. Τα αρχικά αποτελέσματα έδειξαν ότι και οι τρεις μικροπυρήνες επιτυγχάνουν σχετικά μικρούς χρόνους εκτέλεσης σε απόλυτο αριθμό κυκλών, αλλά παρουσιάζουν σημαντικούς περιορισμούς ως προς το Initiation Interval (II) των βασικών βρόχων, καθώς και μικρές αρνητικές τιμές timing slack ως προς τον στόχο των 3 ns.

4.3.3.1 Kernel libxsmm_m12_n9_k20

Για τον kernel libxsmm_m12_n9_k20 η αρχική σύνθεση παρήγαγε latency περίπου **1431 κύκλων** (περίπου 4.29 μs στα 3 ns), ενώ το Interval του κύριου βρόχου ήταν 1432 κύκλοι, γεγονός που υποδηλώνει ότι ο βρόχος δεν κατόρθωσε να γίνει ουσιαστικά pipelined (κάθε νέα επανάληψη ξεκινά μόνο όταν ολοκληρωθεί πλήρως η προηγούμενη). Η χρήση πόρων ήταν πολύ μικρή σε σχέση με τις δυνατότητες της U200 (ενδεικτικά 6 BRAM, 55 DSPs και περίπου 13k LUTs).

Στο HLS Guidance Report εμφανίζονται πολλαπλά μηνύματα **[HLS 200-880] II Violation** για τον βρόχο VITIS_LOOP_65_1, τα οποία υποδεικνύουν carried dependencies πάνω στις τοπικές μεταβλητές accum1[0] και accum1_load_1_0_out, καθώς και επιπλέον εξαρτήσεις κατά τη γραφή στον πίνακα C_buffer. Ο συνδυασμός αυτών των εξαρτήσεων δεν επιτρέπει στον συνθέτη να επιβάλει II=1, ακόμη και όταν Cητείται pipelining, οδηγώντας σε υψηλότερο II και αντίστοιχα μεγαλύτερο latency.

4.3.3.2 Kernel libxsmm_m12_n9_k9

Για τον kernel libxsmm_m12_n9_k9 η αρχική σύνθεση έδωσε latency περίπου **713 κύκλων** (2.14 μs), με Interval 714 κύκλων στον βασικό βρόχο mult. Η χρήση πόρων ήταν μεγαλύτερη σε σχέση με τον k20 (περίπου 6 BRAM, 66 DSPs, 42k LUTs), αλλά και πάλι πολύ μικρή σε σχέση με τις δυνατότητες του FPGA.

Η βασική αιτία για την αποτυχία επίτευξης χαμηλού II δεν ήταν εδώ τα data dependencies, αλλά οι **περιορισμοί στα memory ports**. Τα μηνύματα **[HLS 200-885] II Violation** αναφέρουν ότι ο συνθέτης αδυνατεί να προγραμματίσει τα load από τον πίνακα B_buffer και τα store στον C_buffer με II=1, λόγω περιορισμένου αριθμού θυρών πρόσβασης στην υλοποίηση μνήμης. Ενδεικτικά, καταγράφονται rescheduling constraints με II=1, 2, 3, 4, 11, 15 και 17 σε διάφορες πράξεις ανάγνωσης/γραφής στον C_buffer, κάτι που καθιστά σαφές ότι απαιτείται αναδιάταξη του κώδικα ή partitioning των πινάκων για την άρση του bottleneck.

4.3.3.3 Kernel libxsmm_m20_n9_k10

Ο kernel libxsmm_m20_n9_k10 παρουσιάζει αντίστοιχη εικόνα με τους δύο προηγούμενους: το latency ανέρχεται σε περίπου **1363 κύκλους** (4.09 μs), με Interval 1364 κύκλους για τον εσωτερικό βρόχο VITIS_LOOP_52_1. Η χρήση πόρων είναι

μέτρια (6 BRAM, 132 DSPs, 39k LUTs), γεγονός που επιβεβαιώνει ότι το κύριο Cήτημα δεν είναι η περιοχή, αλλά η δυσκολία επίτευξης μικρού II.

Τα μηνύματα **[HLS 200-885] II Violation** για τον συγκεκριμένο kernel εστιάζουν σχεδόν αποκλειστικά στον πίνακα C_buffer: η μονάδα μνήμης που αντιστοιχίζεται σε αυτόν δεν επαρκεί για να εξυπηρετήσει τον αριθμό ταυτόχρονων load και store ανά κύκλο, με αποτέλεσμα το εργαλείο να αναγκάζεται να αυξήσει το II (π.χ. σε 35, 51 ή 59 κύκλους για συγκεκριμένες εγγραφές). Ταυτόχρονα, το warning αναφέρει ρητά ως πιθανές λύσεις τη χρήση memory core με περισσότερες θύρες ή το partitioning του C_buffer.

4.3.4 Συμπεράσματα αρχικής αξιολόγησης

Στον Πίνακα 4.7 συνοψίζονται τα βασικά αποτελέσματα του πρώτου synthesis για τους τρεις μικροπυρήνες.

Πίνακας 4.1: Αρχικά αποτελέσματα σύνθεσης των original LIBXSMM kernels στο Vitis HLS

Kernel	Slack (ns)	Latency (cycles)	Interval (cycles)	DSP	DSP(%)	LUT	LUT(%)	FF	FF(%)	LUTs on SLR(%)
m12_n9_k20	-0.17	1431	1432	55	0.8	13735	1.2	35288	1.5	3
m12_n9_k9	-0.46	713	714	66	1.0	42403	3.6	49442	2.1	10
m20_n9_k10	-0.38	1363	1364	132	1.9	39644	3.4	50244	2.1	10

΄Οπως φαίνεται, η χρήση πόρων (BRAM, DSP, LUT) είναι ιδιαίτερα χαμηλή σε σχέση με τις δυνατότητες της Alveo U200, ωστόσο το II παραμένει πολύ υψηλό και για τους τρεις kernels, κάτι που οδηγεί σε αυξημένο latency παρά το σχετικά μικρό μέγεθος των πινάκων.

Από την αρχική ανάλυση των αποτελεσμάτων synthesis προκύπτουν δύο κύριες κατηγορίες περιορισμών:

- **Loop-carried εξαρτήσεις** πάνω σε accumulators και ενδιάμεσες μεταβλητές, που επηρεάζουν ιδιαίτερα τον libxsmm_m12_n9_k20 και δυσκολεύουν την εφαρμογή pipelining στον βασικό βρόχο.
- **Περιορισμοί σε memory ports** για τους τοπικούς πίνακες B_buffer και C_buffer, οι οποίοι αποτελούν το κύριο bottleneck για τους kernels libxsmm_m12_n9_k9 και libxsmm_m20_n9_k10.

Κοινό χαρακτηριστικό και των τριών kernels είναι ότι, παρά τη χαμηλή χρήση πόρων του FPGA, οι βασικοί βρόχοι δεν καταφέρνουν να επιτύχουν μικρό II, με αποτέλεσμα το συνολικό latency να παραμένει σημαντικά υψηλότερο από το θεωρητικά εφικτό. Τα ευρήματα αυτά οδήγησαν στην ανάγκη για πιο επιθετικές βελτιστοποιήσεις, όπως αναδιάταξη των loops, partitioning των τοπικών πινάκων και στοχευμένη χρήση pragmas, οι οποίες αναλύονται στην επόμενη ενότητα.

4.4 Αντιμετώπιση των violations και βελτιστοποιήσεις

4.4.1 Βασικά HLS pragmas και η λειτουργία τους

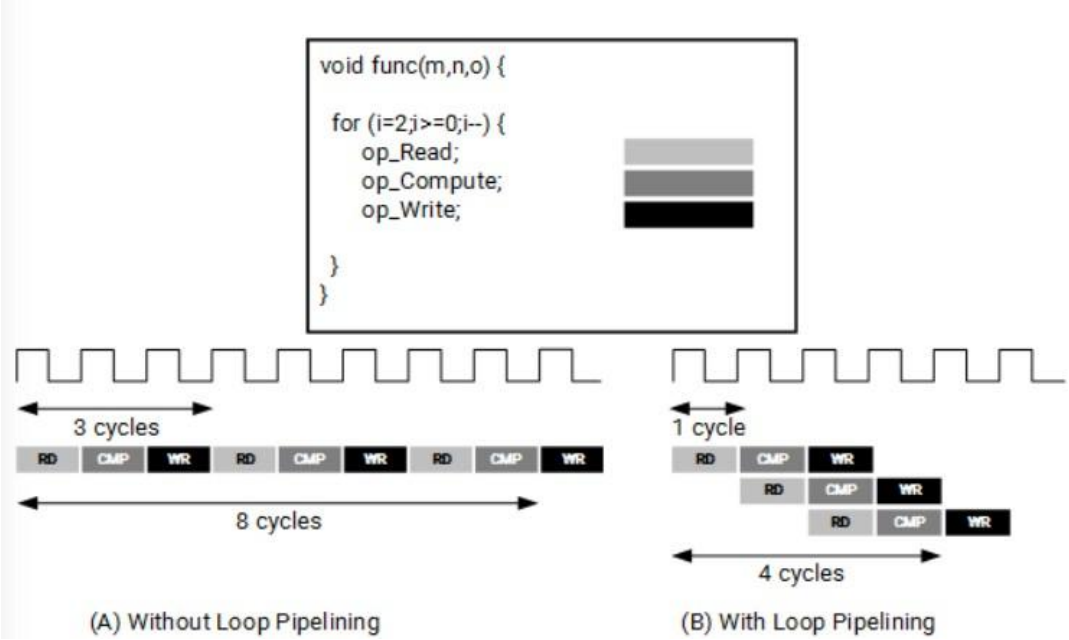
Πριν από την αντιμετώπιση των αρχικών προβλημάτων και την εφαρμογή των βελτιστοποιήσεων, είναι απαραίτητο να παρουσιαστούν συνοπτικά οι βασικές οδηγίες (pragmas)

του Vitis HLS που χρησιμοποιήθηκαν. Τα pragmas αποτελούν τον κύριο μηχανισμό καθοδήγησης του συνθέτη, επιτρέποντας τον έλεγχο της παράλληλης εκτέλεσης, της δομής των βρόχων, της οργάνωσης μνήμης και της εσωτερικής ιεραρχίας του παραγόμενου hardware ([27], Ch. 5: *HLS Pragmas*).

Στις παρακάτω παραγράφους συνοψίζονται οι οδηγίες που χρησιμοποιήθηκαν στην παρούσα εργασία, μαζί με τη λειτουργία και τις επιπτώσεις τους στη σύνθεση του κυκλώματος.

- **#pragma HLS PIPELINE II=1**

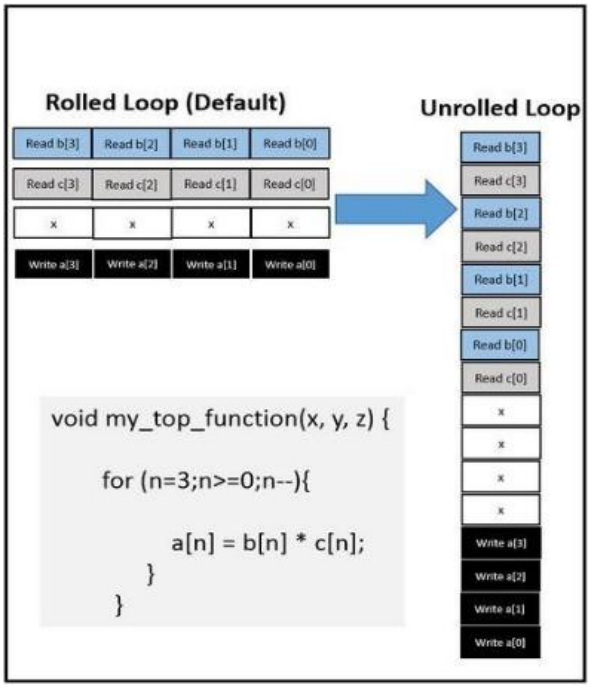
  Η οδηγία **PIPELINE** επιτρέπει στον συνθέτη να δημιουργήσει ένα pipeline σε επίπεδο βρόχου ή συνάρτησης, ώστε νέα επανάληψη να ξεκινά πριν ολοκληρωθεί η προηγούμενη. Στόχος είναι η επίτευξη Initiation Interval $II = 1$, δηλαδή η έναρξη μίας επανάληψης ανά κύκλο ρολογιού. Η επίτευξη $II = 1$ είναι δυνατή εφόσον δεν υπάρχουν loop-carried dependencies ή περιορισμοί σε πόρους μνήμης ([27], Sec. 5.5).



Σχήμα 4.1: Εφαρμογή pipeline σε βρόχο

- **#pragma HLS UNROLL**

  Η οδηγία **UNROLL** αναπαράγει πολλαπλά αντίγραφα ενός βρόχου δημιουργώντας παράλληλο hardware. Το unrolling αυξάνει τον υπολογιστικό παραλληλισμό, αλλά αυξάνει και τη χρήση πόρων (DSPs, LUTs), γι' αυτό και χρησιμοποιείται κυρίως σε εσωτερικούς βρόχους με μικρό trip count. Ο βρόχος μπορεί, επίσης, να γίνει μερικώς unrolled προσδιορίζοντας ένα παράγοντα (factor) N. ([27], Sec. 5.3).



Σχήμα 4.2: Εφαρμογή πλήρους unroll σε βρόχο

- **#pragma HLS ARRAY_PARTITION**

  Η οδηγία **ARRAY_PARTITION** αναλύει έναν πίνακα σε πολλαπλές τράπεζες μνήμης, ώστε να επιτρέπονται πολλαπλές ταυτόχρονες προσβάσεις. Είναι κρίσιμη όταν εμφανίζονται μηνύματα περιορισμένων memory ports ή μη δυνατότητας προγραμματισμού loads/stores στον ίδιο κύκλο. Ο default τύπος διαχωρισμού είναι ο complete, ο οποίος μετατρέπει έναν πίνακα σε μεμονομένα στοιχεία. ([27], Sec. 5.6).

- **#pragma HLS INLINE**

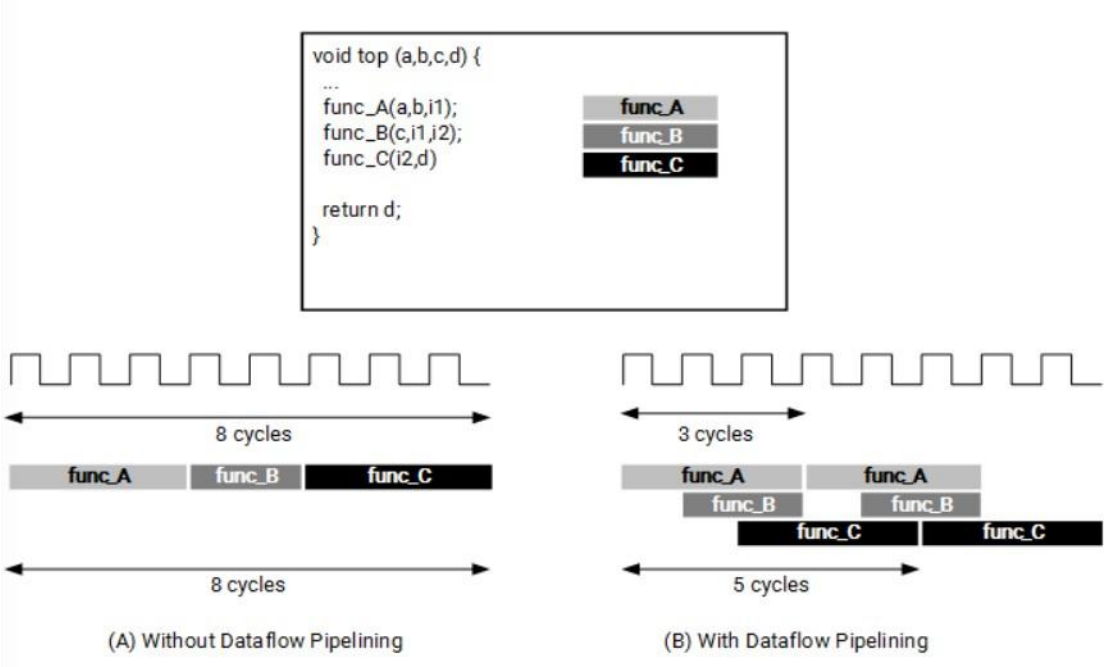
  Η οδηγία **INLINE** επιβάλλει την ενσωμάτωση του σώματος μίας συνάρτησης στο σημείο κλήσης της, μειώνοντας τη λειτουργική ιεραρχία και επιτρέποντας επιθετικότερες βελτιστοποιήσεις από τον συνθέτη ([27], Sec. 5.4).

- **#pragma HLS LOOP_TRIPCOUNT**

  Χρησιμοποιείται για να δοθεί στο εργαλείο μία εκτίμηση του πλήθους επαναλήψεων ενός βρόχου όταν αυτό δεν είναι στατικό. Δεν επηρεάζει την παραγόμενη αρχιτεκτονική, αλλά βοηθά στον ακριβέστερο υπολογισμό του latency ([27], Sec. 5.14).

- **#pragma HLS DATAFLOW**

  Η οδηγία **DATAFLOW** επιτρέπει την κατασκευή pipeline σε επίπεδο block χρησιμοποιώντας ανεξάρτητα στάδια (load, compute, store), τα οποία εκτελούνται ταυτόχρονα. Απαιτεί απουσία κύκλων εξάρτησης μεταξύ των σταδίων, καθώς και χρήση FIFOs ή streams για μεταφορά δεδομένων ([27], Sec. 5.7).



Σχήμα 4.3: Χρήση της οδηγίας dataflow

- **#pragma HLS DEPENDENCE**

  Η οδηγία **DEPENDENCE** ενημερώνει τον συνθέτη ότι κάποια πιθανή εξάρτηση (read-after-write, write-after-read, write-after-write) δεν είναι πραγματική. Η χρήση της πρέπει να γίνεται μόνο όταν υπάρχει απόλυτη βεβαιότητα, καθώς η λανθασμένη δήλωση εξάρτησης μπορεί να οδηγήσει σε λανθασμένο hardware ([27], Sec. 5.9).

- **#pragma HLS LOOP_FLATTEN**

  Η οδηγία **LOOP_FLATTEN** Cητά από τον συνθέτη να συνδυάσει δύο ή περισσότερους εμφωλευμένους βρόχους σε έναν ενιαίο βρόχο, δημιουργώντας μία ενιαία επίπεδη (flattened) επαναληπτική δομή. Η διαδικασία αυτή μπορεί να διευκολύνει την εφαρμογή pipelining όταν η εμφωλευμένη δομή δυσκολεύει την επίτευξη χαμηλού Initiation Interval, καθώς το εργαλείο χειρίζεται πλέον έναν μόνο βρόχο με απλούστερο σχήμα εξαρτήσεων. Ωστόσο, όταν ο βρόχος δεν είναι τέλειος (πχ. αν τα σώμα του βρόχου δε βρίσκεται εξολοκλήρου στον εσωτερικό βρόχο) μπορεί να χρησιμοποιηθεί η λέξη- κλειδί off, η οποία αποτρέπει το συνθέτη να συνδυάσει τους βρόχους.

4.4.2 Αντιμετώπιση των αρχικών violations

Μετά την αρχική σύνθεση των kernels στο Vitis HLS, διαπιστώθηκε ότι όλοι οι μικροπυρήνες παρουσίαζαν σοβαρούς περιορισμούς όσον αφορά το Initiation Interval(II), ακόμη και όταν χρησιμοποιούνταν οι βασικές οδηγίες PIPELINE και UNROLL. Η πλειονότητα των προβλημάτων οφειλόταν είτε σε *loop-carried dependencies* είτε σε περιορισμούς προσβάσεωνσε τοπικές μνήμες BRAM λόγω ανεπαρκών memory ports.

Λόγω της κοινής δομής που εμφανίζουν και οι τρεις LIBXSMM kernels, επιλέχθηκε να γίνει η αρχική διερεύνηση και αντιμετώπιση παραβιάσεων στη συνάρτηση libx-

smm_m12_n9_k9, ώστε οι λύσεις που θα προκύψουν να μπορούν στη συνέχεια να μεταφερθούν συστηματικά και στις υπόλοιπες.

4.4.2.1 Πρώτη προσπάθεια: ενοποίηση των accumulators

Η αρχική μορφή του kernel χρησιμοποιούσε εννέα ανεξάρτητους πίνακες accumulators (accum1–accum9), οι οποίοι ενημερώνονταν στο εσωτερικό του triply-nested loop. Για να απλοποιηθεί η δομή και να διευκολυνθεί το pipelining, επιχειρήθηκε η ενοποίηση σε έναν ενιαίο πίνακα 36 στοιχείων:

Κώδικας 4.2: Ενοποίηση των 9 accumulators σε έναν ενιαίο πίνακα.

```
for (int l = 0; l < 4; l++) {
    accum[0*4 + l] += temp1B * A[j*12 + l];
    accum[1*4 + l] += temp1B * A[j*12 + l + 4];
    accum[2*4 + l] += temp1B * A[j*12 + l + 8];

    accum[3*4 + l] += temp2B * A[j*12 + l];
    ...
    accum[8*4 + l] += temp3B * A[j*12 + l + 8];
}
```

Ακόμη, λόγω της επεξεργασίας των δεδομένων σε τριάδες, μεταβλήθηκε ελαφρώς ο βρόχος επανάληψης ώστε να αριθμεί εώς το 12, προσφέροντας τη δυνατότητα να μειωθεί το σώμα του loop ως εξής:

Κώδικας 4.3: Μεταβολή βρόχου επανάληψης.

```
            for (int l = 0; l < 12; l++) {
                // Update accumulators
                accum[l]    += temp1B * A[j * 12 + l];
                accum[l + 12] += temp2B * A[j * 12 + l];
                accum[l + 24] += temp1B * A[j * 12 + l];
            }
```

Η χρήση ενός ενιαίου πίνακα επέτρεψε την εφαρμογή της οδηγίας ARRAY_PARTITION complete, μειώνοντας τα memory-port conflicts. Ωστόσο, οι εγγραφές μορφής accum[l] += . . . δημιούργησαν ισχυρές **loop-carried εξαρτήσεις**, καθώς κάθε επανάληψη του βρόχου αναγκαζόταν να περιμένει την προηγούμενη για να φορτώσει την ενημερωμένη τιμή ([27], Σες. 5.9).

Ακόμη και με UNROLL σε διάφορους συντελεστές, και χρήση PIPELINE II=1, το εργαλείο εξακολουθούσε να εμφανίζει μηνύματα:

```
[HLS 200-880] unable to enforce a carried dependence constraint
```

λόγω των αναπόφευκτων RAW (Read-After-Write) dependencies.

4.4.2.2 Δεύτερη προσπάθεια: επανασχεδιασμός του εσωτερικού loop

Για να μειωθούν οι εξαρτήσεις, επιχειρήθηκε η μετατροπή του εσωτερικού βρόχου σε ενιαίο loop 36 επαναλήψεων, στο οποίο η στήλη, ο block δείκτης και το offset υπολογίζονταν μέσω integer arithmetic:

Κώδικας 4.4: Μετατροπή του εσωτερικού loop σε ενιαίο loop 36 επαναλήψεων.

```
for (int k = 0; k < 36; k++) {
```

```
    int b = k / 12;        // 0,1,2 -> choose tempB
    int c = (k % 12) / 4; // 0,1,2 -> block of the row
    int l = k % 4;         // 0..3  -> offset
    accum[k] += tempB[b] * A[j*12 + c*4 + l];
}
```

Παρά την ευθυγράμμιση του loop και την πλήρη ARRAY_PARTITION, οι εξαρτήσεις λόγω του += παρέμειναν, καθώς κάθε στοιχείο accum[k] ενημερώνεται σε κάθε επανάληψη του j-loop.

4.4.2.3 Ανεπιτυχείς προσπάθειες: ping–pong accumulators

Δοκιμάστηκε η χρήση δύο πινάκων (π.χ. accumA, accumB) με εναλλαγή ρόλων ανά επανάληψη, ώστε η μία δομή να χρησιμοποιείται ως input και η άλλη ως output. Στόχος ήταν η αποφυγή της άμεσης ανάγνωσης μιας τιμής που είχε μόλις γραφτεί.

Ωστόσο, οι εξαρτήσεις παρέμειναν, καθώς η ενημέρωση των accumulators απαιτεί την τιμή της προηγούμενης επανάληψης, άρα η διαδοχή READ ⟶ WRITE δεν μπορεί να αρθεί με απλή εναλλαγή buffers.

4.4.2.4 Καθοριστική λύση: read/write accumulators

Τελικά επιλέχθηκε μια πιο δομική λύση: η πλήρης διάσπαση των accumulators σε δύο πίνακες read_accum και write_accum. Η λογική είναι η εξής:

- Κατά την επανάληψη j, όλες οι προσπελάσεις ανάγνωσης γίνονται από τον read_accum.
- Όλες οι ενημερώσεις (+=) γίνονται στον write_accum.
- Στο τέλος κάθε j επανάληψης, το write_accum αντιγράφεται στον read_accum.

Το κρίσιμο loop τότε παίρνει τη μορφή:

Κώδικας 4.5: Read/write accumulators για άρση των RAW εξαρτήσεων.

```
for (int l = 0; l < 36; l++) {
#pragma HLS UNROLL factor=2
    int d = l / 12;
    int f = (l % 12) / 4;
    int g = l % 4;
    write_accum[l] = read_accum[l] + tempB[d] * local_A[j*12 + f*4 + g];
}

// Update read_accum for the next iteration
for (int l = 0; l < 36; l++) {
#pragma HLS PIPELINE
    read_accum[l] = write_accum[l];
}
```

Με τον τρόπο αυτό:

- αποτρέπεται πλήρως η άμεση loop-carried εξάρτηση,
- ο συνθέτης μπορεί πλέον να επιβάλει PIPELINE,

- το UNROLL είναι εφικτό, αλλά μόνο σε μικρούς συντελεστές, καθώς μεγάλα factors οδηγούν σε νέα violations,
- ο κώδικας παραμένει αριθμητικά ισοδύναμος.

4.4.2.5 Παράλληλη βελτιστοποίηση των memory accesses

Για να βελτιωθεί και η ροή δεδομένων, εφαρμόστηκαν:

- ARRAY_PARTITION complete για local_A, local_B, read_accum και write_accum,
- PIPELINE στα loops φόρτωσης και αποθήκευσης
- UNROLL στην εγγραφή του τοπικού πίνακα C με τις τιμές του write_accum

ώστε να ελαχιστοποιηθεί η πιθανότητα memory-port stalls και να υποστηριχθεί ο αυξημένος ρυθμός πρόσβασης που απαιτείται από το pipelined compute.

Η συγκεκριμένη δομή αποτέλεσε το πρώτο σταθερό σημείο του design space exploration: έναν kernel χωρίς violations και με επιτυχημένη εφαρμογή pipelining στα loops ανάγνωσης/εγγραφής, καθώς και στα εσωτερικά loops ενημέρωσης των accumulators. Στον παρακάτω πίνακα παρουσιάζονται τα αποτελέσματα σύνθεσης της πρώτης προσπάθειας του kernel 2:

Πίνακας 4.2: Αποτελέσματα 1ης προσπάθειας βελτιστοποίησης για τον kernel m12_n9_k9

Kernel	Slack (ns)	Latency (cycles)	Interval (cycles)	DSP	DSP%	LUT	LUT%	FF	FF%	LUTs on SLR%
m12_n9_k9 (v1)	–	2700	2701	198	2.9	47835	4.0	102085	4.3	12

4.4.3 Διερεύνηση επιπλέον τεχνικών και συμπεριφοράς των kernels

Εκτός από τις άμεσες βελτιστοποιήσεις που εφαρμόστηκαν για την άρση των αρχικών violations, πραγματοποιήθηκε μια σειρά από πειραματικές δοκιμές με στόχο τη διερεύνηση της συμπεριφοράς των kernels υπό διαφορετικές συνθήκες. Οι δοκιμές αυτές δεν οδήγησαν σε πρακτική βελτίωση της απόδοσης, αποτέλεσαν όμως σημαντικό τμήμα του design space exploration της εργασίας και συνέβαλαν στην τελική κατανόηση των ορίων και των δυνατοτήτων των συγκεκριμένων micro-kernels.

4.4.3.1 Αύξηση των διαστάσεων των πινάκων

Η πρώτη πειραματική προσέγγιση ήταν η αύξηση των διαστάσεων όλων των πινάκων (π.χ. 10× μεγαλύτεροι πίνακες), με στόχο:

- μεγαλύτερη κατανάλωση πόρων,
- μεγαλύτερη διάρκεια εκτέλεσης,
- πιο εμφανή επίδραση των βελτιστοποιήσεων.

Ωστόσο, οι kernels που προήλθαν από το LIBXSMM έχουν **απόλυτα εξειδικευμένη δομή**: όλες οι πράξεις, τα indices, τα offsets και τα block sizes ($12\times9\times9$ για το δεύτερο

kernel) είναι χειροκίνητα παραμετροποιημένα και ενσωματωμένα σε συγκεκριμένους υπολογιστικούς χάρτες. Η απλή αύξηση των διαστάσεων, όπως:

$$M = 120,\ N = 90,\ K = 90,$$

δεν αποτελεί απλή επέκταση μνήμης, αλλά ένα εντελώς διαφορετικό πρόβλημα GEMM που απαιτεί:

- νέα ανάλυση,
- νέα διάσπαση σε blocks,
- νέο mapping των παραμέτρων k, (row, col),
- αλλαγή των δεικτών σε ολόκληρο τον πυρήνα.

Ως εκ τούτου, μια τέτοια αλλαγή δε θεωρήθηκε βελτιστοποίηση, αλλά αλλαγή του ίδιου του αλγορίθμου, και έτσι απορρίφθηκε.

4.4.3.2 Μεταβολή της χρήσης πόρων

Για τη διερεύνηση του συστήματος scheduling, επιχειρήθηκε η ελαχιστοποίηση των χρησιμοποιούμενων πόρων, ακολουθούμενη από σταδιακή αύξησή τους. Ωστόσο, λόγω:

- του πολύ μικρού μεγέθους των πυρήνων,
- της ήδη χαμηλής κατανάλωσης LUTs, BRAMs και DSPs,
- της χρήσης πλήρους ARRAY_PARTITION στους τοπικούς πίνακες,

διαπιστώθηκε ότι η περιοχή **δεν αποτελεί bottleneck**. Η μείωση των πόρων είχε αρνητικό αποτέλεσμα, ενώ η αύξησή τους δεν προσέφερε πρόσθετη απόδοση. Έτσι, οι τελικές επιλογές διατήρησαν:

- ARRAY_PARTITION complete σε όλους τους μικρούς πίνακες,
- επιθετικό pipelining,
- μικρούς και στοχευμένους συντελεστές UNROLL.

4.4.3.3 Προσπάθεια γενίκευσης της συνάρτησης

Στη συνέχεια, δοκιμάστηκε η υλοποίηση μιας ενιαίας 'γενικής' συνάρτησης, η οποία θα μπορούσε να αντικαταστήσει και τους τρεις kernels, με τα μεγέθη των πινάκων να δίνονται ως ορίσματα. Θεωρητικά, κάτι τέτοιο ήταν εφικτό, καθώς όλα τα indices (π.χ. j*12 + f*4 + g) προκύπτουν από τις εκάστοτε διαστάσεις των πινάκων.

Ωστόσο, παρουσιάστηκαν δύο κύρια προβλήματα:

1. Ο τρίτος kernel (`m20_n9_k10`) χρησιμοποιεί **μη μηδενικό** πίνακα εξόδου ως είσοδο σε κάθε iteration, οπότε απαιτείται διαφορετική λογική accumulators.
2. Η γενίκευση ανάγκαζε και τους πυρήνες `m12_n9_k20` και `m12_n9_k9` να ακολουθούν μια περιττά περίπλοκη δομή, χωρίς κανένα όφελος σε απόδοση.

Εφόσον οι LIBXSMM micro-kernels είναι αυστηρά σχεδιασμένοι για συγκεκριμένες διαστάσεις και οι βελτιστοποιήσεις είναι στοχευμένες για τη μορφή τους, η δημιουργία

μιας γενικής συνάρτησης δεν θεωρήθηκε βελτιστοποίηση και η ιδέα εγκαταλείφθηκε.

4.4.3.4 Επανάληψη του kernel για μεγάλο αριθμό εκτελέσεων

Πραγματοποιήθηκε επίσης δοκιμή εισαγωγής ενός εξωτερικού βρόχου επαναλήψεων, ώστε ο kernel να εκτελείται 1–10 000 φορές με βήμα 1000. Στόχος ήταν η αύξηση του συνολικού runtime, ώστε να παρατηρηθεί η συμπεριφορά της συνάρτησης αναφορικά με τις επαναλήψεις και η διαφορά μεταξύ software - hardware να γίνεται πιο εμφανής.

Ωστόσο, εφόσον δεν εφαρμόστηκε pipelining στο εξωτερικό loop:

- Η χρήση πόρων παρέμεινε αμετάβλητη.
- Η ταχύτητα hardware/software αυξήθηκε γραμμικά ανάλογα με τον αριθμό των iterations.

Παρόλα αυτά, παρατηρήθηκε ότι γύρω στις **1000 επαναλήψεις** οι διαφορές μεταξύ hardware και software και η επίδραση των βελτιστοποιήσεων στο χρόνο γίνονταν πολύ πιο ευδιάκριτες. Έτσι, η επιλογή των **1000 iterations** διατηρήθηκε στις μετρήσεις των επόμενων υποενοτήτων.

4.4.4 Χρήση Streams και Dataflow για Pipeline σε επίπεδο Kernel

Ένα από τα βασικά πλεονεκτήματα του Vitis HLS είναι η δυνατότητα δημιουργίας pipeline σε επίπεδο kernel μέσω των `hls::stream` και της οδηγίας `DATAFLOW`. Τα streams λειτουργούν ως FIFO κανάλια στο εσωτερικό του kernel, επιτρέποντας σε διαφορετικά στάδια ενός αλγορίθμου (*load*, *compute*, *store*) να εκτελούνται ταυτόχρονα, εφόσον δεν υπάρχουν πραγματικές εξαρτήσεις δεδομένων ([27], Σες. 5.7).

Σκοπός της ενότητας αυτής ήταν να εξεταστεί κατά πόσο οι LIBXSMM micro-kernels μπορούν να επιταχυνθούν μέσω dataflow parallelism και να επιτευχθεί ταχύτερη ροή δεδομένων από την παγκόσμια μνήμη προς τα τοπικά buffers και πίσω.

Στις επόμενες παραγράφους παρουσιάζονται τρεις διαδοχικές εκδοχές του δεύτερου kernel, οι οποίες αξιοποιούν streams και dataflow με διαφορετικό βαθμό παραλληλισμού. Κάθε εκδοχή συνοδεύεται από ενδεικτικά αποσπάσματα κώδικα από τα αντίστοιχα αρχεία.

4.4.4.1 Πρώτη εκδοχή: απλό dataflow pipeline τριών σταδίων

Η πρώτη προσπάθεια βασίστηκε στην κλασική δομή:

$$\text{Read} \;\longrightarrow\; \text{Compute} \;\longrightarrow\; \text{Write},$$

με τρεις ξεχωριστές συναρτήσεις, καθεμία από τις οποίες υλοποιούσε ένα στάδιο του pipeline. Η τοπική συνάρτηση read_input αντέγραφε σειριακά τα δεδομένα των πινάκων A και B σε streams:

Κώδικας 4.6: Αποστολή δεδομένων A και B σε streams (v1).

```
// v1 read_input()  (from 12x9x9dataflow_v1.cpp)
for (int i = 0; i < A_SIZE; i++) {
#pragma HLS PIPELINE II=1
    A_stream << A[i];
}
for (int i = 0; i < B_SIZE; i++) {
#pragma HLS PIPELINE II=1
    B_stream << B[i];
}
```

Στη συνάρτηση compute, τα streams εκφορτώνονταν σε τοπικούς buffers και έπειτα εκτελούνταν η ίδια μαθηματική πράξη όπως στον αρχικό kernel, με read_accum και write_accum να ενημερώνονται εντός του loop:

Τέλος, το αποτέλεσμα προωθείται στον `C_stream` και από εκεί στη μνήμη.

Η δομή αυτή λειτούργησε σωστά και ήταν πλήρως συμβατή με dataflow. Ωστόσο, λόγω της απόλυτης εξάρτησης των υπολογισμών από ολόκληρο τον πίνακα A, δεν υπήρχε ουσιαστική επικάλυψη μεταξύ σταδίων (read–compute– write) και το performance κέρδος ήταν περιορισμένο.

4.4.4.2 Δεύτερη εκδοχή: dataflow ανά chunk

Για να ενισχυθεί η ταυτόχρονη εκτέλεση των σταδίων, στη δεύτερη εκδοχή έγινε προσπάθεια διάσπασης των πινάκων σε *chunks*. Κάθε chunk περιείχε:

- 27 στοιχεία από το B (ένα block ανά iteration),
- 36 στοιχεία από το C.

Η ανάγνωση γίνεται σε τρεις διαδοχικές φάσεις ανά chunk:

Κώδικας 4.7: Ανάγνωση A και B κατά chunks (v2).

```
for (int chunk = 0; chunk < NUM_CHUNKS; chunk++) {
    for (int i = 0; i < A_SIZE; i++)
        A_stream << A[i];       // the entire A panel in each iteration

    for (int i = 0; i < B_CHUNK; i++)
        B_stream << B[chunk * B_CHUNK + i];
}
```

Η ιδέα ήταν ότι η compute μονάδα θα μπορούσε να αρχίσει να επεξεργάζεται το chunk ̋0 πριν ολοκληρωθεί πλήρως η ανάγνωση των chunk ̋1 και ̋2.

΄Ομως διαπιστώθηκε ότι **κάθε chunk του C απαιτεί ολόκληρο τον πίνακα A.**

Επομένως:

- η chunk-based εκτέλεση δεν οδηγεί σε πραγματική επικάλυψη,
- το A πρέπει να μεταδοθεί πλήρως πριν από κάθε chunk,
- άρα ο μηχανισμός dataflow δεν αποδίδει τα μέγιστα.

4.4.4.3 Τρίτη εκδοχή: ξεχωριστά streams A και B με διαφορετικά AXI bundles

Στην τρίτη εκδοχή, έγινε ένα σημαντικό βήμα: η διάσπαση της ανάγνωσης σε δύο ανεξάρτητες συναρτήσεις `read_A` και `read_B`, με ξεχωριστά AXI master ports:

Κώδικας 4.8: Ξεχωριστά AXI bundles για A, B και C (v3).

```
#pragma HLS INTERFACE m_axi port=A bundle=gmem0
#pragma HLS INTERFACE m_axi port=B bundle=gmem1
#pragma HLS INTERFACE m_axi port=C bundle=gmem2
```

Αυτό επιτρέπει στο hardware να διαβάζει ταυτόχρονα από δύο διαφορετικά memory controllers, ενισχύοντας το parallel memory bandwidth.

Επιπλέον, ενοποιήθηκαν δύο loops:

- η αρχικοποίηση των αςσυμυλατορς,
- το πέρασμα `write_accum` → `read_accum`.

Κώδικας 4.9: Ενοποιημένο initialization και update (v3).

```
for(int l = 0; l < C_CHUNK; l++) {
#pragma HLS PIPELINE
    if (j == 0) {
        read_accum[l]  = 0;
        write_accum[l] = 0;
    } else {
        read_accum[l] = write_accum[l];
    }
}
```

Τέλος, εγκαταλείφθηκε πλήρως την ιδέα του chunk-based streaming της προηγούμενης έκδοσης και αντικαταστάθηκε με έναν πιο καθαρό dataflow πυρήνα.

Η έκδοση αυτή ήταν η πρώτη όπου παρατηρήθηκε ουσιαστική σταθεροποίηση του dataflow pipeline, κυρίως λόγω:

- ανεξάρτητων καναλιών ανάγνωσης,
- λιγότερων buffering loops,
- απλούστερου update των accumulators.

Παράλληλα, όλα τα buffers διατηρήθηκαν πλήρως partitioned ώστε τα εσωτερικά loops να μπορούν να κάνουν unroll και pipeline.

Τα αποτελέσματα της σύνθεσης ήταν τα εξής:

Πίνακας 4.3: Αποτελέσματα dataflow υλοποίησης για τον kernel m12_n9_k9

Kernel	Slack (ns)	Latency (cycles)	Interval (cycles)	DSP	DSP(%)	LUT	LUT(%)	FF	FF(%)	LUTs on SLR(%)
m12_n9_k9 (dataflow)	–	2598	2528	198	2.9	63307	5.0	91694	3.0	16

Η έκδοση αυτή αποτέλεσε το ιδανικό σημείο εκκίνησης για τις μεταγενέστερες βελτιστοποιήσεις με `uint512_t`, οι οποίες περιγράφονται στην επομένη ενότητα.

4.4.5 Βελτιστοποίηση μέσω χρήσης δεδομένων 512-bit (uint512_t)

Στο επόμενο στάδιο της βελτιστοποίησης αξιοποιήθηκαν λέξεις πλάτους 512 bits (uint512_t), οι οποίες επιτρέπουν τη συσκευασία (packing) μέχρι και 8 τιμών double σε έναν μόνο AXI transaction. Η τεχνική αυτή στοχεύει στη μείωση του αριθμού των προσβάσεων στη μνήμη και, κατά συνέπεια, στη βελτίωση του memory throughput. Η χρήση της εφαρμόστηκε πάνω στην τελευταία έκδοση των πυρήνων που χρησιμοποιούσε hls::stream και DATAFLOW .

Η τεχνική αυτή στοχεύει στη μείωση:

- των συνολικών AXI transactions,
- του latency ανά μεταφορά,
- της πίεσης στο memory bandwidth,

βελτιστοποιώντας τη ροή δεδομένων στο read_A, read_B και write_C στάδιο.

4.4.5.1 Αναπροσαρμογή διαστάσεων και βοηθητικές συναρτήσεις

Κάθε λέξη uint512_t περιέχει 8 στοιχεία double, επομένως οι πίνακες A, B και C κωδικοποιούνται σε μικρότερα arrays. Για παράδειγμα, για το δεύτερο kernel:

$$\text{A_SIZE} = 108(\text{doubles}) / 8 = 14 \text{ uint512_t}, \text{B_SIZE} = 81 / 8 = 11, \text{C_SIZE} = 108 / 8 = 14.$$

Επιπρόσθετα, χρησιμοποιούνται helper functions που επιτρέπουν τη μετατροπή στοιχείων double σε unsigned integer και αντίστροφα:

Κώδικας 4.10: Μετατροπείς double σε uint512_t και αντίστροφα.

```
inline double uint_to_double(din_type x) {
#pragma HLS INLINE
    union { uint64_t i; double f; } conv;
    conv.i = (uint64_t)x;
    return conv.f;
}

inline din_type double_to_uint(double x) {
#pragma HLS INLINE
    union { uint64_t i; double f; } conv;
    conv.f = x;
    return din_type(conv.i);
}
```

ενώ, παράλληλα δημιουργήθηκαν συναρτήσεις που πραγματοποιούν ανάγνωση και εγγραφή μεμονωμένων στοιχείων double εντός ενός uint512_t:

Κώδικας 4.11: Ανάγνωση double από uint512_t.

```
inline double get_double(const tl{uint512_dt}* buf, int index) {
    int word = index / tl{VECTOR_SIZE};
    int elem = index % tl{VECTOR_SIZE};
    tl{din_type} bits = buf[word].range(64*(elem+1)-1, 64*elem);
    return uint_to_double(bits);
}
```

Κώδικας 4.12: Εγγραφή double σε uint512_t.

```
inline void set_double(tl{uint512_dt}* buf, int index, double val) {
    int word = index / tl{VECTOR_SIZE};
    int elem = index % tl{VECTOR_SIZE};
    buf[word].range(64*(elem+1)-1, 64*elem) = double_to_uint(val);
}
```

Οι συναρτήσεις αυτές επιτρέπουν στο compute stage να παραμένει σε επίπεδο double-precision αριθμητικής, ενώ η μεταφορά δεδομένων μεταξύ kernels και μνήμης γίνεται μέσω packed 512-bit blocks.

4.4.5.2 Πρώτη εκδοχή με uint512_t: packing και stream-based dataflow

Στην πρώτη εκδοχή οι πίνακες A, B, C, καθώς και οι τοπικοί bufferskαι accumulators μετατράπηκαν σε arrays τύπου uint512_t.

Το στάδιο υπολογισμού είχε την εξής μορφή:

Κώδικας 4.13: Κεντρικός βρόχος της πρώτης εκδοχής.

```
double prev = (j == 0) ? 0.0 :
              get_double((const tl{uint512_dt}*)read_accum, l);

double a = get_double((const tl{uint512_dt}*)A_buf, indexA);
double b = get_double((const tl{uint512_dt}*)B_buf, indexB);
double sum = prev + a * b;

write_accum[l / VECTOR_SIZE].range(...) = double_to_uint(sum);
```

Η λογική των loops παρέμεινε η ίδια με την τελική εκδοχή της προηγούμεμης ενότητας. Τα streams συνέχισαν να μεταφέρουν words των 512 bits και το DATAFLOW pipeline read_A → read_B → compute → write_C λειτουργούσε ορθά.

Ωστόσο, εντοπίστηκε ένα σημαντικό Cήτημα : το C_stream ενημερωνόταν μέσα στο outer loop των chunks, δηλαδή 3 φορές, οδηγώντας σε πολλαπλή εγγραφή του αποτελέσματος.

4.4.5.3 Δεύτερη εκδοχή: διόρθωση της εξόδου και πλήρης συσχέτιση των accumulators

Στη δεύτερη εκδοχή, το μέγεθος των πινάκων read_accum και write_accum αυξήθηκε από:

$$C_CHUNK = 36 / 8 = 5$$

σε:

$$C_SIZE = 108 / 8 = 14,$$

ώστε να καλύπτει ολόκληρο το αποτέλεσμα του kernel.

Το νέο compute stage πήρε την παρακάτω μορφή:

Κώδικας 4.14: Δεύτερη Εκδοχή: πλήρης accumulator 14 words.

```
for (int l = 0; l < 36; l++) {
    int offset = chunk * 36 + l;
    double prev = (j == 0) ? 0.0 : get_double(read_accum, offset);
    double a   = get_double(A_buf, indexA);
    double b   = get_double(B_buf, indexB);
    double sum  = prev + a * b;

    set_double(write_accum, offset, sum);
}
```

Μετά την ολοκλήρωση υπολογισμού όλων των chunks, γίνεται μία και μοναδική εγγραφή ολόκληρου του C:

Κώδικας 4.15: Εγγραφή της εξόδου ως packed uint512.

```
for (int w = 0; w < tl{C_SIZE}; ++w) {
#pragma HLS PIPELINE II=1
    C_stream << write_accum[w];
}
```

Η σύνθεση αυτής της εκδοχής στο Vitis HLS έφερα τα παρακάτω αποτελέσματα:

Πίνακας 4.4: Αποτελέσματα dataflow υλοποίησης για τον kernel m12_n9_k9

Kernel	Slack (ns)	Latency (cycles)	Interval (cycles)	DSP	DSP(%)	LUT	LUT(%)	FF	FF(%)	LUTs on SLR(%)
m12_n9_k9 (dataflow)	–	1798	1728	23	0.3	85144	7.0	547321	23	21

4.4.5.4 Περιορισμοί: αναποτελεσματική εκμετάλλευση των 512-bit vectors

Παρά τη βελτίωση στο memory throughput, η συνολική επιτάχυνση παρέμεινε μικρότερη από την αναμενόμενη για δύο βασικούς λόγους:

1. **Πρόσβαση στα δεδομένα με μη συνεχόμενα patterns**

   Η αριθμητική του LIBXSMM kernel βασίζεται σε σύνθετους δείκτες:

   $$\text{indexA} = j * 12 + f * 4 + g, \qquad \text{indexB} = j + 9 * d + 27 * i,$$

   δηλαδή:

   - το A προσπελάζεται σε blocks των 12,
   - το B προσπελάζεται με offsets των 9 και 27,
   - το C ενημερώνεται σε διάσπαρτες θέσεις.

   Αυτό σημαίνει ότι τα 8 doubles που βρίσκονται σε μία λέξη uint512_t **δεν επεξεργάζονται ποτέ ταυτόχρονα**. Το compute loop πραγματοποιεί έναν μόνο πολλαπλασιασμό-άθροισμα ανά iteration, χρησιμοποιώντας μία μόνο «lane» κάθε φορά. Άρα το vectorization που θεωρητικά προσφέρει το 512-bit πλάτος δεν μπορεί να αξιοποιηθεί στην παρούσα αριθμητική δομή.

2. **Περιορισμένες ευκαιρίες για παράλληλο υπολογισμό**

   Εξαιτίας των εξαρτήσεων των συσσωρευτών (read_accum → write_accum), οι βρόχοι στο l μπορούσαν να γίνουν UNROLL μόνο με μικρούς factors. Παράλληλα, τα hls::stream δεν είναι σε θέση να προσφέρουν ουσιαστικό παραλληλισμό στο

στάδιο compute, καθώς τα δεδομένα των A_stream και B_stream πρέπει πρώτα να φορτωθούν πλήρως σε τοπικούς πίνακες πριν από τον πραγματικό υπολογισμό. Επιπλέον, η ανάγκη του αλγορίθμου για τυχαία πρόσβαση και επαναλαμβανόμενη ενημέρωση των στοιχείων του C μέσω των read_accum και write_accum δεν συμβαδίζει με τη FIFO φύση των streams, γεγονός που οδήγησε σε περιορισμένες ευκαιρίες για overlap και σε επαναλαμβανόμενα φαινόμενα υποαξιοποίησης των πόρων.

4.4.5.5 Δοκιμή χρήσης float αντί για double

Με σκοπό την παρατήρηση της συμπεριφοράς της συνάρτησης, δοκιμάστηκε και υλοποίηση με float, αντί για double. Ωστόσο, ο χρόνος εκτέλεσης, αν και μικρότερος, δεν μειώθηκε στο αναμενόμενο 1/2 του αρχικού. Οι λόγοι περιλαμβάνουν:

- το pipeline καθορίζεται κυρίως από εξαρτήσεις δεδομένων, όχι από το πλάτος τους,
- οι πίνακες ήταν ήδη συσκευασμένοι σε uint512_t, επομένως το μέγεθος του float δεν άλλαξε τον αριθμό των AXI transactions,
- το compute loop εξακολουθούσε να κάνει ένα μόνο FMA ανά iteration,
- το περιοριστικό στοιχείο ήταν το scheduling και όχι το throughput των functional units.

΄Αρα η μετάβαση σε float δεν μπορούσε να αποδώσει ουσιαστικό speedup στην παρούσα αρχιτεκτονική.

4.4.5.6 Συμπέρασμα και σύνδεση με την τελική βελτιστοποίηση

Η χρήση uint512_t βελτίωσε σημαντικά την απόδοση του memory subsystem και το χρόνο εκτέλεσης, αλλά δεν μπόρεσε να απελευθερώσει τον πλήρη παραλληλισμό του αλγορίθμου όσο τον περιορίζει:

- η FIFO φύση των streams,
- η αδυναμία επεξεργασίας και των 8 lanes ενός 512-bit vector σε κάθε iteration,
- η ανάγκη για επαναλαμβανόμενη και τυχαία πρόσβαση στα δεδομένα του C.

Το αποτέλεσμα ήταν ένα λειτουργικό αλλά όχι πλήρως παραλληλισμένο pipeline. Αυτά τα στοιχεία οδήγησαν στη μετάβαση στη **τελική έκδοση του πυρήνα**.

4.4.6 Τελική βελτιστοποίηση των LIBXSMM πυρήνων

Μετά από μια σειρά εκδόσεων με streams, DATAFLOW και uint512_t, η τελική αρχιτεκτονική των LIBXSMM πυρήνων σταθεροποιήθηκε σε μια κοινή μορφή: τα δεδομένα A, B, C φορτώνονται μία φορά σε πλήρως partitioned τοπικά buffers, ο υπολογισμός γίνεται σε επίπεδο λέξης 512 bits με χρήση βοηθητικών συναρτήσεων get_double, get_lane, set_lane, και οι συσσωρευτές του C διαχωρίζονται σε read_accum και write_accum ώστε να αρθούν οι loop-carried εξαρτήσεις. Σε αυτή την ενότητα παρουσιάζεται αναλυτικά ο πυρήνας m12_n9_k9, ο οποίος λειτουργεί ως «πρότυπο» της τελικής αρχιτεκτονικής, και

στη συνέχεια συνοψίζεται η προσαρμογή της ίδιας ιδέας στους πυρήνες m12_n9_k20 και m20_n9_k10.

4.4.6.1 Πυρήνας m12_n9_k9: αναλυτική τελική αρχιτεκτονική

Ο πυρήνας m12_n9_k9 είναι ο πιο αντιπροσωπευτικός ως προς τη δομή και τα μοτίβα πρόσβασης της αρχικής LIBXSMM υλοποίησης. Στην τελική έκδοση, οι πίνακες εισόδου/εξόδου κωδικοποιούνται σε λέξεις uint512_t, ώστε κάθε λέξη να περιέχει 8 στοιχεία double. Οι λογικές διαστάσεις (σε αριθμό 512-bit λέξεων) είναι, ενδεικτικά:

- A_SIZE = 14 (14 * 8 = 112 στοιχεία double, εκ των οποίων χρησιμοποιούνται τα 108),
- B_SIZE = 11 (11 * 8 = 88 στοιχεία, χρησιμοποιούνται τα 81),
- C_SIZE = 14 (14 * 8 = 112 στοιχεία, χρησιμοποιούνται τα 108).

Τα δεδομένα φορτώνονται μία φορά από την global memory σε τοπικά buffers που βρίσκονται σε on-chip μνήμη (BRAM / καταχωρητές) και είναι πλήρως partitioned. Στους βρόχους ανάγνωσης των δεδομένων χρησιμοποιείται το pragma HLS UNROLL:

Κώδικας 4.16: Τοπικά buffers και αρχική φόρτωση δεδομένων για τον πυρήνα m12_n9_k9.

```
#pragma HLS ARRAY_PARTITION variable=A_buf complete
#pragma HLS ARRAY_PARTITION variable=B_buf complete
#pragma HLS ARRAY_PARTITION variable=read_accum  complete
#pragma HLS ARRAY_PARTITION variable=write_accum complete

load_A:
for (int i = 0; i < A_SIZE; ++i) {
  #pragma HLS UNROLL
  A_buf[i] = A[i];
}

load_B:
for (int i = 0; i < B_SIZE; ++i) {
  #pragma HLS UNROLL
  B_buf[i] = B[i];
}
```

Ο βασικός υπολογισμός επαναλαμβάνεται 1000 φορές μέσα στο kernel, χωρίς επαναλαμβανόμενη εκκίνηση του kernel από τον host. Η επανάληψη αυτή χρησιμοποιήθηκε για την αύξηση του χρόνου εκτέλεσης τόσο στο software, όσο και στο harware, ώστε τα αποτελέσματα να είναι συγκρίσιμα.

Η αριθμητική δομή του m12_n9_k9 βασίζεται σε 108 τελικά στοιχεία εξόδου, τα οποία χωρίζονται σε 3 chunks των 36 στοιχείων έκαστο (CHUNK_LEN = 36). Για κάθε chunk επεξεργάζονται 9 στήλες του πίνακα B. Πριν από κάθε νέα στήλη j, τα περιεχόμενα του write_accum αντιγράφονται στο read_accum, ώστε όλες οι αναγνώσεις να γίνονται από την προηγούμενη εγγραφή και όλες οι εγγραφές στο write_accum. Αυτός ο διαχωρισμός άρει τις loop-carried εξαρτήσεις που εμπόδιζαν το pipelining στην αρχική υλοποίηση:

Κώδικας 4.17: Ενημέρωση συσσωρευτών ανά στήλη j.

```
compute_loop:
for (int chunk = 0; chunk < NUM_CHUNKS; ++chunk) {
  #pragma HLS UNROLL

loop_j:
  for (int j = 0; j < 9; ++j) {

  copy_accum:
    for (int w = 0; w < C_SIZE; ++w) {
      #pragma HLS UNROLL
      read_accum[w] = write_accum[w];
    }

    // ... computation ...
  }
}
```

Κάθε chunk καλύπτει CHUNK_LEN στοιχεία double του C, ξεκινώντας από base_double = chunk * CHUNK_LEN. Από αυτό προκύπτουν οι δείκτες των 512-bit λέξεων που πρέπει να επεξεργαστούν:

Κώδικας 4.18: Ορισμός περιοχής C που αντιστοιχεί σε κάθεchunk.

```
int base_double = chunk * CHUNK_LEN;            // 0, 36, 72
int last_double = base_double + CHUNK_LEN - 1; // 35, 71, 107
int first_word  = base_double / VECTOR_SIZE;
int last word   = last double / VECTOR SIZE;
```

Το εξωτερικό word_loop είναι PIPELINE II=1 και επεξεργάζεται διαδοχικά τις λέξεις του C που τέμνονται με το chunk. Για κάθε λέξη w, ο παλιός συσσωρευτής prev_word φορτώνεται από το read_accum, ενώ ο new_word αρχικοποιείται με την ίδια τιμή.

Στον εσωτερικό βρόχο inner_loop η μεταβλητή e διατρέχει τα 8 lanes της τρέχουσας 512-bit λέξης του C. Για κάθε συνδυασμό (w,e) υπολογίζεται αρχικά ο global_idx = w * 8 + e, δηλαδή το συνολικό μονοδιάστατο index του στοιχείου C ως προς την αρχή του πίνακα. Στη συνέχεια, ο l = global_idx - base_double εκφράζει τη θέση του στοιχείου μέσα στο τρέχον chunk μήκους CHUNK_LEN = 36. Εάν το l βρεθεί εκτός του εύρους [0, CHUNK_LEN), το αντίστοιχο lane δεν ανήκει στο συγκεκριμένο chunk και αγνοείται (continue).

Για τα ενεργά lanes, ο δείκτης l αποσυντίθεται σε τριάδα (d,f,g) σύμφωνα με τη δομή του LIBXSMM μικρο-πυρήνα m12_n9_k9: κάθε chunk των 36 στοιχείων αντιστοιχεί σε 3 γραμμές των 12 στοιχείων η καθεμία, με τις 12 στήλες να ομαδοποιούνται σε 3 blocks των 4 στοιχείων. Η σχέση d = l / 12 επιλέγει τη γραμμή (d = 0..2), η f = (l % 12) / 4 το block των 4 στοιχείων (f = 0..2) και η g = l % 4 τη θέση μέσα στο block (g = 0..3). Με βάση αυτά, ο δείκτης indexA = j * 12 + f * 4 + g επιλέγει το κατάλληλο στοιχείο του πίνακα A για τη στήλη j, ενώ ο indexB = chunk * 27 + (j + 9 * d) δείχνει στο στοιχείο B[d,j] εντός του αντίστοιχου 3×9 υποπίνακα του B. Τέλος, για κάθε lane υπολογίζεται sum = prev + a * b και η νέα τιμή γράφεται πίσω στο new_word μέσω της set_lane. Επειδή ο inner_loop είναι πλήρως UNROLL, όλα τα ενεργά lanes μιας λέξης ενημερώνονται παράλληλα σε έναν κύκλο του word_loop, επιτρέποντας στο HLS να εκμεταλλευτεί πλήρως το vectorization σε επίπεδο 512 bits.

Κώδικας 4.19: Ενημέρωση του C σε επίπεδο λέξης και lane για τον m12_n9_k9.

```
word_loop:
for (int w = first_word; w <= last_word; ++w) {
#pragma HLS UNROLL factor=2
  uint512_dt prev_word = read_accum[w];
  uint512_dt new_word  = prev_word;

inner_loop:
  for (int e = 0; e < VECTOR_SIZE; ++e) {
    #pragma HLS UNROLL
    int global_idx = w * VECTOR_SIZE + e;   // 0..111
    int l          = global_idx - base_double; // 0..35

    if (l < 0 || l >= CHUNK_LEN)
      continue;

    // mapping l -> (d,f,g)
    int f = (l % 12) / 4;   // divides row of 12 elements into groups of
       4
    int g = (l % 4);        // selects element in group
    int d = (l / 12);       // 0..2 (3 rows per chunk)

    int indexA = j * 12 + f * 4 + g;
    int indexB = chunk * 27 + (j + 9 * d);

    double prev = get_lane(prev_word, e);
    double a    = get_double(A_buf, indexA);
    double b    = get_double(B_buf, indexB);
    double sum  = prev + a * b;

    set_lane(new_word, e, sum);
  }

  write_accum[w] = new_word;
}
```

Μετά την ολοκλήρωση όλων των chunks και όλων των στηλών j, το write_accum περιέχει το τελικό C για την τρέχουσα επανάληψη iter και αντιγράφεται πίσω στο C_buf. Στο τέλος των 1000 επαναλήψεων, το C_buf γράφεται πίσω στην global memory.

Η αρχιτεκτονική αυτή:

- αντιμετωπίζει πλήρως τις εξαρτήσεις που δημιουργούσαν violations στην αρχική υλοποίηση,
- εκμεταλλεύεται το vectorization σε επίπεδο 512-bit λέξης μέσω UNROLL στο inner_loop και PIPELINE στο word_loop,
- διατηρεί πιστά τη μαθηματική συμπεριφορά του LIBXSMM πυρήνα m12_n9_k9, καθώς το mapping των δεικτών (chunk,j,l) σε (indexA,indexB) είναι ταυτόσημο με την αρχική υλοποίηση.

4.4.6.2 Προσαρμογή της αρχιτεκτονικής στον πυρήνα m12_n9_k20

Ο πυρήνας m12_n9_k20 (kernel 1) ακολουθεί ακριβώς την ίδια μικρο-αρχιτεκτονική με τον m12_n9_k9: χρήση uint512_t για τις μεταφορές δεδομένων, τοπικά πλήρως partitioned buffers A_buf, B_buf, C_buf, διαχωρισμός των συσσωρευτών σε read_accum/write_accum

και word-wise ενημέρωση του C μέσω get_lane/set_lane.

Η βασική διαφορά είναι οι διαστάσεις του προβλήματος και, κατά συνέπεια, η χαρτογράφηση του δείκτη l σε indexA και indexB. Ενώ στον m12_n9_k9 ο πίνακας εισόδου B έχει k = 9, στον m12_n9_k20 ο ίδιος πίνακας έχει k = 20, γεγονός που αυξάνει τον αριθμό των όρων που συμβάλλουν σε κάθε στοιχείο του C. Στο τελικό word-wise kernel, η δομή των loops (chunk, j, word_loop, inner_loop) παραμένει η ίδια, αλλά οι εξισώσεις για indexA και indexB προσαρμόζονται στα νέα μεγέθη.

Ενδεικτικά, ένα χαρακτηριστικό απόσπασμα του κώδικα στον πυρήνα m12_n9_k20 είναι:

Κώδικας 4.20: Ενδεικτικό mapping l σε δείκτες A και B για τον m12_n9_k20.

```
int f = (l % 20) / 4;
int g = (l % 4);
int d = (l / 20);

int indexA = j * 20 + f * 4 + g;
int indexB = chunk * 36 + (j + 12 * d);
```

το οποίο αντικαθιστά την αντίστοιχη χαρτογράφηση του kernel 2. Κατά τα λοιπά, η ροή δεδομένων, η χρήση των uint512_t και τα πραγμας UNROLL / PIPELINE είναι πλήρως ανάλογα με τον m12_n9_k9.

4.4.6.3 Πυρήνας m20_n9_k10: επέκταση σε μεγαλύτερη διάσταση m

Ο πυρήνας m20_n9_k10 (kernel 3) επεκτείνει την ίδια αρχιτεκτονική σε περίπτωση όπου η διάσταση m αυξάνεται σε 20, ενώ το n = 9 και το k = 10. Στην τελική υλοποίηση, ο πίνακας C περιέχει 180 στοιχεία double, τα οποία χωρίζονται σε 3 chunks των 60 στοιχείων (CHUNK_LEN = 60), και κωδικοποιούνται σε C_SIZE = 23 λέξεις uint512_t.

Η δομή των loops (chunk, j, word_loop, inner_loop) και η χρήση των read_accum/write_accum είναι κοινή με τον m12_n9_k9, με τη διαφορά πως ο πίνακας C περιέχει δεδομένα πριν τον υπολογισμό. Επομένως, προστέθηκε ένα for loop, το οποίο περνάει τα στοιχεία του C στο συσσωρευτή write_accum πριν το computation loop και κατά συνέπεια και στο read_accum μέσα στο loop j, ώστε να χρησιμοποιήθουν στον πολλαπλασιασμό.

Ωστόσο, η βασική διαφοροποίηση αφορά το πώς τα 60 στοιχεία ενός chunk αντιστοιχούν σε 3 γραμμές των 20 στοιχείων. Αυτό αποτυπώνεται στην παρακάτω χαρτογράφηση:

Κώδικας 4.21: Mapping l σε A και B για τον m20_n9_k10.

```
int f = (l % 20) / 4;
int g = (l % 4);
int d = (l / 20);

int indexA = j * 20 + f * 4 + g;
int indexB = (chunk * 36) + (j + 12 * d);
```

Αντίστοιχα, οι βοηθητικές συναρτήσεις get_double, get_lane, set_lane και το σχήμα ενημέρωσης:

- ανάγνωση της παλιάς τιμής του C από το read_accum,

- φόρτωση των αντίστοιχων A[indexA] και B[indexB],
- υπολογισμός sum = prev + a * b,
- εγγραφή της νέας τιμής στο write_accum,

είναι ταυτόσημα με αυτά που παρουσιάστηκαν λεπτομερώς για τον m12_n9_k9. Με αυτόν τον τρόπο, η ίδια μικρο-αρχιτεκτονική γενικεύεται σε διαφορετικές διαστάσεις m, n και k χωρίς να απαιτείται αλλαγή στη βασική ροή του κερνελ.

Οι τρεις kernels υποβλήθηκαν σε σύνθεση στο Vitis HLS παράγοντας τα παρακάτω αποτελέσματα:

Πίνακας 4.5: Τελικές εκδοχές των τριών kernels: επιδόσεις και εκτιμήσεις πόρων από Vitis HLS

Kernel	Slack (ns)	Latency (cycles)(min/max)	Interval (cycles)(min/max)	DSP	DSP(%)	LUT	LUT(%)	FF	FF(%)	LUTs on SLR(%)
m12_n9_k20	–	2986/3166	2987/3167	212	3	633760	53	224273	9	160
m12_n9_k9	–	1430/1511	1431/1512	212	3	625856	52	206332	8	158
m20_n9_k10	–	1600/1600	1601/1601	188	2	597790	50	175895	7	151

4.5 Βελτιστοποίηση και Υλοποίηση του πυρήνα derivativeTaylorExpansion

Η συνάρτηση derivativeTaylorExpansion::execute() αποτελεί έναν μικρού μεγέθους αλλά υπολογιστικά πυκνό πυρήνα του SeisSol, ο οποίος υλοποιεί μία επέκταση Taylor πρώτης τάξης για τέσσερις πίνακες παραγώγων dQ(0), dQ(1), dQ(2), dQ(3). Ο πυρήνας υπολογίζει τον πίνακα εξόδου I μεγέθους $20 \times 9 = 180$ στοιχείων μέσω της σχέσης:

$$I = p_0\, dQ_0 \;+\; p_1\, dQ_1 \;+\; p_2\, dQ_2 \;+\; p_3\, dQ_3,$$

όπου p_i είναι οι συντελεστές power(i). Η αρχική υλοποίηση στον CPU χρησιμοποιεί #pragma omp simd, το οποίο επιτρέπει την ταυτόχρονη εκτέλεση των επαναλήψεων του βρόχου:

Κώδικας 4.22: Βρόχοι υπολογισμού της συνάρτησης derivativeTaylorExpansion

```
for (int b = 0; b < 9; ++b) {
  #pragma omp simd
  for (int a = 0; a < 20; ++a) {
    I[20*b + a] = power(0) * dQ0[20*b + a];
  }
}
...
```

4.5.1 Αρχική Μεταφορά στο Vitis HLS και Προβλήματα II

Η απευθείας μεταφορά της συνάρτησης στο Vitis HLS δημιούργησε αμέσως υψηλό Initiation Interval (II) και εξαρτήσεις μεταξύ επαναλήψεων (loop-carried dependences). Ο λόγος είναι ότι κάθε επανάληψη του εσωτερικού βρόχου:

$$I[idx] \leftarrow I[idx] + (\cdot)$$

προκαλεί ένα *read-modify-write* μοτίβο στο ίδιο στοιχείο, το οποίο δημιουργεί αναδρομική εξάρτηση μεταξύ επαναλήψεων. Το HLS, μη μπορώντας να εκτελέσει δύο iterations παράλληλα, αναγκάστηκε να αυξήσει το II, κάτι που καθιστά τον πυρήνα πρακτικά μη παραλληλοποιήσιμο.

Έτσι, ο κύριος στόχος ήταν να εξαλειφθεί το RMW μοτίβο και να επιτραπεί πλήρης pipeline με II=1.

4.5.2 Version 2: Υλοποίηση με Τοπικά Buffers και Ενιαίο Compute Loop

Η πρώτη αποτελεσματική βελτιστοποίηση ήταν η μετάβαση σε σχεδίαση με τοπικά buffers.

Κώδικας 4.23: Ανάγνωση πινάκων σε buffers.

```
// Stage 1: LOAD (each in its own tight loop ? long AXI bursts)
  load_I:
  for (int i = 0; i < I_SIZE; ++i) {
#pragma HLS PIPELINE II=1
#pragma HLS LOOP_TRIPCOUNT min=I_SIZE max=I_SIZE
    I_buf[i] = I[i]; // initial values (used only if you intend to
       accumulate)
  }

  load_dQ0:
  for (int i = 0; i < DQ0_SIZE; ++i) {
#pragma HLS PIPELINE II=1
#pragma HLS LOOP_TRIPCOUNT min=DQ0_SIZE max=DQ0_SIZE
    dQ0_buf[i] = dQ0[i];
  }

  load_dQ1:
  for (int i = 0; i < DQ1_SIZE; ++i) {
#pragma HLS PIPELINE II=1
#pragma HLS LOOP_TRIPCOUNT min=DQ1_SIZE max=DQ1_SIZE
    dQ1_buf[i] = dQ1[i];
  }

  load_dQ2:
  for (int i = 0; i < DQ2_SIZE; ++i) {
#pragma HLS PIPELINE II=1
#pragma HLS LOOP_TRIPCOUNT min=DQ2_SIZE max=DQ2_SIZE
    dQ2_buf[i] = dQ2[i];
  }

  load_dQ3:
  for (int i = 0; i < DQ3_SIZE; ++i) {
#pragma HLS PIPELINE II=1
#pragma HLS LOOP_TRIPCOUNT min=DQ3_SIZE max=DQ3_SIZE
    dQ3_buf[i] = dQ3[i];
  }
```

Όλα τα δεδομένα dQ0, dQ1, dQ2, dQ3, καθώς και το I, φορτώθηκαν μία φορά σε τοπικούς πίνακες BRAM, ενώ εισήχθη μία ενιαία λούπα που υπολογίζει και τους τέσσερις όρους μέσω λογικών συνθηκών. Το αντίστοιχο τμήμα του κώδικα στο Version 2 έχει τη

μορφή:

Κώδικας 4.24: Νέο compute loop για τη συνάρτηση Taylor.

```
double acc = p0 * dQ0_buf[idxI];
if (a < 10) acc += p1 * dQ1_buf[idx1];
if (a < 4)  acc += p2 * dQ2_buf[idx2];
if (a == 0) acc += p3 * dQ3_buf[idx3];
I_buf[idxI] = acc;
```

Τα βασικά πλεονεκτήματα του σχεδιασμού είναι:

- Εξάλειψη των read-modify-write εξαρτήσεων.
- Δυνατότητα εφαρμογής pragma HLS PIPELINE II=1 στο εσωτερικό λοοπ.
- Ελαχιστοποίηση των προσβάσεων στη μνήμη: όλα τα δεδομένα βρίσκονται σε on-chip buffers.
- Σταθερή και προβλέψιμη λειτουργία σε hardware.

4.5.3 Version 3: Απόπειρα με hls::stream και DATAFLOW

Για να διερευνηθεί πιθανή επιπλέον επιτάχυνση μέσω παραλληλοποίησης μεταξύ των σταδίων φόρτωσης, υπολογισμού και αποθήκευσης, δοκιμάστηκε μια αρχιτεκτονική βασισμένη σε hls::stream και pragma HLS DATAFLOW. Στο Version 3 ο κώδικας χωρίστηκε σε:

- **read stage:** ανάγνωση power, dQ(i), I σε streams,
- **compute stage:** κατανάλωση των streams και παραγωγή νέων τιμών,
- **write stage:** εγγραφή των αποτελεσμάτων πίσω στη μνήμη.

Ωστόσο, η χρήση streams δεν προσέφερε ουσιαστικό όφελος, για τους εξής λόγους:

1. Όλα τα δεδομένα έπρεπε να φορτωθούν πρώτα σε buffers πριν αρχίσει ο υπολογισμός, λόγω τυχαίων προσβάσεων (random indexing). Επομένως, δεν επιτεύχθηκε πραγματικό overlap μεταξύ read/compute/write.
2. Ο πυρήνας είναι πολύ μικρού μεγέθους (180 στοιχεία), οπότε ο υπολογισμός είναι ήδη γρηγορότερος από την καθυστέρηση του AXI μεταφοράς.
3. Η εισαγωγή streams αύξησε την πολυπλοκότητα χωρίς να μειώσει το latency.

Συνεπώς, το Version 3 απορρίφθηκε.

4.5.4 Τελική Επιλογή: Η Βέλτιστη Buffer-Based Υλοποίηση

Η τελική υλοποίηση του πυρήνα είναι η buffer-based εκδοχή του Version 2. Με αυτή τη σχεδίαση:

- το εσωτερικό loop πετυχαίνει II=1,
- όλες οι πράξεις γίνονται σε on-chip μνήμη,
- οι συνθήκες if(a<10), if(a<4), if(a==0) επιτρέπουν την ενοποίηση των τεσσάρων loops,

- οι μόνες προσβάσεις στη global memory είναι τα αρχικά load και το τελικό store.

Πρόκειται για μια εξαιρετικά αποδοτική και σταθερή αρχιτεκτονική, η οποία συνοδεύεται από μικρή κατανάλωση πόρων, υψηλή συχνότητα ρολογιού, και γραμμική κλιμάκωση.

4.5.5 Συνολική Αξιολόγηση

Ο πυρήνας Taylor αποτελεί χαρακτηριστικό παράδειγμα όπου οι κλασικές τεχνικές βελτιστοποίησης (buffers, pipeline, consolidation loops) υπερέχουν σημαντικά έναντι των dataflow/stream-based υλοποιήσεων, και όπου η χρήση uint512 δεν έχει πραγματική αξία. Η τελική υλοποίηση είναι απλή, σταθερή, χρονικά αποδοτική και με άριστη συσχέτιση μεταξύ software και hardware αποτελεσμάτων. Η σύνθεση των παραπάνω εκδοχών στο Vitis HLS έφερε τα εξής αποτελέσματα:

Πίνακας 4.6: Αποτελέσματα της σύνθεσης της συνάρτησης Taylor στο Vitis HLS

Kernel	Slack (ns)	Latency (cycles)	Interval (cycles)	DSP	DSP(%)	LUT	LUT(%)	FF	FF(%)	LUTs on SLR(%)
original	–	1285	1286	11	0~	1513	0~	1884	0~	0~
version 2	–	669	670	56	0~	24247	2	24393	1	6
version 3	–	815	810	1377	20	51925	4	141192	5	13

4.6 Ενσωμάτωση των πυρήνων στο Vitis και εκτέλεση στο FPGA

Στο τμήμα αυτό περιγράφεται η διαδικασία ενσωμάτωσης των βελτιστοποιημένων πυρήνων στο πρόγραμμα Vitis, η δημιουργία των αντίστοιχων xclbin αρχείων για την πλατφόρμα Alveo U200 και, τέλος, η εκτέλεσή τους στο FPGA με τη χρήση XRT/OpenCL host προγραμμάτων.

΄Ολα τα πειράματα υλοποιήθηκαν στην πλατφόρμα xilinx_u200_gen3x16_xdma_2_202110_1, η οποία στοχεύει την κάρτα Alveo U200 Data Center Accelerator Card με FPGA τύπου xcu200-fsgd2104-2-e και διασύνδεση PCIe Gen3 x16. Η πλατφόρμα αυτή παρέχει έως και τέσσερα κανάλια DDR4-2400 SDRAM, τα οποία χαρτογραφούνται δυναμικά ανάλογα με τις απαιτήσεις των kernels, καθώς και υποσύστημα XDMA για τη μεταφορά δεδομένων μέσω PCIe.

4.6.1 Δημιουργία hardware kernel και αρχείου xclbin

Για κάθε ένα από τους τέσσερις kernels που μελετήθηκαν (libxsmm_m12_n9_k20, libxsmm_m12_n9_k9, libxsmm_m20_n9_k10 και derivativeTaylorExpansion) δημιουργήθηκε αρχικά ένα ξεχωριστό Vitis HLS project, στο οποίο γινόταν η σύνθεση (synthesis) και η επαλήθευση σε επίπεδο C/C++ και RTL. Μετά την επίτευξη αποδεκτών χαρακτηριστικών (πχ. II, χρήση πόρων, χρονισμός), ο κάθε κερνελ εξάγονταν ως hardware function σε μορφή .xo αρχείου.

Στο επόμενο βήμα, τα .xo αρχεία συνδέθηκαν (v++ –link) με την πλατφόρμα Alveo U200 για τη δημιουργία του τελικού xclbin:

- αρχικά, για κάθε kernel δημιουργήθηκε ένα ξεχωριστό xclbin (ένα kernel ανά bitstream),

- στη συνέχεια, δημιουργήθηκε ένα ενιαίο xclbin που περιλαμβάνει και τους τρεις kernels libxsmm_m12_n9_k20, libxsmm_m12_n9_k9 και libxsmm_m20_n9_k10 στο ίδιο δυναμικό τμήμα του FPGA.

Στις ρυθμίσεις του project των τριών kernel ορίστηκε διαφορετικό DDR και SLR για τον κάθε kernel:

Πίνακας 4.7: Ρυθμίσεις των compute units για τους τρεις kernels στο Vitis κατά το hardware build.

Kernel	Associated Memory Resource	Standard Logic Region
libxsmm_m12_n9_k20	DDR[0]	SLR0
libxsmm_m12_n9_k9	DDR[1]	SLR1
libxsmm_m20_n9_k10	DDR[3]	SLR2

Κατά τη σύνδεση του κοινού xclbin για τους τρεις LIBXSMM kernels, προέκυψαν αρχικά σφάλματα υλοποίησης (place_design ERROR) που σχετίζονταν με την κατανάλωση πόρων στη δυναμική περιοχή. Το χαρακτηριστικό μήνυμα σφάλματος από το εργαλείο v++ ανέφερε ότι η φάση impl_1 του Vivado απέτυχε να υλοποιήσει τη δυναμική περιοχή λόγω περιορισμών τοποθέτησης. Για την αντιμετώπιση του προβλήματος μειώθηκε η παραλληλία στο εσωτερικό των kernels: στο βρόχο word_loop αφαιρέθηκε το pragma HLS UNROLL factor=2 και αντικαταστάθηκε από pragma HLS PIPELINE II=1. Η αλλαγή αυτή μείωσε τη στιγμιαία χρήση πόρων (ιδίως LUTs και FFs) και επέτρεψε στο Vivado να ολοκληρώσει επιτυχώς τη φάση place & route για το κοινό xclbin.

Αντίθετα, η προσπάθεια να προστεθεί και ο τέταρτος κερνελ derivativeTaylorExpansion στο ίδιο xclbin απέτυχε, καθώς η συνολική απαίτηση σε πόρους της δυναμικής περιοχής υπερέβαινε τα διαθέσιμα όρια της πλατφόρμας. Για τον λόγο αυτό η Taylor συνάρτηση υλοποιήθηκε σε ξεχωριστό Vitis project, με δικό της xclbin.

4.6.2 Host πρόγραμμα και διαχείριση μνήμης

Η αλληλεπίδραση με το FPGA γίνεται μέσω host προγραμμάτων γραμμένων σε C++ με χρήση της βιβλιοθήκης XRT/OpenCL. Τα προγράμματα αυτά επιτελούν τις εξής βασικές λειτουργίες:

1. *Δέσμευση και αρχικοποίηση host μνήμης.* Για κάθε είσοδο και έξοδο του κερνελ δεσμεύονται std::vector<double, aligned_allocator<double» buffers, με ίδιες διαστάσεις και τιμές με τα αντίστοιχα δεδομένα του testbench στο Vitis HLS, έτσι ώστε η επαλήθευση hardware–software να είναι άμεση.

2. *Εκτέλεση σε CPU.* Στον host κώδικα υλοποιείται αυτούσια η software εκδοχή του κάθε κερνελ, η οποία εκτελείται πριν την κλήση του FPGA και παράγει τον πίνακα αναφοράς (source_sw_results). Για τους τρεις LIBXSMM kernels, οι υπολογισμοί αυτοί αντιστοιχούν στις αρχικές εκδόσεις με double accumulators, ενώ για τη Taylor συνάρτηση χρησιμοποιείται ο κώδικας της αρχικής CPU υλοποίησης.

3. *Φόρτωση του xclbin και δημιουργία OpenCL context.* Χρησιμοποιούνται οι βοηθητικές συναρτήσεις get_xil_devices() και read_binary_file() για την ανεύρεση διαθέσιμων συσκευών Xilinx και τη φόρτωση του xclbin. Δημιουργούνται cl::Context,

cl::CommandQueue και cl::Kernel αντικείμενα, ενεργοποιώντας το CL_QUEUE_PROFILING_ENABLE για τη μέτρηση χρόνων εκτέλεσης.

4. *Δέσμευση buffers στη global memory.* Για κάθε πίνακα εισόδου/εξόδου δημιουργείται ένα cl::Buffer με σημαία CL_MEM_USE_HOST_PTR, ώστε να αξιοποιείται άμεσα η ευθυγραμμισμένη host μνήμη. Για παράδειγμα, στο project της Taylor συνάρτησης δημιουργούνται buffers για τους πίνακες dQ0, dQ1, dQ2, dQ3 και I.

5. *Καθορισμός ορισμάτων του kernel.* Τα ορίσματα του kernel χαρτογραφούνται στα αντίστοιχα buffers. Για τη derivativeTaylorExpansion, τα power coefficients περνιούνται ως by-value s_axilite argument, ενώ οι πίνακες dQ0–dQ3 και I δίνονται ως m_axi.

6. *Μεταφορά δεδομένων προς τη συσκευή.* Μέσω της enqueueMigrateMemObjects() μεταφέρονται οι buffers εισόδου από την host μνήμη στη global memory της Alveo U200. Στα projects των τριών LIBXSMM kernels η διαδικασία πραγματοποιείται μόνο για τους πίνακες A, B για τους δύο πρώτους, ενώ για τον τρίτο kernel προστίθεται και ο αρχικός πίνακας C.

7. *Εκκίνηση του kernel.* Οι kernels εκκινούνται με enqueueTask(), καθώς πρόκειται για HLS kernels με σταθερό NDRange. Στο project με τους τρεις LIBXSMM kernels οι πυρήνες libxsmm_m12_n9_k20, libxsmm_m12_n9_k9 και libxsmm_m20_n9_k10 εκτελούνται παράλληλα, έχοντας εκκινηθεί από ξεχωριστές ουρές εντολών (q1, q2, q3).

8. *Μεταφορά αποτελεσμάτων πίσω στον host.* Μετά την ολοκλήρωση του kernel, τα αποτελέσματα από την global memory μεταφέρονται πίσω στην host μνήμη μέσω enqueueMigrateMemObjects(..., CL_MIGRATE_MEM_OBJECT_HOST) και q.finish(). Οι πίνακες source_hw_results περιέχουν πλέον τα αποτελέσματα του FPGA.

9. *Σύγκριση FPGA–CPU.* Τέλος, κάθε στοιχείο του πίνακα εξόδου που παρήχθη από το FPGA συγκρίνεται με το αντίστοιχο στοιχείο της CPU αναφοράς. Για τους τρεις LIBXSMM kernels χρησιμοποιείται αυστηρή σύγκριση ==, ενώ για τον kernel libxsmm_m20_n9_k10 εφαρμόζεται σχετική ανοχή λόγω της διαφορετικής σειράς συσσώρευσης, η οποία μπορεί να προκαλέσει μικρές διαφορές στρογγυλοποίησης.

Σε όλα τα host αρχεία χρησιμοποιήθηκε μια βοηθητική κλάση EventTimer για τη μέτρηση επιμέρους τμημάτων της εκτέλεσης, όπως Allocate Memory in Host Memory, Load Binary File to Alveo U200, Copy input data to device global memory, Launch the Kernel και Copy Result from Device Global Memory to Host Local Memory. Ο συνολικός χρόνος hardware υλοποίησης που αναφέρεται στο Κεφάλαιο 5 αντιστοιχεί στο χρονικό διάστημα από την εκκίνηση του πρώτου enqueueTask() μέχρι την ολοκλήρωση της μεταφοράς των αποτελεσμάτων από τη συσκευή προς τον host.

4.6.3 Πρώτες εκτελέσεις στο FPGA και επιβεβαίωση ορθότητας

Πριν από τη σύνθεση για hardware, όλα τα projects δοκιμάστηκαν σε C simulation (Vitis HLS), σε software emulation (sw_emu) και σε hardware emulation(hw_emu) εντός Vitis, ώστε να επιβεβαιωθεί ότι οι top-level συναρτήσεις και τα host προγράμματα είναι ορθά συνδεδεμένα.

Μετά τη δημιουργία του xclbin για την U200, πραγματοποιήθηκαν εκτελέσεις σε πραγματικό hardware (hw) για:

- κάθε πυρήνα ξεχωριστά (σε ανεξάρτητα projects), με σύγκριση hardware–software αποτελεσμάτων για έναν μικρό αριθμό χρονικών βημάτων, και
- το ενιαίο project με τους τρεις LIBXSMM kernels, όπου μετρήθηκε ο συνολικός χρόνος εκτέλεσης της αλυσίδας και επιβεβαιώθηκε ότι όλες οι έξοδοι είναι σύμφωνες με την αναφορά.

Σε όλες τις περιπτώσεις, οι πυρήνες παρήγαγαν αριθμητικά σωστά αποτελέσματα (εντός των ορίων ανοχής όπου αυτό ήταν αναγκαίο), επιβεβαιώνοντας ότι οι βελτιστοποιήσεις σε επίπεδο HLS (χρήση PIPELINE, UNROLL, ARRAY_PARTITION, uint512 κ.λπ.) δεν επηρέασαν την ορθότητα του αλγορίθμου. Στο επόμενο κεφάλαιο παρουσιάζονται τα πειραματικά αποτελέσματα και αξιολογείται κατά πόσο οι παραπάνω βελτιστοποιήσεις μεταφράζονται σε ουσιαστικό όφελος απόδοσης.

Κεφάλαιο 5

Αποτελέσματα και Αξιολόγηση Επιδόσεων

Στο παρόν κεφάλαιο παρουσιάζονται και αναλύονται τα πειραματικά αποτελέσματα που προέκυψαν από την εκτέλεση των υλοποιήσεων των επιλεγμένων συναρτήσεων τόσο σε επίπεδο λογισμικού (software) όσο και σε επίπεδο υλικού (FPGA). Αρχικά, εξετάζονται τα χρονικά αποτελέσματα για κάθε εκδοχή των kernels, με στόχο τη σύγκριση της απόδοσης μεταξύ software και hardware υλοποιήσεων. Στη συνέχεια, για τις τελικές εκδοχές των συναρτήσεων, καθώς και για το συνολικό project που περιλαμβάνει και τους τέσσερις kernels, παρουσιάζονται στοιχεία χρήσης πόρων και αρχιτεκτονικής ανάλυσης μέσω του εργαλείου Vitis Analyzer. Η ανάλυση αυτή επιτρέπει την εξαγωγή συμπερασμάτων σχετικά με την αποδοτικότητα, την κλιμακωσιμότητα και τους περιορισμούς της προτεινόμενης προσέγγισης.

5.1 Χρονικά αποτελέσματα software και hardware υλοποιήσεων

5.1.1 Μεθοδολογία μετρήσεων

Σημειώνεται ότι οι χρόνοι εκτέλεσης του software implementation προέρχονται πάντα από την ίδια αρχική υλοποίηση των συναρτήσεων σε C/C+ +, η οποία προέκυψε από τη μετατροπή των αντίστοιχων AVX2 assembly kernels. Παρότι η υλοποίηση παραμένει ίδια, παρατηρούνται διαφοροποιήσεις στους μετρούμενους χρόνους μεταξύ διαφορετικών εκτελέσεων, λόγω παραγόντων όπως το scheduling της CPU, η κατάσταση της cache και η ταυτόχρονη δραστηριότητα του συστήματος. Οι τιμές που παρουσιάζονται αντιστοιχούν σε αντιπροσωπευτικές μετρήσεις. ΄Ολοι οι χρόνοι που ακολουθούν εκφράζονται σε χιλιοστά του δευτερολέπτου (ms).

Για τις hardware υλοποιήσεις, ο μετρούμενος χρόνος αντιστοιχεί στο χρονικό διάστημα από την εκκίνηση του kernel (*kernel launch*) έως την ολοκλήρωση της αντιγραφής των αποτελεσμάτων από την παγκόσμια μνήμη του FPGA προς τη μνήμη του host (*copy back*). Με τον τρόπο αυτό, ο χρόνος περιλαμβάνει τόσο τον υπολογισμό όσο και το κόστος επικοινωνίας, παρέχοντας μια ρεαλιστική εκτίμηση της συνολικής επίδοσης.

5.1.2 Original συναρτήσεις

Στον Πίνακα 5.1 παρουσιάζονται οι χρόνοι εκτέλεσης των αρχικών συναρτήσεων (μία εκτέλεση), τόσο στο software όσο και στο hardware.

Table 5.1: Χρόνοι εκτέλεσης original συναρτήσεων σε software και hardware (1 εκτέλεση, ms)

Υλοποίηση	Kernel 1	Kernel 2	Kernel 3
Software	0.002	0.001	0.002
Hardware	0.357	0.347	0.408

Παρατηρείται ότι στην εκτέλεση μίας μόνο κλήσης ο χρόνος hardware είναι κατά πολύ μεγαλύτερος από τον software, γεγονός αναμενόμενο λόγω του σταθερού overhead εκκίνησης και μεταφοράς δεδομένων για το FPGA.

5.1.3 Πρώτες εκδοχές χωρίς violations (εκτελέσεις x1 και x1000 φορές)

Για να μειωθεί η επίδραση του σταθερού overhead και να γίνει πιο συγκρίσιμη η αξιολόγηση, οι συναρτήσεις εκτελέστηκαν τόσο μία φορά όσο και 1000 φορές, μέσω κατάλληλου βρόχου επανάληψης (`for`), όπως περιγράφηκε στο προηγούμενο κεφάλαιο. Τα αποτελέσματα παρουσιάζονται στον Πίνακα 5.2.

Table 5.2: Χρόνοι εκτέλεσης πρώτων εκδοχών χωρίς violations (1 και 1000 εκτελέσεις, ms)

Υλοποίηση	Kernel 1	Kernel 1 ×1000	Kernel 2	Kernel 2 ×1000	Kernel 3	Kernel 3 ×1000
Software	0.002	1.644	0.001	1.606	0.002	1.029
Hardware	0.399	27.798	0.321	14.845	0.420	15.382

Οι software χρόνοι κλιμακώνονται σχεδόν γραμμικά ως προς τον αριθμό επαναλήψεων, ενώ στο hardware, παρότι ο συνολικός χρόνος αυξάνεται, το κόστος ανά εκτέλεση μειώνεται καθώς το αρχικό overhead κατανέμεται σε περισσότερη εργασία.

5.1.4 Εκδοχές με `uint512` και dataflow (εκτελέσεις x1 και x1000 φορές)

Στη συνέχεια αξιολογούνται οι εκδοχές που αξιοποιούν packed μεταφορές δεδομένων τύπου `uint512` και αρχιτεκτονική dataflow. Τα αποτελέσματα παρουσιάζονται στον Πίνακα 5.3.

Table 5.3: Χρόνοι εκτέλεσης εκδοχών με `uint512` και dataflow (1 και 1000 εκτελέσεις, ms)

Υλοποίηση	Function 1	Function 2	Function 3	Function 1 ×1000	Function 2 ×1000	Function 3 ×1000
Software	0.002	0.002	0.003	6.103	1.738	2.349
Hardware	0.307	0.286	0.285	14.136	5.745	8.778

Η εισαγωγή dataflow και packed δεδομένων μειώνει το hardware κόστος σε σχέση με τις προηγούμενες εκδοχές, ειδικά στο σενάριο πολλαπλών επαναλήψεων, όπου το FPGA αξιοποιείται πιο αποδοτικά.

5.1.5 Τελικές εκδοχές: επίδραση compiler optimizations στο hardware (×1000)

Στις τελικές εκδοχές των συναρτήσεων, οι μετρήσεις πραγματοποιήθηκαν για 1000 επαναλήψεις. Το hardware έγινε build και αξιολογήθηκε με διαφορετικά επίπεδα compiler optimizations (`-O0`, `-O2`, `-O3`). Τα επίπεδα optimization ακολουθούν τη συνήθη σημασιολογία: το `-O0` αντιστοιχεί σε default/μη επιθετική βελτιστοποίηση, ενώ το `-O3` ενεργοποιεί πιο επιθετικούς μετασχηματισμούς με στόχο τη μέγιστη απόδοση, με πιθανή αύξηση απαιτήσεων σε πόρους και χρόνο μεταγλώττισης.

5.1.5.1 Function 1 (×1000)

Table 5.4: Τελικές εκδοχές Function 1 (×1000, ms) με hardware optimizations

Υλοποίηση	O0	O2	O3
Software	1.607	1.474	1.800
Hardware	9.968	9.854	9.741

5.1.5.2 Function 2 (×1000)

Table 5.5: Τελικές εκδοχές Function 2 (×1000, ms) με hardware optimizations

Υλοποίηση	O0	O2	O3
Software	0.654	0.726	0.666
Hardware	4.480	4.484	4.600

5.1.5.3 Function 3 (×1000)

Table 5.6: Τελικές εκδοχές Function 3 (×1000, ms) με hardware optimizations

Υλοποίηση	O0	O2	O3
Software	2.418	2.757	1.094
Hardware	5.564	5.607	5.667

Συνολικά, η επίδραση των optimizations στο hardware είναι μικρή σε απόλυτους χρόνους (της τάξης δεκάτων του ms για ×1000), ενώ σε ορισμένες περιπτώσεις η επιλογή optimization μπορεί να επηρεάσει αρνητικά (πχ. ελαφρώς αυξημένο χρόνο στο `-O3` για Function 2 και Function 3). Αυτό είναι συμβατό με το γεγονός ότι η συνολική επίδοση στο FPGA περιορίζεται όχι μόνο από την παραγόμενη αριθμητική λογική αλλά και από το overhead επικοινωνίας και το σύστημα μνήμης.

5.1.6 Software timings εκτός Vitis: επίδραση optimizations

Για να απομονωθεί η επίδραση των compiler optimizations στο software, οι συναρτήσεις, οι οποίες επαναλήφθηκαν x1000 φορές, χρονομετρήθηκαν εκτός Vitis με `-O0` και `-O3`. Τα αποτελέσματα παρουσιάζονται στον Πίνακα 5.7.

Table 5.7: Software timings εκτός Vitis: σύγκριση `-O0` και `-O3` (ms)

Συνάρτηση	O0 (ms)	O3 (ms)
Function 1	11.499	2.029
Function 2	2.663	1.046
Function 3	5.347	1.823

Παρατηρείται ότι στο software τα compiler optimizations έχουν ιδιαίτερα σημαντική επίδραση (πολλαπλάσια βελτίωση), γεγονός που αναδεικνύει ότι η CPU εκμεταλλεύεται αποτελεσματικά τις βελτιστοποιήσεις του compiler (vectorization, loop transformations, κ.λπ.), σε αντίθεση με την επίδραση τους στο hardware. Επίσης, σημειώνεται πως οι μετρήσεις στο hardware προσεγγίζουν αρκετά τις μετρήσεις του software για optimization -O0.

5.2 Αποτελέσματα σε επίπεδο συστήματος: Vitis project με τρεις kernels

5.2.1 Έκδοση `uint512` + dataflow: launch από ίδια/διαφορετικές ουρές (queues)

Στη συνέχεια αξιολογήθηκε Vitis project που περιλαμβάνει και τις τρεις συναρτήσεις. Για την έκδοση `uint512` + dataflow, μελετήθηκε η περίπτωση όπου οι kernels εκκινούνται από την ίδια ουρά εντολών σε σύγκριση με την περίπτωση όπου χρησιμοποιούνται διαφορετικές ουρές (*q1, q2, q3*), επιτρέποντας μεγαλύτερο βαθμό concurrency. Τα αποτελέσματα παρουσιάζονται στον Πίνακα 5.8.

Table 5.8: Project 3 kernels (v1): ίδια ουρά vs διαφορετικές ουρές (ms)

Υλοποίηση	Launch από ίδια ουρά	Launch από διαφορετικές ουρές
Software	5.528	4.119
Hardware	29.645	13.043

Η χρήση διαφορετικών queues στο hardware οδηγεί σε σημαντική μείωση του συνολικού χρόνου (από 29.645 ms σε 13.043 ms), γεγονός που υποδεικνύει ότι ο τρόπος scheduling και η δυνατότητα επικαλυπτόμενης εκτέλεσης/μεταφορών παίζουν καθοριστικό ρόλο στην επίδοση σε επίπεδο συστήματος.

5.2.2 Τελικές εκδοχές project 3 kernels (×1000): optimizations O0/O1/O2

Για τις τελικές εκδοχές του project με τρεις kernels, οι μετρήσεις πραγματοποιήθηκαν για 1000 επαναλήψεις και εξετάστηκαν διαφορετικά επίπεδα optimizations (`-O0`, `-O1`, `-O2`). Η δημιουργία hardware με `-O3` δεν ήταν εφικτή λόγω περιορισμών μνήμης κατά το build. Τα αποτελέσματα παρουσιάζονται στον Πίνακα 5.9.

Παρατηρείται ότι για το ολοκληρωμένο project το `-O1` και `-O2` οδηγούν σε καλύτερη hardware επίδοση σε σχέση με `-O0`, ενώ η αδυναμία build με `-O3` αναδεικνύει ότι σε

Table 5.9: Project 3 kernels (final, ×1000): σύγκριση optimizations (ms)

Υλοποίηση	O0	O1	O2
Software	3.496	3.654	3.768
Hardware	11.166	9.527	9.401

επίπεδο συστήματος η επιθετική βελτιστοποίηση μπορεί να αυξήσει τις απαιτήσεις σε πόρους (ιδίως μνήμη/σύνθεση), καθιστώντας μη βιώσιμη την υλοποίηση.

5.3 Συνάρτηση Taylor (Function 4): εκδοχές και optimizations

Τέλος, εξετάζεται η συνάρτηση Taylor, η οποία αποτελεί σύνθεση επιμέρους υπολογιστικών βημάτων και προσεγγίζει περισσότερο τη συνολική υπολογιστική ροή του SeisSol. Μελετήθηκαν δύο εκδοχές (v1 και v2), με μετρήσεις τόσο για μία εκτέλεση όσο και για 1000 επαναλήψεις, καθώς και επιπλέον μετρήσεις για compiler optimizations (`-O2` και `-O3`) στην τελική επιλογή.

Table 5.10: Function 4 (Taylor): σύγκριση εκδοχών και optimizations (ms)

Υλοποίηση	v1	v2	v1 ×1000	v2 ×1000	v1 ×1000 (O2)	v1 ×1000 (O3)
Software	0.001	0.001	0.546	0.548	0.664	0.566
Hardware	0.367	0.415	3.289	3.935	3.257	3.206

Τα αποτελέσματα δείχνουν ότι, σε επίπεδο μιας μόνο εκτέλεσης, το hardware παραμένει σημαντικά βραδύτερο λόγω overhead. Για 1000 επαναλήψεις, οι χρόνοι μειώνονται ανά εκτέλεση, ωστόσο η συνολική επίδοση εξακολουθεί να περιορίζεται από τη ροή δεδομένων και το κόστος επικοινωνίας. Επιπλέον, τα compiler optimizations στην επιλεγμένη εκδοχή επηρεάζουν μεν τον χρόνο, αλλά η βελτίωση στο hardware είναι περιορισμένη, υποδεικνύοντας ότι οι κύριοι περιορισμοί είναι αρχιτεκτονικοί (memory/overhead) και όχι μόνο υπολογιστικοί.

5.4 Συζήτηση Χρονισμών και Ανάλυση Απόδοσης

Οι χρόνοι εκτέλεσης που μετρήθηκαν για τις τελικές hardware υλοποιήσεις των kernels παρουσιάζουν διαφοροποιήσεις της τάξης των 3 ms, 4 ms, 5 ms και 9 ms, οι οποίες είναι συνεπείς με τα δομικά χαρακτηριστικά κάθε kernel και τον τρόπο εκτέλεσής τους στο σύστημα.

Ο συνολικός χρόνος εκτέλεσης ενός kernel στο FPGA μπορεί να προσεγγιστεί ως συνάρτηση του συνολικού αριθμού κύκλων και της συχνότητας ρολογιού, σύμφωνα με τη σχέση:

$$T \approx \frac{\text{Total Cycles}}{f_{\text{clk}}}$$

όπου οι συνολικοί κύκλοι καθορίζονται κυρίως από το πλήθος των επαναλήψεων των βρόχων (loop trip counts) και το Initiation Interval (II) στους κρίσιμους υπολογιστικούς βρόχους.

Για τους LIBXSMM kernels, ο συνολικός αριθμός κύκλων ανά εκτέλεση εξαρτάται άμεσα από το εύρος των εσωτερικών βρόχων (π. χ. πλήθος δεικτών j) και από τον αριθμό των αριθμητικών πράξεων που εκτελούνται ανά επανάληψη. Παρότι οι kernels παρουσιάζουν παρόμοια δομή, διαφέρουν ως προς το συνολικό υπολογιστικό φορτίο και τον βαθμό παραλληλισμού που μπορεί να επιτευχθεί, γεγονός που οδηγεί σε διαφορετικό συνολικό latency.

Στις περιπτώσεις kernels με μικρότερο πλήθος επαναλήψεων και καλύτερη εκμετάλλευση του διαθέσιμου παραλληλισμού, οι μετρημένοι χρόνοι κυμαίνονται στην περιοχή των 4 ms έως 5 ms. Αντίθετα, kernels με αυξημένο εύρος βρόχων και μεγαλύτερο αριθμό υπολογισμών ανά εκτέλεση παρουσιάζουν σχεδόν γραμμική αύξηση του συνολικού χρόνου, οδηγώντας σε χρόνους της τάξης των 9 ms. Η συμπεριφορά αυτή είναι αναμενόμενη, καθώς το συνολικό πλήθος κύκλων αυξάνεται σχεδόν αναλογικά με το υπολογιστικό φορτίο.

Αντίστοιχη λογική ισχύει και για τον πυρήνα Taylor, όπου το συνολικό υπολογιστικό έργο ανά εκτέλεση είναι ιδιαίτερα μικρό. Στην περίπτωση αυτή, ο χρόνος εκτέλεσης επηρεάζεται σε μεγάλο βαθμό από το κόστος ελέγχου και από τον αριθμό επαναλήψεων της κλήσης του kernel στο host πρόγραμμα. Καθώς ο πυρήνας εκτελείται πολλαπλές φορές εντός εξωτερικού βρόχου, ακόμη και μικρό latency ανά εκτέλεση συσσωρεύεται σε συνολικό χρόνο της τάξης μερικών χιλιοστών του δευτερολέπτου.

Αξίζει να σημειωθεί ότι οι αντίστοιχες υλοποιήσεις λογισμικού είναι ιδιαίτερα μικρού μεγέθους και ήδη εξαιρετικά γρήγορες, γεγονός που περιορίζει το περιθώριο υπέρτερης απόδοσης της hardware υλοποίησης. Συνεπώς, οι μετρημένοι χρόνοι εκτέλεσης των kernels στο FPGA ευθυγραμμίζονται πλήρως με τις εκτιμήσεις που προκύπτουν από τα χαρακτηριστικά χρονισμού, το πλήθος επαναλήψεων και τη δομή των υπολογισμών.

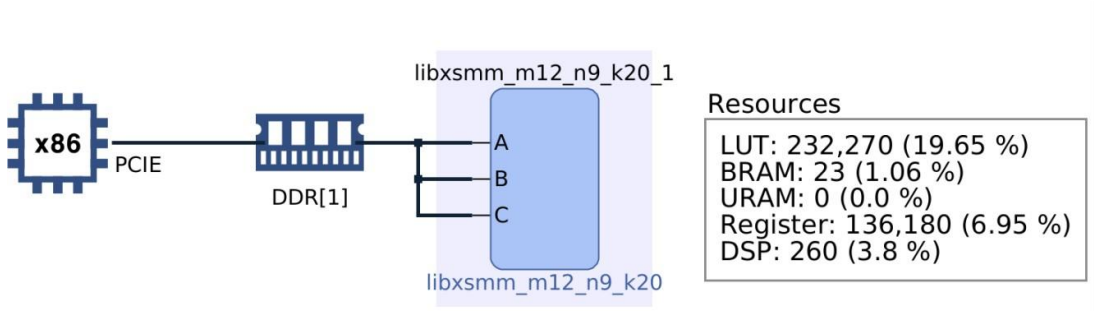
Συνολικά, τα αποτελέσματα καταδεικνύουν ότι η εκτέλεση μεμονωμένων μικροπυρήνων στο FPGA δεν είναι αποδοτική από άποψη χρόνου λόγω του σταθερού κόστους εκκίνησης και μεταφοράς δεδομένων. Ωστόσο, όταν οι συναρτήσεις εκτελούνται επαναληπτικά, η επίδραση του overhead μειώνεται και αναδεικνύεται η σημασία αρχιτεκτονικών βελτιστοποιήσεων, όπως η χρήση packed δεδομένων (`uint512`) και η οργάνωση dataflow. Σε επίπεδο συστήματος, η αξιοποίηση διαφορετικών queues προσφέρει σημαντικό όφελος, υπογραμμίζοντας τη σημασία του concurrency και του scheduling. Τέλος, η συνάρτηση Taylor λειτουργεί ως παράδειγμα φορτίου εργασίας σε επίπεδο εφαρμογής, και αναδεικνύει ότι για ουσιαστική επιτάχυνση απαιτείται περαιτέρω μείωση του communication overhead και καλύτερη ενσωμάτωση των kernels σε μεγαλύτερα υπολογιστικά τμήματα του SeisSol.

5.5 Ανάλυση τελικών εκδοχών μέσω Vitis Analyzer

5.5.1 Kernel 1 (libxsmm_m12_n9_k20)

Στην παρούσα ενότητα παρουσιάζονται τα αποτελέσματα ανάλυσης της τελικής εκδοχής του Kernel 1 (libxsmm_m12_n9_k20) μέσω του εργαλείου Vitis Analyzer. Η ανάλυση αυτή επιτρέπει τη διερεύνηση της αρχιτεκτονικής υλοποίησης σε επίπεδο συστήματος, καθώς και την αποτίμηση της χρήσης πόρων και της κατανομής της λογικής στο FPGA.

Το system diagram του Kernel 1 παρουσιάζεται στο Σχήμα 5.1. Ο kernel συνδέεται με τον x86 host μέσω διασύνδεσης PCIe και επικοινωνεί με την παγκόσμια μνήμη DDR για την ανάγνωση των πινάκων εισόδου A και B, καθώς και για την εγγραφή του πίνακα εξόδου C. Η δομή αυτή αντιστοιχεί σε μια τυπική αρχιτεκτονική offload, όπου ο υπολογισμός εκτελείται αποκλειστικά στο FPGA, ενώ ο host αναλαμβάνει τη διαχείριση της ροής εκτέλεσης και των μεταφορών δεδομένων. Η απουσία πρόσθετων εσωτερικών διασυνδέσεων υποδηλώνει ότι ο kernel λειτουργεί ως ανεξάρτητος accelerator.



Σχήμα 5.1: System diagram για τον Kernel 1 από το Vitis Analyzer

Η χρήση πόρων του accelerator για τον Kernel 1 παρουσιάζεται στα Σχήματα 5.2 και 5.3, τόσο σε απόλυτες τιμές όσο και ως ποσοστό του διαθέσιμου user budget. Συγκεκριμένα, ο kernel καταναλώνει 237 522 LUTs (23.91%), 135 607 registers (6.42%), 23 BRAMs (1.20%) και 260 DSPs (3.81%), ενώ δεν χρησιμοποιούνται URAMs. Η σχετικά χαμηλή χρήση DSPs υποδεικνύει ότι η υλοποίηση δεν περιορίζεται από την αριθμητική ισχύ, αλλά κυρίως από τη λογική ελέγχου και τη δομή των διασυνδέσεων. Αντίθετα, η αυξημένη κατανάλωση LUTs είναι ενδεικτική της πολυπλοκότητας της υλοποίησης και των μηχανισμών που απαιτούνται για την υποστήριξη της ροής δεδομένων και του ελέγχου.

Name	LUT	LUTAsMem	REG	BRAM	URAM	DSP
Platform	188813	19398	250809	251	20	7
User Budget	993427	572442	2113671	1909	940	6833
Used Resources	237522	3954	135607	23	0	260
Unused Resources	755905	568488	1978064	1886	940	6573
libxsmm_m12_n9_k20 (1)	237522	3954	135607	23	0	260
libxsmm_m12_n9_k20_1	237522	3954	135607	23	0	260

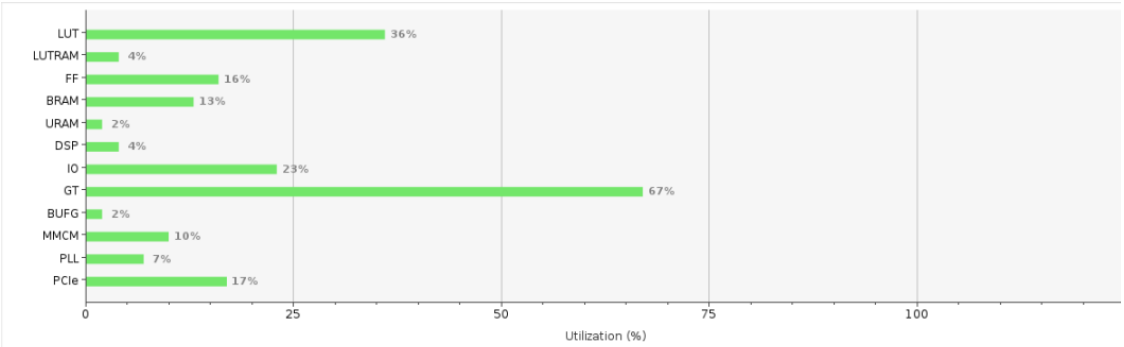
Σχήμα 5.2: Χρήση πόρων accelerator για τον Kernel 1: απόλυτες τιμές

Name	LUT	LUTAsMem	REG	BRAM	URAM	DSP
Platform	15.97%	3.28%	10.61%	11.62%	2.08%	0.10%
User Budget	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Used Resources	23.91%	0.69%	6.42%	1.20%	0.00%	3.81%
Unused Resources	76.09%	99.31%	93.58%	98.80%	100.00%	96.19%
libxsmm_m12_n9_k20 (1)	23.91%	0.69%	6.42%	1.20%	0.00%	3.81%
libxsmm_m12_n9_k20_1	23.91%	0.69%	6.42%	1.20%	0.00%	3.81%

Σχήμα 5.3: Χρήση πόρων accelerator για τον Kernel 1: ποσοστά επί του User Budget

Η συνολική χρήση πόρων της συσκευής παρουσιάζεται στο Σχήμα 5.4. Σε επίπεδο FPGA, παρατηρείται ότι η χρήση LUT ανέρχεται περίπου στο 36%, ενώ η χρήση FF

φτάνει το 16%. Η χρήση BRAM κυμαίνεται γύρω στο 13%, ενώ τα DSPs παραμένουν σε χαμηλά επίπεδα, περίπου στο 4%. Ιδιαίτερο ενδιαφέρον παρουσιάζει η αυξημένη αξιοποίηση των GT πόρων (περίπου 67%), καθώς και των PCIe-σχετικών υποδομών, γεγονός που συνδέεται άμεσα με τη φύση της αρχιτεκτονικής host–device και το κόστος επικοινωνίας. Τα αποτελέσματα αυτά δείχνουν ότι, παρότι ο kernel από μόνος του δεν εξαντλεί τους διαθέσιμους υπολογιστικούς πόρους, η συνολική υποδομή επικοινωνίας επιβαρύνει σημαντικά συγκεκριμένες κατηγορίες πόρων.



Resource	Utilization	Available	Utilization %
LUT	426335	1182240	36.06
LUTRAM	23352	591840	3.95
FF	386411	2364480	16.34
BRAM	274.50	2160	12.71
URAM	20	960	2.08
DSP	267	6840	3.90
IO	157	676	23.22
GT	16	24	66.67
BUFG	44	1800	2.44
MMCM	3	30	10.00
PLL	4	60	6.67
PCIe	1	6	16.67

Σχήμα 5.4: Συνολική χρήση πόρων συσκευής για τον Kernel 1

Η ανάλυση post-placement SLR connectivity, η οποία παρουσιάζεται στο Σχήμα 5.5, αποκαλύπτει ότι η λογική του Kernel 1 κατανέμεται σε περισσότερα από ένα SLRs. Παρατηρείται σημαντικός αριθμός διασυνδέσεων μεταξύ των SLR0, SLR1 και SLR2, με χιλιάδες συνδέσεις που διέρχονται από τα όρια των SLRs. Η εκτεταμένη αυτή διασπορά της λογικής αυξάνει την πολυπλοκότητα της δρομολόγησης και μπορεί να επηρεάσει αρνητικά τόσο τη χρονική επίδοση όσο και τη δυνατότητα κλιμάκωσης της υλοποίησης. Το εύρημα αυτό είναι ιδιαίτερα σημαντικό, καθώς προϊδεάζει για τις δυσκολίες που προκύπτουν κατά την ενσωμάτωση πολλαπλών kernels στο ίδιο FPGA, όπου οι απαιτήσεις σε SLR-to-SLR επικοινωνία αυξάνονται περαιτέρω.

Συνοψίζοντας, η ανάλυση του Kernel 1 μέσω του Vitis Analyzer δείχνει ότι, ενώ η κατανάλωση αριθμητικών πόρων παραμένει περιορισμένη, η χρήση λογικών πόρων και η κατανομή τους σε πολλαπλά SLRs αποτελούν τους κύριους περιοριστικούς παράγοντες. Τα αποτελέσματα αυτά εξηγούν εν μέρει τις επιδόσεις που παρατηρήθηκαν στα χρονικά πειράματα και παρέχουν το απαραίτητο υπόβαθρο για τη σύγκριση με τους υπόλοιπους kernels, καθώς και με το ολοκληρωμένο project που περιλαμβάνει και τις τρεις συναρτήσεις.

5.5.2 Kernel 2 (libxsmm_m12_n9_k20)

Στη συνέχεια εξετάζεται ο Kernel 2 (vadd) μέσω του Vitis Analyzer, με στόχο τη σύγκριση της αρχιτεκτονικής συμπεριφοράς του με αυτή του Kernel 1. Ο Kernel 2 αντιστοιχεί

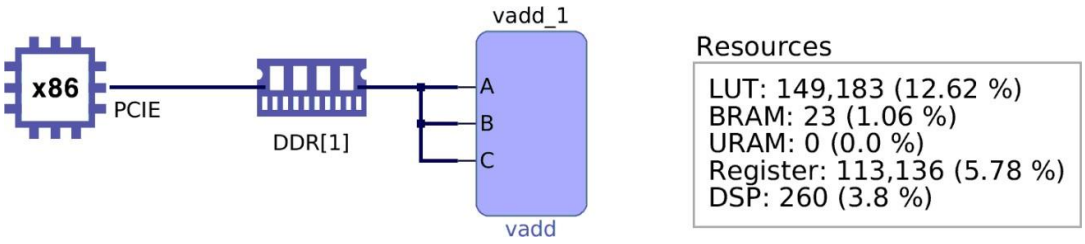
1. SLR Connectivity

	Used	Fixed	Available	Util%
SLR2 <-> SLR1	4435		17280	25.67
SLR1 -> SLR2	2428			14.05
Using TX_REG only	0	0		
Using RX_REG only	0	0		
Using Both TX_REG and RX_REG	5	0		
SLR2 -> SLR1	2007			11.61
Using TX_REG only	0	0		
Using RX_REG only	0	0		
Using Both TX_REG and RX_REG	5	0		
SLR1 <-> SLR0	1632		17280	9.44
SLR0 -> SLR1	679			3.93
Using TX_REG only	0	0		
Using RX_REG only	0	0		
Using Both TX_REG and RX_REG	5	0		
SLR1 -> SLR0	953			5.52
Using TX_REG only	0	0		
Using RX_REG only	0	0		
Using Both TX_REG and RX_REG	5	0		
Total SLLs Used	6067			

Σχήμα 5.5: Post-placement SLR connectivity για τον Kernel 1

σε απλούστερο υπολογιστικό μοτίβο, γεγονός που αναμένεται να αποτυπωθεί τόσο στη χρήση πόρων όσο και στην κατανομή της λογικής στο FPGA.

Το system diagram του kernel 2 παρουσιάζεται στο Σχήμα 5.6. Η συνολική αρχιτεκτονική παραμένει παρόμοια με αυτή του Kernel 1, με τον kernel να επικοινωνεί με τον x86 host μέσω PCIe και να χρησιμοποιεί την παγκόσμια μνήμη DDR για τη διαχείριση των δεδομένων εισόδου και εξόδου. Ωστόσο, λόγω της απλούστερης φύσης του Kernel 2, η εσωτερική λογική του accelerator είναι λιγότερο πολύπλοκη, γεγονός που αντανακλάται στη μειωμένη κατανάλωση πόρων.



Σχήμα 5.6: System diagram για τον Kernel 2 από το Vitis Analyzer

Η χρήση πόρων του accelerator για τον Kernel 2 παρουσιάζεται στα Σχήματα 5.7 και 5.8. Συγκεκριμένα, ο kernel καταναλώνει 152 171 LUTs (15.32% του user budget), 113 190 registers (5.36%), 23 BRAMs (1.20%) και 260 DSPs (3.81%), χωρίς χρήση URAMs. Σε σύγκριση με τον Kernel 1, παρατηρείται σημαντική μείωση στη χρήση LUTs και registers, ενώ η χρήση DSPs παραμένει σταθερή. Το εύρημα αυτό υποδεικνύει ότι, παρότι ο αριθμός των αριθμητικών πράξεων είναι συγκρίσιμος, ο Kernel 2 απαιτεί λιγότερη λογική ελέγχου και διασυνδέσεων, οδηγώντας σε πιο συμπαγή

υλοποίηση.

Name	LUT	LUTAsMem	REG	BRAM	URAM	DSP
Platform	188813	19398	250809	251	20	7
User Budget	993427	572442	2113671	1909	940	6833
Used Resources	152171	3957	113190	23	0	260
Unused Resources	841256	568485	2000481	1886	940	6573
vadd (1)	152171	3957	113190	23	0	260
vadd_1	152171	3957	113190	23	0	260

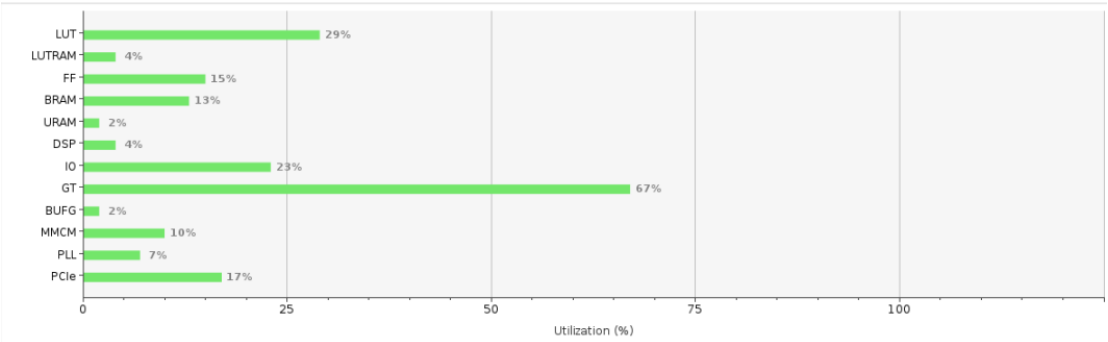
Σχήμα 5.7: Χρήση πόρων accelerator για τον Kernel 2: απόλυτες τιμές

Name	LUT	LUTAsMem	REG	BRAM	URAM	DSP
Platform	15.97%	3.28%	10.61%	11.62%	2.08%	0.10%
User Budget	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Used Resources	15.32%	0.69%	5.36%	1.20%	0.00%	3.81%
Unused Resources	84.68%	99.31%	94.64%	98.80%	100.00%	96.19%
vadd (1)	15.32%	0.69%	5.36%	1.20%	0.00%	3.81%
vadd_1	15.32%	0.69%	5.36%	1.20%	0.00%	3.81%

Σχήμα 5.8: Χρήση πόρων accelerator για τον Kernel 2: ποσοστά επί του User Budget

Η συνολική χρήση πόρων σε επίπεδο συσκευής παρουσιάζεται στο Σχήμα 5.9. Η χρήση LUT ανέρχεται περίπου στο 29%, ενώ η χρήση FF φτάνει το 15%. Η χρήση BRAM παραμένει σε παρόμοια επίπεδα με τον kernel 1 (περίπου 13%), ενώ η χρήση DSPs παραμένει χαμηλή, κοντά στο 4%. Όπως και στην προηγούμενη περίπτωση, παρατηρείται αυξημένη αξιοποίηση των GT πόρων, η οποία σχετίζεται με την υποδομή επικοινωνίας και όχι με τον ίδιο τον υπολογιστικό kernel. Σε σχέση με τον Kernel 1, η συνολική εικόνα δείχνει σαφώς μειωμένο αποτύπωμα λογικών πόρων, γεγονός που καθιστά τον Kernel 2 λιγότερο απαιτητικό ως προς τη χωρητικότητα του FPGA.

Η ανάλυση post-placement SLR connectivity, η οποία παρουσιάζεται στο Σχήμα 5.10, δείχνει ότι και ο Kernel 2 κατανέμεται σε περισσότερα από ένα SLRs. Ωστόσο, ο συνολικός αριθμός συνδέσεων μεταξύ SLR0, SLR1 και SLR2 είναι μειωμένος σε σχέση με τον Kernel 1. Η μικρότερη αυτή διασπορά της λογικής υποδηλώνει λιγότερες απαιτήσεις σε SLR-to-SLR επικοινωνία και, κατά συνέπεια, χαμηλότερη πολυπλοκότητα δρομολόγησης. Το στοιχείο αυτό είναι κρίσιμο για την κατανόηση της καλύτερης συμπεριφοράς του Kernel 2 σε σενάρια συνύπαρξης πολλαπλών kernels στο ίδιο project.



Resource	Utilization	Available	Utilization %
LUT	340984	1182240	28.84
LUTRAM	23355	591840	3.95
FF	363994	2364480	15.39
BRAM	274.50	2160	12.71
URAM	20	960	2.08
DSP	267	6840	3.90
IO	157	676	23.22
GT	16	24	66.67
BUFG	44	1800	2.44
MMCM	3	30	10.00
PLL	4	60	6.67
PCIe	1	6	16.67

Σχήμα 5.9: Συνολική χρήση πόρων συσκευής για τον Kernel 2

1. SLR Connectivity

	Used	Fixed	Available	Util%
SLR2 <-> SLR1	4437		17280	25.68
SLR1 -> SLR2	2425			14.03
Using TX_REG only	0	0		
Using RX_REG only	0	0		
Using Both TX_REG and RX_REG	7	0		
SLR2 -> SLR1	2012			11.64
Using TX_REG only	0	0		
Using RX_REG only	0	0		
Using Both TX_REG and RX_REG	7	0		
SLR1 <-> SLR0	1481		17280	8.57
SLR0 -> SLR1	624			3.61
Using TX_REG only	0	0		
Using RX_REG only	0	0		
Using Both TX_REG and RX_REG	7	0		
SLR1 -> SLR0	857			4.96
Using TX_REG only	0	0		
Using RX_REG only	0	0		
Using Both TX_REG and RX_REG	7	0		
Total SLLs Used	5918			

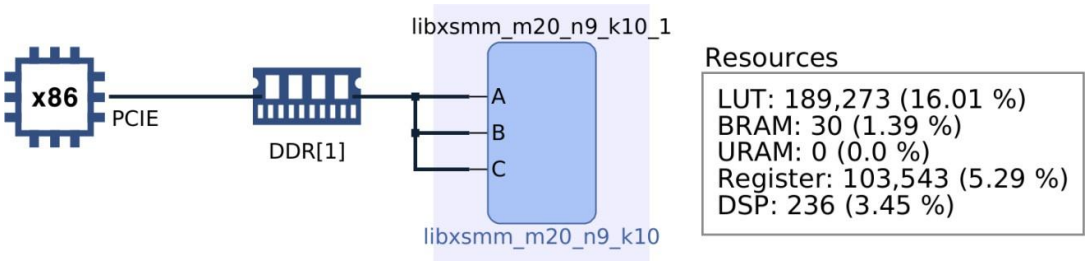
Σχήμα 5.10: Post-placement SLR connectivity για τον Kernel 2

Συνολικά, η ανάλυση του Kernel 2 αναδεικνύει μια πιο αποδοτική αρχιτεκτονική υλοποίηση σε σχέση με τον Kernel 1, με σαφώς μειωμένη χρήση λογικών πόρων και περιορισμένη SLR διασπορά. Τα χαρακτηριστικά αυτά καθιστούν τον Kernel 2 πιο φιλικό ως προς την κλιμάκωση και τη συνδυαστική υλοποίηση, στοιχείο που θα ληφθεί υπόψη στην επόμενη ενότητα, όπου εξετάζεται ο Kernel 3 και, στη συνέχεια, το ολοκληρωμένο project με τους τρεις kernels.

5.5.3 Kernel 3 (libxsmm_m20_n9_k10)

Στην παρούσα υποενότητα ολοκληρώνεται η ανάλυση των μεμονωμένων kernels μέσω του Vitis Analyzer, με την εξέταση του Kernel 3 (libxsmm_m20_n9_k10). Ο συγκεκριμένος kernel διαφοροποιείται από τους προηγούμενους ως προς τις διαστάσεις του υπολογισμού, καθώς παρουσιάζει αυξημένο μέγεθος στον δείκτη m, γεγονός που αναμένεται να επηρεάσει τόσο την κατανάλωση πόρων όσο και την κατανομή της λογικής στο FPGA.

Το system diagram του Kernel 3 παρουσιάζεται στο Σχήμα 5.11. Η συνολική αρχιτεκτονική παραμένει αντίστοιχη με εκείνη των προηγούμενων kernels, με τον accelerator να συνδέεται με τον host μέσω PCIe και να χρησιμοποιεί την παγκόσμια μνήμη DDR για την εξυπηρέτηση των εισόδων και εξόδων. Ωστόσο, η εσωτερική υλοποίηση του Kernel 3 εμφανίζει μεγαλύτερη υπολογιστική πυκνότητα σε σχέση με τον Kernel 2, γεγονός που αντανακλάται στη συνολική χρήση πόρων.



Σχήμα 5.11: System diagram για τον Kernel 3 από το Vitis Analyzer

Η χρήση πόρων του accelerator για τον Kernel 3 παρουσιάζεται στα Σχήματα 5.12 και 5.13. Ειδικότερα, ο kernel καταναλώνει 193 334 LUTs (19.48% του user budget), 103 212 registers (4.89%), 30 BRAMs (1.58%) και 236 DSPs (3.45%), χωρίς χρήση URAMs. Συγκριτικά με τον kernel 2, παρατηρείται αύξηση στη χρήση LUTs και BRAMs, η οποία συνδέεται άμεσα με το αυξημένο μέγεθος του υπολογισμού και τις επιπλέον απαιτήσεις αποθήκευσης ενδιάμεσων δεδομένων. Παράλληλα, η χρήση DSPs παραμένει σε παρόμοια επίπεδα, υποδεικνύοντας ότι και σε αυτή την περίπτωση ο kernel δεν περιορίζεται από την αριθμητική ισχύ, αλλά από τη λογική και τη μνήμη.

Name	LUT	LUTAsMem	REG	BRAM	URAM	DSP
Platform	189944	19455	252755	260	20	7
⌄ User Budget	992296	572385	2111725	1900	940	6833
Used Resources	193334	3149	103212	30	0	236
Unused Resources	798962	569236	2008513	1870	940	6597
⌄ libxsmm_m20_n9_k10 (1)	193334	3149	103212	30	0	236
libxsmm_m20_n9_k10_1	193334	3149	103212	30	0	236

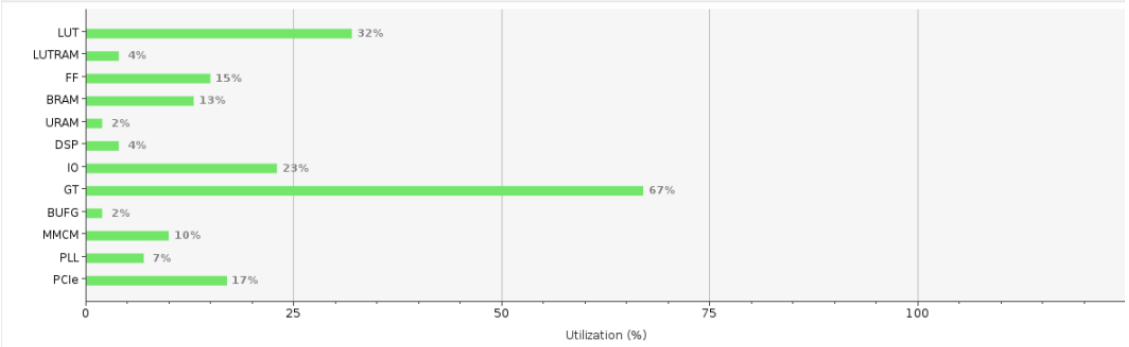
Σχήμα 5.12: Χρήση πόρων accelerator για τον Kernel 3: απόλυτες τιμές

Η συνολική χρήση πόρων της συσκευής παρουσιάζεται στο Σχήμα 5.14. Παρατηρείται ότι η χρήση LUT ανέρχεται περίπου στο 32%, ενώ η χρήση FF παραμένει κοντά στο 15%. Η χρήση BRAM κυμαίνεται γύρω στο 13%, παρουσιάζοντας ελαφρά αύξηση σε σχέση με τους προηγούμενους kernels, γεγονός που συνάδει με την αυξημένη απαίτηση για τοπική αποθήκευση. Όπως και στις προηγούμενες περιπτώσεις, η υψηλή

Name	LUT	LUTAsMem	REG	BRAM	URAM	DSP
Platform	16.07%	3.29%	10.69%	12.04%	2.08%	0.10%
User Budget	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Used Resources	19.48%	0.55%	4.89%	1.58%	0.00%	3.45%
Unused Resources	80.52%	99.45%	95.11%	98.42%	100.00%	96.55%
libxsmm_m20_n9_k10 (1)	19.48%	0.55%	4.89%	1.58%	0.00%	3.45%
libxsmm_m20_n9_k10_1	19.48%	0.55%	4.89%	1.58%	0.00%	3.45%

Σχήμα 5.13: Χρήση πόρων accelerator για τον Kernel 3: ποσοστά επί του User Budget

αξιοποίηση των GT πόρων οφείλεται στην υποδομή επικοινωνίας και όχι στον ίδιο τον υπολογιστικό πυρήνα. Σε σύγκριση με τον Kernel 1 και τον Kernel 2, ο Kernel 3 εμφανίζει ενδιάμεσο αποτύπωμα λογικών πόρων, αντανακλώντας τον πιο σύνθετο υπολογισμό σε σχέση με τον Kernel 2, αλλά χωρίς να φτάνει την πολυπλοκότητα του Kernel 1.



Resource	Utilization	Available	Utilization %
LUT	383278	1182240	32.42
LUTRAM	22604	591840	3.82
FF	355962	2364480	15.05
BRAM	291	2160	13.47
URAM	20	960	2.08
DSP	243	6840	3.55
IO	157	676	23.22
GT	16	24	66.67
BUFG	44	1800	2.44
MMCM	3	30	10.00
PLL	4	60	6.67
PCIe	1	6	16.67

Σχήμα 5.14: Συνολική χρήση πόρων συσκευής για τον Kernel 3

Η ανάλυση post-placement SLR connectivity, η οποία παρουσιάζεται στο Σχήμα 5.15, δείχνει ότι ο kernel 3 κατανέμεται επίσης σε πολλαπλά SLRs, με σημαντικό αριθμό διασυνδέσεων μεταξύ των SLR0, SLR1 και SLR2. Σε σχέση με τον Kernel 2, παρατηρείται αυξημένος αριθμός SLR-to-SLR συνδέσεων, γεγονός που υποδηλώνει μεγαλύτερη διασπορά της λογικής. Ωστόσο, η εικόνα παραμένει λιγότερο έντονη σε σύγκριση με τον Kernel 1, υποδεικνύοντας μια ενδιάμεση κατάσταση ως προς την πολυπλοκότητα δρομολόγησης και τις απαιτήσεις σε διασύνδεση.

1. SLR Connectivity

	Used	Fixed	Available	Util%
SLR2 <-> SLR1	4998		17280	28.92
SLR1 -> SLR2	2944			17.04
Using TX_REG only	0	0		
Using RX_REG only	0	0		
Using Both TX_REG and RX_REG	7	0		
SLR2 -> SLR1	2054			11.89
Using TX_REG only	0	0		
Using RX_REG only	0	0		
Using Both TX_REG and RX_REG	7	0		
SLR1 <-> SLR0	1545		17280	8.94
SLR0 -> SLR1	668			3.87
Using TX_REG only	0	0		
Using RX_REG only	0	0		
Using Both TX_REG and RX_REG	7	0		
SLR1 -> SLR0	877			5.08
Using TX_REG only	0	0		
Using RX_REG only	0	0		
Using Both TX_REG and RX_REG	7	0		
Total SLLs Used	6543			

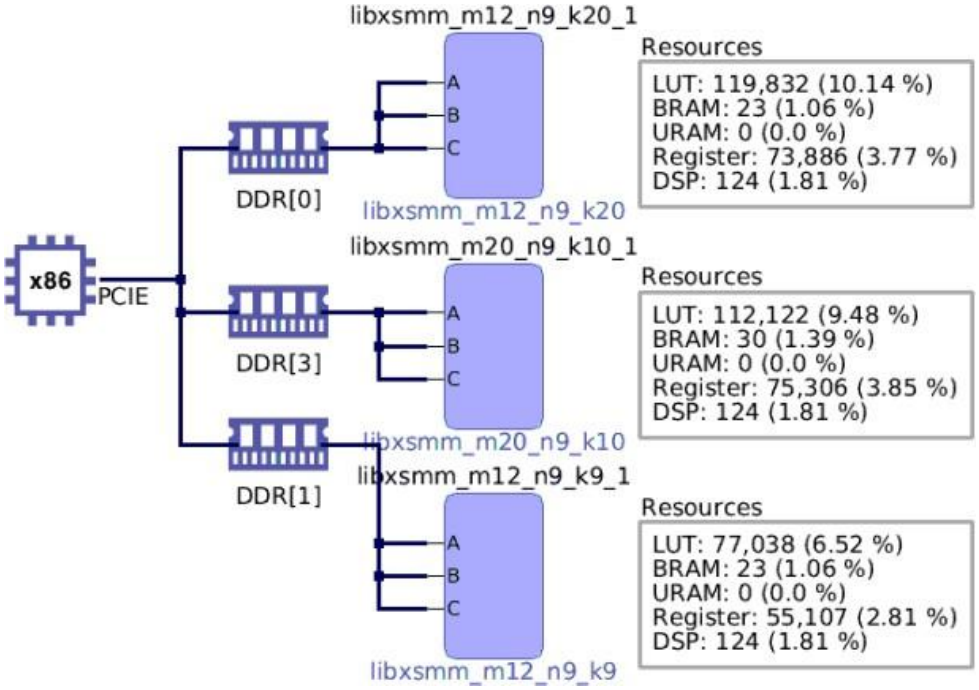
Σχήμα 5.15: Post-placement SLR connectivity για τον Kernel 3

Συνολικά, η ανάλυση του Kernel 3 αναδεικνύει έναν συμβιβασμό μεταξύ υπολογιστικής πυκνότητας και αποτυπώματος πόρων. Σε σύγκριση με τον Kernel 2, ο Kernel 3 απαιτεί περισσότερους λογικούς και μνημονικούς πόρους, ενώ σε σχέση με τον Kernel 1 εμφανίζει πιο περιορισμένη χρήση LUTs και λιγότερο εκτεταμένη SLR διασπορά. Τα χαρακτηριστικά αυτά καθιστούν τον Kernel 3 ενδεικτικό παράδειγμα του τρόπου με τον οποίο οι διαστάσεις του υπολογισμού επηρεάζουν την αρχιτεκτονική συμπεριφορά στο FPGA, και παρέχουν το κατάλληλο υπόβαθρο για την επόμενη ενότητα, όπου εξετάζεται το συνολικό project με την ταυτόχρονη ενσωμάτωση και των τριών kernels.

5.5.4 Συνολικό project με τρεις kernels

Αφού αναλύθηκαν οι μεμονωμένες υλοποιήσεις των τριών kernels, στη συνέχεια εξετάζεται το συνολικό Vitis project στο οποίο οι kernels libxsmm_m12_n9_k20, vadd και libxsmm_m20_n9_k10 συνυπάρχουν και εκτελούνται στο ίδιο FPGA. Η ανάλυση αυτή είναι ιδιαίτερα κρίσιμη, καθώς αποτυπώνει τη συμπεριφορά του συστήματος σε ένα πιο ρεαλιστικό σενάριο, όπου πολλαπλοί accelerators μοιράζονται κοινούς πόρους και υποδομές επικοινωνίας.

Το system diagram του συνολικού project παρουσιάζεται στο Σχήμα 5.16. Παρατηρείται ότι κάθε kernel συνδέεται με ξεχωριστό DDR κανάλι, επιλογή που στοχεύει στη μείωση της συμφόρησης μνήμης και στη βελτίωση της ταυτόχρονης εξυπηρέτησης αιτημάτων. Παρότι η διάταξη αυτή αυξάνει την πολυπλοκότητα του σχεδιασμού, επιτρέπει καλύτερη απομόνωση των ροών δεδομένων σε σχέση με την περίπτωση όπου όλοι οι kernels μοιράζονται το ίδιο κανάλι μνήμης. Ωστόσο, η κοινή διασύνδεση με τον host μέσω PCIe παραμένει κοινό σημείο συμφόρησης, στοιχείο που πρέπει να ληφθεί υπόψη κατά την ερμηνεία των χρονικών αποτελεσμάτων.



Σχήμα 5.16: System diagram για το project με τους 3 Kernels από το Vitis Analyzer

Η χρήση πόρων του συνολικού accelerator παρουσιάζεται στα Σχήματα 5.17 και 5.18. Σε επίπεδο user budget, το project καταναλώνει συνολικά περίπου 33.7% των LUTs, 10.1% των registers, 4.3% των BRAMs και 5.5% των DSPs. Σε σύγκριση με τις μεμονωμένες υλοποιήσεις, παρατηρείται ότι η συνολική χρήση πόρων δεν είναι απλώς το άθροισμα των τριών kernels, αλλά επηρεάζεται και από πρόσθετη λογική που σχετίζεται με τη διαχείριση του συστήματος και των διασυνδέσεων. Ιδιαίτερα, η αύξηση στη χρήση LUTs υποδηλώνει επιπλέον κόστος ελέγχου και συντονισμού, το οποίο δεν εμφανίζεται όταν οι kernels εξετάζονται απομονωμένα.

Name	LUT	LUTAsMem	REG	BRAM	URAM	DSP
Platform	253452	27651	336720	403	20	13
⌄ User Budget	928788	564189	2027760	1757	940	6827
Used Resources	313138	7280	204308	76	0	372
Unused Resources	615650	556909	1823452	1681	940	6455
⌄ libxsmm_m12_n9_k20 (1)	121211	2364	73780	23	0	124
libxsmm_m12_n9_k20_1	121211	2364	73780	23	0	124
⌄ libxsmm_m12_n9_k9 (1)	78376	2366	55219	23	0	124
libxsmm_m12_n9_k9_1	78376	2366	55219	23	0	124
⌄ libxsmm_m20_n9_k10 (1)	113551	2550	75309	30	0	124
libxsmm_m20_n9_k10_1	113551	2550	75309	30	0	124

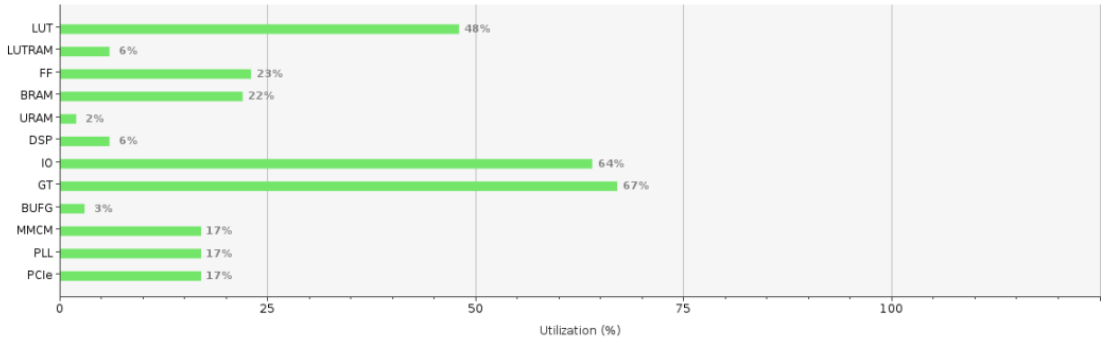
Σχήμα 5.17: Χρήση πόρων accelerator για το project με τους 3 Kernels: απόλυτες τιμές

Η συνολική χρήση πόρων σε επίπεδο συσκευής παρουσιάζεται στο Σχήμα 5.19. Η χρήση LUT προσεγγίζει το 48%, ενώ η χρήση FF φτάνει περίπου το 23%. Η χρήση

Name	LUT	LUTAsMem	REG	BRAM	URAM	DSP
Platform	21.44%	4.67%	14.24%	18.66%	2.08%	0.19%
User Budget	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Used Resources	33.71%	1.29%	10.08%	4.33%	0.00%	5.45%
Unused Resources	66.29%	98.71%	89.92%	95.67%	100.00%	94.55%
libxsmm_m12_n9_k20 (1)	13.05%	0.42%	3.64%	1.31%	0.00%	1.82%
libxsmm_m12_n9_k20_1	13.05%	0.42%	3.64%	1.31%	0.00%	1.82%
libxsmm_m12_n9_k9 (1)	8.44%	0.42%	2.72%	1.31%	0.00%	1.82%
libxsmm_m12_n9_k9_1	8.44%	0.42%	2.72%	1.31%	0.00%	1.82%
libxsmm_m20_n9_k10 (1)	12.23%	0.45%	3.71%	1.71%	0.00%	1.82%
libxsmm_m20_n9_k10_1	12.23%	0.45%	3.71%	1.71%	0.00%	1.82%

Σχήμα 5.18: Χρήση πόρων accelerator για το project με τους 3 Kernels: ποσοστά επί του User Budget

BRAM ανέρχεται στο 22%, εμφανίζοντας σαφή αύξηση σε σχέση με τις μεμονωμένες υλοποιήσεις, γεγονός που αντανακλά τις ταυτόχρονες απαιτήσεις μνήμης των τριών kernels. Παράλληλα, η χρήση DSPs παραμένει σχετικά χαμηλή (περίπου 6%), επιβεβαιώνοντας ότι ακόμη και σε επίπεδο συστήματος ο περιορισμός δεν προέρχεται από την αριθμητική ισχύ, αλλά από τη λογική και τη μνήμη. Ιδιαίτερη σημασία έχει και η υψηλή αξιοποίηση των GT και IO πόρων, η οποία συνδέεται με τη διαχείριση πολλαπλών καναλιών μνήμης και την επικοινωνία με τον host.



Resource	Utilization	Available	Utilization %
LUT	566590	1182240	47.93
LUTRAM	34931	591840	5.90
FF	541021	2364480	22.88
BRAM	480	2160	22.22
URAM	20	960	2.08
DSP	385	6840	5.63
IO	435	676	64.35
GT	16	24	66.67
BUFG	62	1800	3.44
MMCM	5	30	16.67
PLL	10	60	16.67
PCIe	1	6	16.67

Σχήμα 5.19: Συνολική χρήση πόρων συσκευής για το project με τους 3 Kernels

Η ανάλυση post-placement SLR connectivity, η οποία παρουσιάζεται στο Σχήμα 5.20, αποκαλύπτει μια πιο έντονη και σύνθετη κατανομή της λογικής σε σχέση με τις μεμονωμένες περιπτώσεις. Παρατηρείται σημαντικός αριθμός συνδέσεων μεταξύ όλων των SLRs, με αυξημένες απαιτήσεις σε SLR-to-SLR επικοινωνία. Αν και η συνολική διασπορά της λογικής είναι μικρότερη από το απλό άθροισμα των αντίστοιχων διασπορών των τριών kernels, το πλήθος των διασυνδέσεων παραμένει υψηλό, γεγονός που αυξάνει την πολυπλοκότητα της δρομολόγησης και περιορίζει τα περιθώρια περαιτέρω κλιμάκωσης.

Το εύρημα αυτό συνδέεται άμεσα με τα χρονικά αποτελέσματα που παρουσιάστηκαν προηγουμένως, όπου παρατηρήθηκε ότι η ταυτόχρονη εκτέλεση πολλαπλών kernels δεν οδηγεί σε ανάλογη βελτίωση της συνολικής απόδοσης.

1. SLR Connectivity

	Used	Fixed	Available	Util%
SLR2 <-> SLR1	2757		17280	15.95
SLR1 -> SLR2	1424			8.24
Using TX_REG only	4	0		
Using RX_REG only	4	0		
Using Both TX_REG and RX_REG	4	0		
SLR2 -> SLR1	1333			7.71
Using TX_REG only	4	0		
Using RX_REG only	4	0		
Using Both TX_REG and RX_REG	4	0		
SLR1 <-> SLR0	1676		17280	9.70
SLR0 -> SLR1	683			3.95
Using TX_REG only	0	0		
Using RX_REG only	0	0		
Using Both TX_REG and RX_REG	0	0		
SLR1 -> SLR0	993			5.75
Using TX_REG only	0	0		
Using RX_REG only	0	0		
Using Both TX_REG and RX_REG	0	0		
Total SLLs Used	4433			

Σχήμα 5.20: Post-placement SLR connectivity για το project με τους 3 Kernels

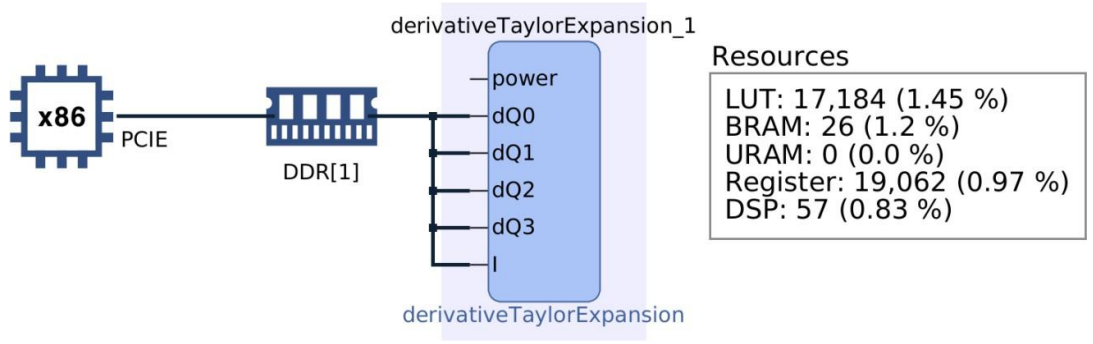
Συνοψίζοντας, η ανάλυση του συνολικού project μέσω του Vitis Analyzer καταδεικνύει ότι η συνύπαρξη πολλαπλών kernels στο ίδιο FPGA εισάγει πρόσθετους περιορισμούς που δεν είναι εμφανείς στις μεμονωμένες υλοποιήσεις. Παρότι η χρήση αριθμητικών πόρων παραμένει χαμηλή, η αυξημένη κατανάλωση λογικών και μνημονικών πόρων, σε συνδυασμό με την εκτεταμένη SLR διασπορά, περιορίζει τη δυνατότητα περαιτέρω επέκτασης του σχεδιασμού. Τα αποτελέσματα αυτά επιβεβαιώνουν ότι, για την επίτευξη ουσιαστικής επιτάχυνσης σε επίπεδο συστήματος, απαιτείται όχι μόνο βελτιστοποίηση των μεμονωμένων kernels, αλλά και προσεκτικός σχεδιασμός της συνολικής αρχιτεκτονικής και της κατανομής πόρων.

5.5.5 Taylor kernel (derivativeTaylorExpansion)

Ως τελευταίο στάδιο της ανάλυσης μέσω του Vitis Analyzer, εξετάζεται ο Taylor kernel (derivativeTaylorExpansion), ο οποίος διαφέρει ουσιαστικά από τις προηγούμενες υλοποιήσεις, καθώς δεν αποτελεί μεμονωμένο μικροπυρήνα τύπου LIBXSMM, αλλά συνθέτει πολλαπλές υπολογιστικές πράξεις που σχετίζονται με την ανάπτυξη Taylor. Για τον λόγο αυτό, ο kernel αυτός προσεγγίζει περισσότερο ένα ρεαλιστικό υπολογιστικό τμήμα του SeisSol και λειτουργεί ως αντιπροσωπευτικό παράδειγμα σύνθετου kernel σε επίπεδο εφαρμογής.

Το system diagram του Taylor kernel παρουσιάζεται στο Σχήμα 5.21. Παρατηρείται ότι ο kernel διαχειρίζεται πολλαπλές εισόδους και εξόδους (power, dQ0–dQ3, I), γεγονός

που αυξάνει τη λειτουργική του πολυπλοκότητα σε σχέση με τους προηγούμενους πυρήνες. Παρά τη μεγαλύτερη λογική πολυπλοκότητα, η συνολική αρχιτεκτονική παραμένει συνεπής με τις προηγούμενες περιπτώσεις, βασιζόμενη στη μεταφορά δεδομένων μέσω PCIe και DDR.



Σχήμα 5.21: System diagram για τον Kernel Taylor από το Vitis Analyzer

Η χρήση πόρων του accelerator για τον Taylor kernel παρουσιάζεται στα Σχήματα 5.22 και 5.23. Συγκεκριμένα, ο kernel καταναλώνει 19 621 LUTs (1.98% του user budget), 19 490 registers (0.92%), 26 BRAMs (1.37%) και 57 DSPs (0.83%), χωρίς χρήση URAMs. Τα ποσοστά αυτά είναι σημαντικά χαμηλότερα από εκείνα που παρατηρήθηκαν στους προηγούμενους kernels, γεγονός που υποδηλώνει ότι, παρά τη λειτουργική πολυπλοκότητα του Taylor kernel, η υλοποίηση παραμένει ιδιαίτερα ελαφριά από πλευράς πόρων. Το εύρημα αυτό σχετίζεται με τη φύση των υπολογισμών, οι οποίοι βασίζονται περισσότερο σε συνδυασμούς απλών αριθμητικών πράξεων και λιγότερο σε βαριά παραλληλοποιημένη αριθμητική.

Name	LUT	LUTAsMem	REG	BRAM	URAM	DSP
Platform	193079	19936	254446	269	20	7
User Budget	989161	571904	2110034	1891	940	6833
Used Resources	19621	4723	19490	26	0	57
Unused Resources	969540	567181	2090544	1865	940	6776
derivativeTaylorExpansion (1)	19621	4723	19490	26	0	57
derivativeTaylorExpansion_1	19621	4723	19490	26	0	57

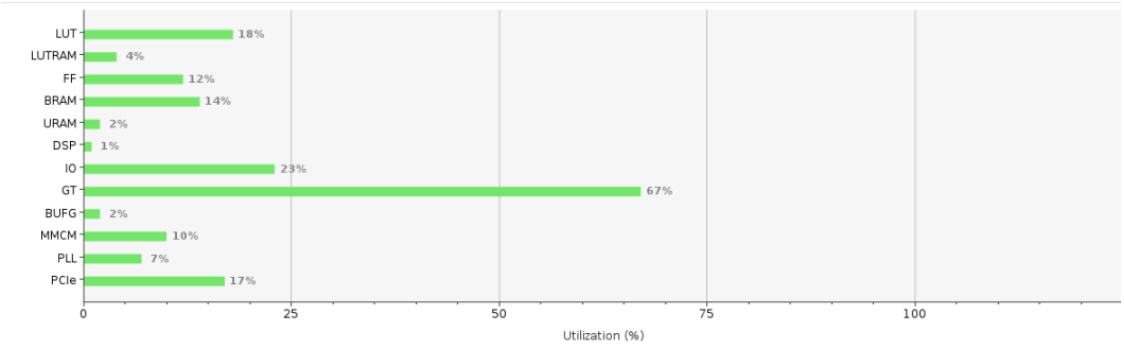
Σχήμα 5.22: Χρήση πόρων accelerator για τον Kernel Taylor: απόλυτες τιμές

Name	LUT	LUTAsMem	REG	BRAM	URAM	DSP
Platform	16.33%	3.37%	10.76%	12.45%	2.08%	0.10%
User Budget	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Used Resources	1.98%	0.83%	0.92%	1.37%	0.00%	0.83%
Unused Resources	98.02%	99.17%	99.08%	98.63%	100.00%	99.17%
derivativeTaylorExpansion (1)	1.98%	0.83%	0.92%	1.37%	0.00%	0.83%
derivativeTaylorExpansion_1	1.98%	0.83%	0.92%	1.37%	0.00%	0.83%

Σχήμα 5.23: Χρήση πόρων accelerator για τον Kernel Taylor: ποσοστά επί του User Budget

Η συνολική χρήση πόρων της συσκευής παρουσιάζεται στο Σχήμα 5.24. Σε επίπεδο FPGA, η χρήση LUT ανέρχεται περίπου στο 18%, ενώ η χρήση FF φτάνει το 12%.

Η χρήση BRAM κυμαίνεται γύρω στο 14%, παραμένοντας συγκρίσιμη με εκείνη των προηγούμενων υλοποιήσεων, ενώ η χρήση DSPs είναι ιδιαίτερα χαμηλή (περίπου 1%). ΄Οπως και στις προηγούμενες περιπτώσεις, η αυξημένη αξιοποίηση των GT και PCIe-σχετικών πόρων οφείλεται στην υποδομή επικοινωνίας και όχι στον ίδιο τον kernel. Συνολικά, το αποτύπωμα πόρων του Taylor kernel είναι σαφώς μικρότερο από εκείνο των micro- kernels, γεγονός που τον καθιστά κατάλληλο για συνύπαρξη με άλλα υπολογιστικά τμήματα στο ίδιο FPGA.



Resource	Utilization	Available	Utilization %
LUT	212700	1182240	17.99
LUTRAM	24659	591840	4.17
FF	273931	2364480	11.59
BRAM	295	2160	13.66
URAM	20	960	2.08
DSP	64	6840	0.94
IO	157	676	23.22
GT	16	24	66.67
BUFG	44	1800	2.44
MMCM	3	30	10.00
PLL	4	60	6.67
PCIe	1	6	16.67

Σχήμα 5.24: Συνολική χρήση πόρων συσκευής για τον Kernel Taylor

Η ανάλυση post-placement SLR connectivity, η οποία παρουσιάζεται στο Σχήμα 5.25, δείχνει ότι ο Taylor kernel κατανέμεται σε πολλαπλά SLRs, ωστόσο με αισθητά μικρότερο αριθμό SLR-to-SLR συνδέσεων σε σχέση με το project των τριών kernels. Η πιο περιορισμένη αυτή διασπορά της λογικής μειώνει την πολυπλοκότητα δρομολόγησης και περιορίζει τις απαιτήσεις σε διασύνδεση μεταξύ SLRs, στοιχείο που ευνοεί τη σταθερότητα και την προβλεψιμότητα της υλοποίησης. Το εύρημα αυτό είναι ιδιαίτερα σημαντικό, καθώς υποδεικνύει ότι σύνθετοι kernels σε επίπεδο εφαρμογής δεν είναι απαραίτητα πιο απαιτητικοί από άποψη χωρικής κατανομής λογικής.

1. SLR Connectivity

	Used	Fixed	Available	Util%
SLR2 <-> SLR1	2610		17280	15.10
SLR1 -> SLR2	1337			7.74
Using TX_REG only	3	0		
Using RX_REG only	3	0		
Using Both TX_REG and RX_REG	3	0		
SLR2 -> SLR1	1273			7.37
Using TX_REG only	3	0		
Using RX_REG only	3	0		
Using Both TX_REG and RX_REG	3	0		
SLR1 <-> SLR0	1468		17280	8.50
SLR0 -> SLR1	617			3.57
Using TX_REG only	0	0		
Using RX_REG only	0	0		
Using Both TX_REG and RX_REG	0	0		
SLR1 -> SLR0	851			4.92
Using TX_REG only	0	0		
Using RX_REG only	0	0		
Using Both TX_REG and RX_REG	0	0		
Total SLLs Used	4078			

Σχήμα 5.25: Post-placement SLR connectivity για τον Kernel Taylor

Συνοψίζοντας, η ανάλυση του Taylor kernel μέσω του Vitis Analyzer αναδεικνύει μια υλοποίηση με εξαιρετικά χαμηλή κατανάλωση πόρων και περιορισμένη SLR διασπορά, παρά τη λειτουργική της πολυπλοκότητα. Σε αντίθεση με τα LIBXSMM micro-kernels και το συνολικό project με τρεις kernels, ο Taylor kernel δεν επιβαρύνει σημαντικά το FPGA και παρουσιάζει πιο ευνοϊκά χαρακτηριστικά ως προς την κλιμάκωση. Το αποτέλεσμα αυτό υποδηλώνει ότι η μεταφορά πιο σύνθετων, αλλά λιγότερο αριθμητικά εντατικών υπολογιστικών τμημάτων στο FPGA μπορεί να αποτελεί πιο βιώσιμη προσέγγιση για μελλοντική επιτάχυνση του SeisSol.

Κεφάλαιο 6

Συμπεράσματα και Μελλοντική Εργασία

6.1 Συμπεράσματα

Η παρούσα εργασία διερεύνησε τη δυνατότητα επιτάχυνσης επιλεγμένων αριθμητικών πυρήνων της βιβλιοθήκης LIBXSMM, καθώς και ενός πυρήνα υπολογισμού ανάπτυξης Taylor, μέσω υλοποίησής τους σε FPGA με τη χρήση εργαλείων υψηλού επιπέδου σύνθεσης. Παρά την επιτυχή μεταφορά των kernels και την επίτευξη χρονικά αποδοτικών hardware υλοποιήσεων, τα πειραματικά αποτελέσματα έδειξαν ότι οι τελικές FPGA εκδοχές δεν κατάφεραν να υπερβούν τις αντίστοιχες υλοποιήσεις λογισμικού σε CPU.

Το αποτέλεσμα αυτό ερμηνεύεται ως άμεση συνέπεια της φύσης των συγκεκριμένων kernels. Οι συναρτήσεις λογισμικού είναι εξαιρετικά μικρού μεγέθους, με πολύ περιορισμένο αριθμό αριθμητικών πράξεων, και έχουν ήδη βελτιστοποιηθεί σε βάθος για σύγχρονες αρχιτεκτονικές CPU. Ως εκ τούτου, ο διαθέσιμος χώρος για περαιτέρω επιτάχυνση μέσω FPGA είναι εγγενώς περιορισμένος, ενώ το κόστος εκκίνησης kernel και μεταφοράς δεδομένων καθίσταται συγκρίσιμο με τον ίδιο τον χρόνο υπολογισμού.

Σε επίπεδο αρχιτεκτονικής, οι τελικές υλοποιήσεις των kernels μπορούν να χαρακτηριστούν ως όσο το δυνατόν βέλτιστες, διατηρώντας παράλληλα τη λογική και τη μαθηματική δομή των αρχικών υπολογισμών. Η μεταφορά των δεδομένων σε τοπικά buffers, η απομάκρυνση εξαρτήσεων μεταξύ επαναλήψεων, το επιλεκτικό unroll μικρών βρόχων και η διοχέτευση των μεγαλύτερων βρόχων (pipelining) επέτρεψαν την προσέγγιση του πρακτικού ορίου απόδοσης που μπορεί να επιτευχθεί μέσω HLS.

Οι διαφορές στους χρόνους εκτέλεσης μεταξύ των επιμέρους kernels αποδείχθηκαν συνεπείς με το πλήθος των επαναλήψεων, το διαθέσιμο επίπεδο παραλληλισμού και το συνολικό αριθμό αριθμητικών πράξεων. Ιδιαίτερα για τον πυρήνα Taylor, η πολύ μικρή υπολογιστική ένταση καθιστά την απόδοσή του κυρίως εξαρτώμενη από το κόστος ελέγχου και επανάληψης, γεγονός που περιορίζει περαιτέρω το όφελος από επιθετικές αρχιτεκτονικές βελτιστοποιήσεις.

Επιπλέον, η ανάλυση της χρήσης πόρων (resource utilization) του FPGA καταδεικνύει ότι οι τελικές υλοποιήσεις δεν περιορίστηκαν από τη διαθεσιμότητα υπολογιστικών ή

αποθηκευτικών πόρων. Και στις δύο περιπτώσεις (project με τρεις LIBXSMM kernels και πυρήνας Taylor), η αξιοποίηση των DSP, LUT και FF παρέμεινε σε σχετικά χαμηλά επίπεδα σε σχέση με τις δυνατότητες της πλατφόρμας Alveo U200.

Το εύρημα αυτό υποδηλώνει ότι η απόδοση των kernels δεν περιορίζεται από έλλειψη παραλληλισμού σε επίπεδο υλικού ή από κορεσμό των διαθέσιμων πόρων, αλλά από τη φύση των ίδιων των υπολογισμών. Συγκεκριμένα, το μικρό μέγεθος των πινάκων, ο περιορισμένος αριθμός επαναλήψεων και η διατήρηση της αρχικής λογικής των υπολογισμών περιορίζουν τον διαθέσιμο παραλληλισμό που μπορεί να αξιοποιηθεί από το HLS.

Παρά το γεγονός ότι η χαμηλή χρήση πόρων θα επέτρεπε θεωρητικά την περαιτέρω αύξηση του βαθμού παραλληλισμού (π. χ. μέσω επιθετικότερου unrolling ή πολλαπλών compute units), τέτοιες επιλογές θα οδηγούσαν είτε σε παραβιάσεις χρονισμού είτε σε σημαντική απόκλιση από τη λογική και τη δομή των αρχικών αλγορίθμων. Ως εκ τούτου, οι τελικές υλοποιήσεις αντιπροσωπεύουν έναν ισορροπημένο συμβιβασμό μεταξύ απόδοσης, χρονισμού και αρχιτεκτονικής καθαρότητας.

Συνολικά, η εργασία αναδεικνύει ότι τα FPGA αποτελούν ιδιαίτερα ισχυρό εργαλείο επιτάχυνσης όταν το πρόβλημα διαθέτει επαρκή αριθμητική ένταση και παραλληλισμό, ωστόσο δεν είναι πανάκεια. Σε περιπτώσεις πολύ μικρών και ήδη άριστα βελτιστοποιημένων kernels, η FPGA υλοποίηση λειτουργεί περισσότερο ως επιβεβαίωση της αρχιτεκτονικής αποδοτικότητας και της ορθής μοντελοποίησης μέσω HLS, παρά ως μέσο επίτευξης θεαματικής επιτάχυνσης.

6.2 Προτάσεις για Μελλοντικές Βελτιστοποιήσεις

Μελλοντική εργασία θα μπορούσε να επικεντρωθεί σε σενάρια όπου τα πλεονεκτήματα των FPGA μπορούν να αξιοποιηθούν πληρέστερα. Ενδεικτικά:

- Χαρτογράφηση μεγαλύτερων υπολογιστικών kernels ή ολόκληρων υπό-ρουτινών, ώστε να αυξηθεί η αριθμητική ένταση και να αποσβεστεί το κόστος επικοινωνίας.
- Συνδυασμός πολλαπλών kernels σε ενιαία dataflow αρχιτεκτονική, με στόχο την ελαχιστοποίηση των μεταφορών δεδομένων μέσω host.
- Ενσωμάτωση των kernels σε πλήρη ροή εφαρμογής (πχ. ολόκληρο το SeisSol), ώστε η επιτάχυνση να αξιολογηθεί σε ρεαλιστικό, end-to-end σενάριο.

Παράρτημα Α′

Ακρωνύμια και συντομογραφίες

ADER Arbitrary High Order DERivatives

ASAGI Adaptive Seismic Data Access and Grid Interface

AVX2 Advanced Vector Extensions 2

AXI Advanced eXtensible Interface

CMake Cross Platform Make

CPU Central Processing Unit

DDR Double Data Rate

DG Discontinuous Galerkin

FF Flip-Flop

FMA Fused Multiply-Add

FPGA Field-Programmable Gate Array

GCC GNU Compiler Collection

Gmsh A three-dimensional finite element mesh generator

GPU Graphics Processing Unit

HBM High Bandwidth Memory

HDF5 Hierarchical Data Format version 5

HDL Hardware Description Language

HLS High Level Synthesis

HPC High Performance Computing

II Initiation Interval

LIBXSMM Library for small matrix multiplications

LUT Look-Up Table

MPI Message Passing Interface

NUMA Non-Uniform Memory Access

OMP Open Multi-Processing (OpenMP)

OpenMP Open Multi-Processing

ParMETIS Parallel Graph Partitioning and Fill-reducing Matrix Ordering Library

PCIe Peripheral Component Interconnect Express

PUMGen Partitioned Unstructured Mesh Generator

PSpaMM Parallel Small Matrix Multiplication library

RAM Random Access Memory

RAW Read-After-Write

RTL Register Transfer Level

SLR Standard Logic Region

SoC System on Chip

SDRAM Synchronous Dynamic Random-Access Memory

TPV Test Problem Version

WSL Windows Subsystem for Linux

XRT Xilinx Runtime

YATeTo Yet Another Tensor Toolbox

Bibliography

[1] A. Heinecke, A. Breuer, A. Gabriel, C. Pelties, S. Rettenberger, M. Bader, and S. Wenk, "Optimized kernels for large scale earthquake simulations with SeisSol, an unstructured ADER-DG code," in *Proceedings of the International Conference on High Performance Computing (ISC High Performance)*, 2014.

[2] J. M. Rodríguez-Borbón, A. Kalantar, S. S. R. K. C. Yamijala, M. B. Oviedo, W. Najjar, and B. M. Wong, "Field programmable gate arrays for enhancing the speed and energy efficiency of quantum dynamics simulations," *Journal of Chemical Theory and Computation*, vol. 16, no. 4, pp. 2085–2098, 2020.

[3] M. Käser, C. Castro, V. Hermann, and C. Pelties, "Seissol – a software for seismic wave propagation simulations," in *High Performance Computing in Science and Engineering. Proceedings of the International Conference, Garching/Munich 2009*. Springer Berlin Heidelberg, 2010, pp. 281–292.

[4] R. Dorozhinskii and M. Bader, "Seissol on distributed multi-gpu systems: Cuda code generation for the modal discontinuous galerkin method," in *Proceedings of the International Conference on High Performance Computing in Asia-Paciftc Region (HPC Asia 2021)*. Association for Computing Machinery, 2021, pp. 69–82.

[5] P. García-Risueño and P. E. Ibáñez, "A review of high performance computing foundations for scientists," *International Journal of Modern Physics C*, vol. 23, no. 7, pp. 1–33, 2012.

[6] P. O. A. Navaux, A. F. Lorenzon, and M. da Silva Serpa, "Challenges in high-performance computing," *Journal of the Brazilian Computer Society*, vol. 29, no. 1, pp. 51–62, 2023. [Online]. Available: https://journals-sol.sbc.org.br/index.php/jbcs/article/view/2219

[7] J. A. Dodd, "The application of high performance computing in rock art documentation and research," *Adoranten*, vol. 2018, pp. 92–104, 2018. [Online]. Available: http://www.rockartscandinavia.com/images/articles/a18dodd.pdf

[8] R. Macedo, J. Paulo, J. Pereira, and A. Bessani, "A survey and classification of software-defined storage systems," *ACM Computing Surveys*, vol. 53, no. 3, pp. 48:1–48:36, 2020.

[9] C. Carugno, J. Muff, M. P. Johansson, S. Karlsson, and A. Lanzanova, "Nordiquest: the nordic-estonian quantum computing e-infrastructure

quest," *arXiv preprint arXiv:2406.02216*, 2024. [Online]. Available: https://arxiv.org/abs/2406.02216

[10] A. J. S. Tipu, P. O. Conbhuí, and E. Howley, "Seismic data i/o and sorting optimization in hpc through anns prediction-based auto-tuning for exseisdat," *Neural Computing and Applications*, vol. 35, no. 8, pp. 5855–5888, 2023. [Online]. Available: https://link.springer.com/article/10.1007/s00521-022-07991-y

[11] S. V. Adams, R. W. Ford, M. Hambley, J. M. Hobson, I. Kavcic, C. M. Maynard, T. Melvin, E. Müller, S. Mullerworth, A. R. Porter, M. Rezney, B. J. Shipway, and R. Wong, "Lfric: Meeting the challenges of scalability and performance portability in weather and climate models," *Journal of Parallel and Distributed Computing*, vol. 132, pp. 383–396, 2019.

[12] D. Degen, D. C. Voulliéme, S. Buiter, H.-J. H. Franssen, H. Vereecken, A. González-Nicolás, and F. Wellmann, "Perspectives of physics-based machine learning strategies for geoscientific applications governed by partial differential equations," *Geoscientiftc Model Development*, vol. 16, pp. 7375–7409, 2023.

[13] F. Lehmann, F. Gatti, M. Bertin, and D. Clouteau, "Synthetic ground motions in heterogeneous geologies from various sources: the hemew^{S}-3d database," *Earth System Science Data*, vol. 16, pp. 3949–3972, 2024.

[14] S. Rettenberger and M. Bader, "Optimizing i/o for petascale seismic simulations on unstructured meshes," in *2015 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 2015, pp. 702–709. [Online]. Available: https://doi.org/10.1109/CLUSTER.2015.51

[15] I. Corporation. (2024) Intel vtune profiler – performance analysis tool for hpc and ai applications. Accessed: 2025-11-09. [Online]. Available: https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler.html

[16] V. Ravi, S. Sridharan, and R. Gopalakrishnan, "Performance bottleneck analysis of scientific applications using intel vtune profiler," in *Proceedings of the 2020 IEEE International Conference on Computing, Power and Communication Technologies (ICPCT)*. IEEE, 2020, pp. 1–6.

[17] J. Ruiz-Rosero, G. Ramirez-Gonzalez, and R. Khanna, "Field programmable gate array applications — a scientometric review," *Computation*, vol. 7, no. 4, p. 63, 2019. [Online]. Available: https://doi.org/10.3390/computation7040063

[18] A. Guerrieri, A. Upegui, and L. Gantel, "Applications enabled by fpga-based technology," *Electronics*, vol. 12, no. 15, p. 3302, 2023. [Online]. Available: https://doi.org/10.3390/electronics12153302

[19] K. Yang, H. Liu, Y. Zhao, and T. Deng, "A new design approach of hardware implementation through natural language entry," *IET Collaborative Intelligent Manufacturing*, vol. 5, no. 4, p. e12087, 2023. [Online]. Available: https://doi.org/10.1049/cim2.12087

[20] Z.-X. Di, R. Tao, J. Mai, L. Chen, and Y. Lin, "Leaps: Topological-layout-adaptable multi-die fpga placement for super long line minimization," *IEEE*

Transactions on Circuits and Systems I: Regular Papers, 2023. [Online]. Available: https://doi.org/10.1109/TCSI.2023.3340554

[21] AMD Xilinx, *Vitis Unifted Software Platform Documentation (UG1416)*, AMD Xilinx, 2024. [Online]. Available: https://docs.xilinx.com/r/en-US/ug1416-vitis-documentation

[22] ——, *Vitis High-Level Synthesis (HLS) User Guide (UG1399)*, AMD Xilinx, 2024. [Online]. Available: https://docs.xilinx.com/r/en-US/ug1399-vitis-hls

[23] SeisSol Development Team, "A first example — SeisSol documentation," https://seissol.readthedocs.io/en/latest/a-first-example.html, 2024, accessed: 2025-11-09.

[24] Xilinx, "Alveo u200 data center accelerator card," https://www.xilinx.com/applications/data-center/alveo.html, 2021.

[25] ——, "Alveo u200 and u250 accelerator cards — product brief," https://www.xilinx.com/support/documentation/data-sheets/ds962-u200-u250.pdf, 2020.

[26] A. Kapetanakis, A. Ferikoglou, G. Anagnostopoulos, and S. Xydis, "Dataflow optimized reconfigurable acceleration for fem-based cfd simulations," *arXiv preprint arXiv:2411.16245*, 2025. [Online]. Available: https://arxiv.org/abs/2411.16245

[27] *Vitis High-Level Synthesis User Guide*, v2022.2 ed., AMD Xilinx, 2022, accessed 2025. [Online]. Available: https://docs.amd.com/r/2022.2-English/ug1399-vitis-hls/HLS-Pragmas